



INSTITUTO POLITÉCNICO
NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Trabajo Terminal

**“Aplicación Web para la práctica de escritura a mano y
detección automática de errores ortográficos con IA en tercer y
cuarto grado de educación primaria.”**

Presentan

Ayala Chacón David

Barragán Osorio Diego José María

Giles Macias Alexis

Director

Enrique Torres González

CDMX a 5 de mayo de 2026

Resumen

Escribir de forma legible y ortográficamente correcta es una habilidad fundamental para los alumnos de educación primaria y en general para cualquier estudiante. Como pilar del desarrollo escolar debe ser reforzado de manera continua para evitar errores y fortalecer conceptos que debilitan sus habilidades, como lo son errores ortográficos que se repiten o trazos que no terminan de definirse. Estos vicios se arrastran por el resto de la vida académica hasta que el problema se ataca de manera tardía.

Frente a este problema buscamos brindar una aplicación web donde los alumnos de tercer y cuarto grado (un momento esencial en el desarrollo escolar) puedan reforzar los conocimientos y practicar recibiendo retroalimentación adecuada al momento; tarea que para el profesor es complicada ya que no puede dar atención personalizada a cada alumno en una clase.

La aplicación permitirá al alumno ingresar su escritura de dos formas: de forma digital o cargando una foto de su trabajo en papel. Una vez capturado el texto, el sistema utilizará técnicas de Reconocimiento Óptico de Caracteres (Optical Character Recognition, OCR por sus siglas en inglés) para poder realizar un análisis de calidad de letra con diferentes métricas específicas, como legibilidad, el tamaño de las letras, la alineación y el espaciado; y después convertirlo en texto digital para su posterior análisis con herramientas de Procesamiento de Lenguaje Natural (Natural Language Processing, NLP por sus siglas en inglés) que buscan detectar errores ortográficos y ofrecer sugerencias de corrección en el momento.

Para comprobar que la herramienta será validada mediante una prueba piloto con 25 alumnos durante un mes. Se registrará el desempeño al inicio y al final del periodo para medir si hubo mejora sustancial.

Índice de contenido

Índice de tablas

Índice de figuras

Glosario

API REST (Representational State Transfer Application Programming Interface):

Interfaz de programación que permite la comunicación y el intercambio de datos entre diferentes aplicaciones de software a través del protocolo HTTP, siguiendo una arquitectura sin estado.

Back-end: Capa lógica y de acceso a datos de una aplicación web. Se encarga de procesar las peticiones del usuario, interactuar con la base de datos y ejecutar la lógica de negocio en el servidor.

Base64: Sistema de codificación que traduce datos binarios (como imágenes) a una cadena de texto basada en caracteres ASCII. Facilita la transmisión de medios a través de redes y su almacenamiento en bases de datos.

Canvas (Lienzo digital): Elemento estandarizado de HTML5 que permite la renderización dinámica de gráficos 2D y mapas de bits mediante el uso de JavaScript. En este proyecto, sirve como la superficie interactiva donde el alumno realiza el trazo manuscrito.

CNN (Convolutional Neural Network / Red Neuronal Convolutacional): Arquitectura de aprendizaje profundo especializado en el procesamiento de datos con estructura de cuadrícula, como las imágenes, siendo fundamental para la extracción de características visuales en modelos OCR.

Endpoint: Punto final de comunicación en una API web. Representa una URL específica mediante la cual un cliente (front-end) puede acceder a los recursos o funciones del servidor (por ejemplo, guardar un intento o consultar métricas).

Front-end: Capa de presentación o interfaz gráfica de usuario de una aplicación. Es la parte visual con la que interactúan directamente los alumnos y tutores desde sus navegadores web.

JSON (JavaScript Object Notation): Formato de texto ligero y estructurado utilizado para el intercambio de datos entre el cliente y el servidor, caracterizado por ser legible tanto para humanos como para máquinas.

JWT (JSON Web Token): Estándar de la industria utilizado para la creación de tokens de acceso que permiten la autenticación segura y el intercambio de información verificable entre el cliente y el servidor, evitando la necesidad de mantener sesiones en memoria.

MVC (Model-View-Controller / Modelo-Vista-Controlador): Patrón de arquitectura de software que organiza el código separando la representación de la información (Modelo), la interfaz de usuario (Vista) y la lógica que maneja las interacciones

(Controlador).

NLP (Natural Language Processing / Procesamiento de Lenguaje Natural): Rama de la inteligencia artificial que permite a las computadoras interpretar, analizar y manipular el lenguaje humano. En el sistema, se utiliza para evaluar la ortografía y coherencia de las palabras detectadas.

OCR (Optical Character Recognition / Reconocimiento Óptico de Caracteres): Tecnología de visión computacional que permite convertir imágenes que contienen texto (manuscrito, impreso o tipografiado) en formatos de texto digital procesables por un equipo informático.

Otsu (Método de Otsu): Algoritmo utilizado en el procesamiento de imágenes para realizar una binarización automática. Calcula el umbral óptimo para separar los píxeles de primer plano (el trazo) de los del fondo, eliminando ruido visual.

Pangrama: Frase o texto breve que utiliza todas (o la gran mayoría) de las letras del alfabeto de un idioma. Se emplea didácticamente para practicar el trazo general de la caligrafía.

Payload (Carga útil): Conjunto de datos reales y esenciales que se transmiten en el cuerpo de una petición o respuesta de red entre el cliente y el servidor, excluyendo las cabeceras de protocolo.

Tokenización: Proceso analítico utilizado en NLP que consiste en dividir una cadena de texto continuo en unidades semánticas más pequeñas llamadas “tokens” (que pueden ser palabras, caracteres o sílabas) para facilitar su evaluación sintáctica y ortográfica.

Introducción

La escritura es una actividad compleja que demanda al estudiante no solo habilidades motrices sino cognitivas y lingüísticas por igual. En su etapa formativa implica comprender, organizar y representar ideas para comunicarse de manera efectiva. Cuando un estudiante consolida sus reglas ortográficas, afina su trazo o comprende una estructura gramatical, está construyendo una base que sostendrá gran parte de su escolaridad [?]. Por lo que, entonces, las dificultades en estas habilidades persisten latentes repercutiendo en el rendimiento académico, y también erosionan la confianza del estudiante en sus propias capacidades [?].

Hoy, las aulas no pueden ignorar la presencia de la tecnología, pero tampoco pueden rendirse ante ella. El verdadero desafío no es elegir entre lo digital y lo pedagógico, sino encontrar y fortalecer los mutuamente al otro. En esa dirección, organismos internacionales han señalado que integrar herramientas digitales al proceso de enseñanza, particularmente en competencias como la escritura, es una necesidad real y no una moda pasajera [?].

En paralelo, los avances en inteligencia artificial han abierto posibilidades que hace una década parecían lejanas. El OCR ha madurado hasta el punto de interpretar con gran precisión la escritura manuscrita, mientras que el NLP permite analizar textos de formas cada vez más sofisticadas [?][?]. Sin embargo, su aplicación concreta en el fortalecimiento de la escritura dentro de la educación básica sigue siendo un terreno poco explorado, lo que representa tanto una brecha como una oportunidad.

Es precisamente desde esa brecha que surge este trabajo. La idea central es desarrollar e implementar una aplicación web que ayude a niños de tercero y cuarto grado de primaria a mejorar su escritura en letra de molde, combinando reconocimiento de texto manuscrito con análisis ortográfico automatizado. Todo esto dentro de un entorno controlado y supervisado, con el propósito de evaluar su funcionamiento real mediante una prueba piloto que permita medir, ajustar y aprender.

1 Capítulo 1: Problemática

En este capítulo presentamos el contexto que da origen al desarrollo de una aplicación web orientada al fortalecimiento de la escritura en educación primaria. Se describe la situación actual relacionada con las dificultades en ortografía y calidad del trazo manuscrito en tercer y cuarto grado, así como la necesidad de incorporar herramientas tecnológicas que apoyen el proceso de retroalimentación académica. Además, se establecen los objetivos del proyecto y se revisan antecedentes que permiten comprender el panorama educativo y tecnológico en el que se enmarca la propuesta.

1.1. Planteamiento del problema

La adquisición y consolidación de habilidades de escritura durante la educación primaria constituye un elemento determinante en el desarrollo académico posterior de los estudiantes. La alfabetización temprana no solo impacta el desempeño en el área de lenguaje, sino que influye transversalmente en otras disciplinas escolares [?]. Sin embargo, diversos informes educativos han señalado que persisten dificultades en competencias relacionadas con redacción y ortografía en niveles básicos de educación [?].

En el contexto nacional, los indicadores de logro educativo han evidenciado áreas de oportunidad en producción escrita y comunicación, particularmente en educación primaria [?]. Estas dificultades pueden manifestarse en errores ortográficos recurrentes y problemas en la legibilidad del trazo manuscrito, lo que afecta la claridad de los textos producidos por los alumnos.

La evaluación de estos aspectos depende principalmente de la revisión manual realizada por el docente. Aunque esta intervención es esencial, en grupos numerosos puede limitarse la frecuencia y profundidad de la retroalimentación individualizada. La literatura pedagógica ha destacado que la retroalimentación oportuna y continua es un factor clave para la mejora del aprendizaje [?].

Paralelamente, la incorporación de tecnologías digitales en el ámbito educativo ha crecido de manera significativa en los últimos años; no obstante, muchas herramientas se centran en ejercicios mecanografiados y no abordan directamente la práctica y análisis de la escritura manuscrita [?]. Esta situación genera una brecha entre el avance tecnológico y la necesidad de fortalecer habilidades tradicionales mediante enfoques innovadores.

Ante este panorama, surge la necesidad de explorar alternativas que permitan analizar textos manuscritos y ofrecer retroalimentación automatizada en un entorno educativo supervisado. En consecuencia, el problema central que da origen al presente trabajo se formula de la siguiente manera:

¿Cómo apoyar el fortalecimiento de la escritura en letra de molde en alumnos de tercer y cuarto grado de primaria mediante una herramienta tecnológica que permita la detección automatizada de errores ortográficos y la evaluación básica de la calidad del trazo?

1.2. Propuesta de solución

Para atender la problemática identificada, se propone el desarrollo e implementación de una aplicación web orientada al fortalecimiento de la escritura en letra de molde en alumnos de tercer y cuarto grado de educación primaria. La solución integra herramientas tecnológicas que permiten analizar textos manuscritos y generar retroalimentación automatizada dentro de un entorno educativo supervisado.

La aplicación permitirá la captura de ejercicios mediante escritura directa en pantalla o carga de imágenes digitalizadas. Posteriormente, se emplearán técnicas de OCR para convertir el texto manuscrito en formato digital, y herramientas de NLP para detectar posibles errores ortográficos y sugerir correcciones.

Asimismo, se evaluarán parámetros básicos relacionados con la calidad del trazo, alineación, tamaño, espaciado e inclinación. La aplicación será desplegada en un servidor funcional y validado mediante una prueba piloto, con el fin de analizar su impacto preliminar en el fortalecimiento de la escritura.

1.3. Justificación

La práctica de las habilidades de escritura en educación primaria representa una necesidad prioritaria dentro del proceso formativo, debido a su impacto directo en el desempeño académico y en el desarrollo integral del estudiante. La alfabetización efectiva en etapas tempranas se asocia con mejores resultados educativos a largo plazo y mayor capacidad de aprendizaje en distintas áreas del conocimiento [?].

En el contexto nacional, los reportes recientes sobre el estado de la educación han señalado la importancia de implementar estrategias que apoyen el desarrollo de competencias comunicativas desde los primeros grados escolares [?]. La detección temprana de errores ortográficos y dificultades en la escritura permite intervenir oportunamente, evitando que dichas problemáticas se consoliden en niveles educativos posteriores.

Desde el ámbito tecnológico, el uso de herramientas basadas en inteligencia artificial en entornos educativos ha demostrado potencial para mejorar procesos de retroalimentación y seguimiento académico cuando se emplean como apoyo complementario al docente [?]. La posibilidad de automatizar ciertos procesos de análisis no busca sustituir la labor pedagógica, sino fortalecerla mediante información adicional que facilite la toma de decisiones

educativas.

Asimismo, el proyecto resulta viable desde el punto de vista técnico, al apoyarse en tecnologías consolidadas como el reconocimiento de patrones y el análisis lingüístico automatizado, las cuales han mostrado niveles adecuados de precisión en distintos contextos [?]. Su implementación en un entorno controlado mediante una prueba piloto permite evaluar su funcionamiento real y analizar su impacto preliminar.

Por lo anterior, el desarrollo de la aplicación propuesta se justifica por su pertinencia educativa, su aporte tecnológico y su potencial contribución al fortalecimiento de habilidades fundamentales en educación básica.

1.4. Objetivos

1.4.1. *Objetivo general*

Desarrollar una aplicación web interactiva para la práctica de la escritura manuscrita en estudiantes de tercer y cuarto grado de primaria, que integre captura digital, reconocimiento automático de texto, análisis caligráfico y retroalimentación ortográfica, utilizando técnicas de visión por computadora y procesamiento de lenguaje natural.

1.4.2. *Objetivos específicos*

Los objetivos específicos del proyecto se centran en:

- OE-01. Seleccionar y adaptar datasets existentes de escritura manuscrita con letra de molde, incorporando muestras con caracteres acentuados para garantizar la variabilidad necesaria en el entrenamiento y validación de los modelos.
- OE-02. Desarrollar el módulo de captura digital de escritura manuscrita con letra de molde, con validaciones de calidad de imagen en el cliente.
- OE-03. Desarrollar un sistema de OCR especializado en escritura infantil, que alcance una precisión mínima del 85 % en un conjunto de validación representativo.
- OE-04. Desarrollar un módulo de extracción de métricas caligráficas (alineación, tamaño, espaciado e inclinación) que permita evaluar la calidad de la escritura con criterios cuantificables.
- OE-05. Implementar un módulo de análisis ortográfico basado en NLP que identifique errores y genere sugerencias de corrección con una tasa de detección mayor o igual al 85 % en pruebas controladas.
- OE-06. Diseñar e implementar un módulo de ejercicios correctivos que asigne actividades de refuerzo a partir de las métricas caligráficas y el análisis ortográfico.

- OE-07. Desarrollar el Front-End interactivo y responsivo, adecuado al nivel cognitivo de estudiantes de primaria y compatible con distintos dispositivos.
- OE-08. Desarrollar el Back-End y la Base de Datos del sistema, incluyendo autenticación y gestión de usuarios.
- OE-09. Integrar todos los módulos desarrollados y realizar pruebas de aceptación con estudiantes reales de tercer y cuarto grado.

1.5. Estado del arte

Para comprender el panorama actual de las herramientas de asistencia a la escritura y el reconocimiento de texto, se realizó un estado del arte que permitió identificar soluciones existentes con funcionalidades similares a las propuestas en este proyecto. A continuación, se presenta la Tabla ?? que compara aplicaciones comerciales y herramientas de código abierto detallando su descripción, características y un costo aproximado de su desarrollo.

Tabla 1: Herramientas de asistencia a la escritura y el reconocimiento de texto.

Proyecto	Descripción	Características	Precio Aproximado
Google Handwriting Input [?]	Aplicación para escribir a mano en dispositivos Android y convertir la escritura a texto.	-Compatible con Android. -Soporta múltiples idiomas. -Escribe en cualquier campo de texto. -Reconocimiento en tiempo real.	El costo de desarrollo de una app similar sería de \$20,000 a \$50,000 USD.
Tesseract OCR [?]	Motor OCR de código abierto que convierte imágenes de texto manuscrito o tipografiado en texto editable.	- Soporta más de 100 idiomas. - Compatible con imágenes escaneadas o fotos. - Usado para entrenar modelos de OCR. - Requiere configuraciones para mejorar precisión en manuscritos.	Proyecto similar costaría entre \$50,000 a \$150,000 USD dependiendo de la personalización.

Proyecto	Descripción	Características	Precio Aproximado
Grammarly [?]	Plataforma de corrección gramatical y ortográfica basada en inteligencia artificial, disponible como extensión o en la web.	- Corrección gramatical y ortográfica. - Recomendaciones de estilo. - Compatible con múltiples plataformas (navegadores, MS Word, Google Docs). - Versión gratuita con funciones limitadas.	\$12 USD/mes (versión premium). Desarrollo similar podría costar \$500,000 USD.
Calligrapher [?]	Aplicación para mejorar la caligrafía, proporcionando retroalimentación en tiempo real sobre la calidad de la letra.	- Evaluación en tiempo real de escritura. - Enfoque en caligrafía y legibilidad. - Compatible con dispositivos móviles y tabletas. - Ofrece ejercicios estructurados y personalizados.	\$30,000 a \$100,000 USD para desarrollo de plataforma similar.
WriteApp [?]	Plataforma educativa gamificada que mejora la escritura manual, ofreciendo retroalimentación sobre ortografía y calidad de letra.	- Juego interactivo para mejorar la escritura. - Seguimiento de la calidad de letra y puntuación. - Genera informes para padres y maestros. - Personalización de ejercicios para cada usuario.	\$50,000 a \$200,000 USD para desarrollo de plataforma similar.

Proyecto	Descripción	Características	Precio Aproximado
Aplicación Web para la práctica de escritura manual y detección automática de errores ortográficos	Aplicación educativa enfocada en la mejora de la escritura manual y la corrección ortográfica para estudiantes de primaria, con retroalimentación automática y evaluación de la calidad de la letra.	- Integración de módulos de escritura manual, OCR y NLP. - Evaluación de calidad de letra (legibilidad, tamaño, alineación). - Detección de errores ortográficos. - Aplicación web interactiva. - Uso en dispositivos de escritorio y móviles.	En desarrollo.

2 Capítulo 2: Marco teórico

En este capítulo se presentan los fundamentos teóricos que sustentan el desarrollo del proyecto, abordando tanto los aspectos pedagógicos relacionados con la adquisición de la escritura en educación primaria como los elementos tecnológicos que permiten su análisis automatizado. Se revisan conceptos vinculados al proceso de consolidación de la ortografía y la calidad del trazo manuscrito, así como los principios básicos del OCR, el NLP y la arquitectura de sistemas web. El propósito de este marco teórico es proporcionar el sustento conceptual necesario para comprender la propuesta planteada y establecer la base académica sobre la cual se desarrolla la solución tecnológica.

2.1. Fundamentos de la escritura en la educación

La escritura en educación primaria constituye una habilidad esencial para el desarrollo académico y social de los estudiantes. La alfabetización temprana permite que los niños adquieran competencias básicas de lectura y escritura que influyen directamente en su capacidad de aprendizaje a lo largo de la trayectoria escolar [?]. El dominio de estas habilidades no solo impacta el área de lenguaje, sino que favorece el desempeño en otras disciplinas al facilitar la comprensión y expresión de ideas, así como el desarrollo de habilidades comunicativas fundamentales.

Organismos internacionales han señalado que fortalecer la lectura y escritura desde los primeros grados escolares contribuye significativamente a mejorar los resultados educativos y reducir brechas de aprendizaje [?]. En este sentido, la escritura no se limita a la producción mecánica de palabras, sino que implica procesos cognitivos complejos relacionados con la organización del pensamiento, la memoria y la estructuración de ideas. Estos procesos permiten al estudiante no solo reproducir información, sino también interpretarla, analizarla y expresarla de manera coherente.

La práctica de la escritura manuscrita desempeña un papel relevante en este proceso, ya que favorece la coordinación motora fina y la consolidación de la relación entre grafemas y fonemas. Diversos recursos educativos destacan que la enseñanza explícita de la escritura a mano fortalece la identificación de letras, la ortografía y la retención de información [?]. Asimismo, la escritura activa múltiples procesos cognitivos vinculados con la atención, la memoria de trabajo y el aprendizaje significativo, lo que contribuye a una mayor internalización del conocimiento [?].

Además, el desarrollo de la escritura en esta etapa implica la interacción de distintos factores que van más allá del aspecto lingüístico, incluyendo habilidades motoras, cognitivas y pedagógicas. La integración de estos elementos permite que el estudiante desarrolle progresivamente la capacidad de expresar ideas de manera clara, estructurada y coherente,

lo cual es fundamental para su desempeño académico.

En este sentido, el desarrollo de la escritura en educación primaria puede entenderse como un proceso integral que combina aspectos lingüísticos, cognitivos y motrices. La identificación de sus componentes fundamentales permite establecer criterios claros para su evaluación y fortalecimiento dentro de entornos educativos formales. Como se presenta en la Tabla ??, dichos componentes permiten comprender de manera estructurada los elementos que influyen directamente en el aprendizaje de la escritura.

Tabla 2: Componentes de la escritura y su impacto en el aprendizaje.

Componente	Descripción	Impacto en el aprendizaje
Motricidad fina	Coordinación y control del trazo manuscrito	Mejora la legibilidad
Ortografía	Aplicación correcta de reglas lingüísticas	Favorece claridad textual
Conciencia fonológica	Relación entre sonidos y grafías	Reduce errores frecuentes
Organización escrita	Estructuración coherente de ideas	Facilita comprensión
Retroalimentación	Corrección continua del desempeño	Refuerza el aprendizaje

2.2. Alfabetización digital en entornos educativos

La alfabetización digital se refiere a la capacidad de utilizar tecnologías de la información y la comunicación de manera efectiva, crítica y responsable dentro de distintos contextos, incluido el educativo. En los últimos años, organismos internacionales han destacado la necesidad de integrar herramientas digitales como parte complementaria del proceso de enseñanza-aprendizaje, especialmente en educación básica [?]. Esta integración no solo implica el uso de dispositivos tecnológicos, sino también el desarrollo de competencias que permitan a los estudiantes interactuar con la información de manera reflexiva y autónoma.

La incorporación de tecnología en el aula no implica sustituir la función del docente, sino ampliar las posibilidades de retroalimentación, seguimiento y acceso a recursos educativos. La Organización de las Naciones Unidas para la Educación, la Ciencia y la Cultura (United Nations Educational, Scientific and Cultural Organization, UNESCO por sus siglas en inglés) ha señalado que el uso adecuado de tecnologías digitales puede fortalecer competencias fundamentales cuando se implementa bajo criterios pedagógicos claros [?]. Asimismo, el Banco Mundial ha subrayado que las soluciones tecnológicas en educación

deben diseñarse con un enfoque centrado en el aprendizaje y no únicamente en la herramienta tecnológica [?]. Esto implica que el valor de la tecnología radica en su correcta integración dentro de estrategias didácticas bien estructuradas.

En el caso de la escritura, muchas plataformas digitales actuales se enfocan en ejercicios mecanografiados, dejando en segundo plano la práctica manuscrita. Esta situación evidencia la necesidad de explorar propuestas que integren tecnología sin abandonar habilidades tradicionales esenciales. La alfabetización digital, por tanto, no consiste únicamente en el uso de dispositivos, sino en la integración estratégica de herramientas que potencien el aprendizaje significativo y mantengan un equilibrio entre lo digital y lo manual. Además, es importante considerar que la implementación de tecnologías en entornos educativos conlleva tanto beneficios como retos, los cuales deben ser analizados para garantizar un uso adecuado. Factores como la infraestructura tecnológica, la capacitación docente y el acceso equitativo a los recursos influyen directamente en el impacto de estas herramientas dentro del proceso educativo.

Las principales ventajas y desafíos asociados al uso de tecnología en educación básica se presentan en la Tabla ??, donde se sintetizan los elementos más relevantes que intervienen en la integración tecnológica dentro del aula.

Tabla 3: Ventajas y desafíos de la integración tecnológica en educación básica.

Aspecto	Descripción
Retroalimentación inmediata	Permite correcciones en tiempo real
Seguimiento individualizado	Facilita monitoreo del progreso
Motivación del estudiante	Incrementa interés y participación
Dependencia tecnológica	Requiere infraestructura adecuada
Brecha digital	Puede generar desigualdades de acceso

El análisis de estos factores permite comprender que la tecnología, cuando se implementa como herramienta complementaria y supervisada, puede contribuir significativamente al fortalecimiento de competencias académicas fundamentales, favoreciendo tanto el aprendizaje individual como el desarrollo de habilidades digitales necesarias en el contexto actual.

2.3. OCR

El OCR es una tecnología que permite convertir imágenes que contienen texto impreso o manuscrito en información digital procesable por un sistema computacional [?]. Su aplicación se ha extendido en distintos ámbitos, como la digitalización de documentos, la

automatización de formularios y el procesamiento de información textual a partir de imágenes escaneadas, facilitando la gestión y análisis de grandes volúmenes de información.

El funcionamiento del OCR se basa en una serie de etapas secuenciales que permiten transformar la imagen original en texto estructurado. De manera general, el proceso inicia con la adquisición de una imagen que contiene caracteres y continúa con procedimientos de análisis y reconocimiento automatizado, los cuales permiten interpretar la información visual de forma sistemática. En la Figura ?? se presenta el flujo general del proceso OCR, el cual describe las etapas principales involucradas en la transformación de una imagen en texto digital.

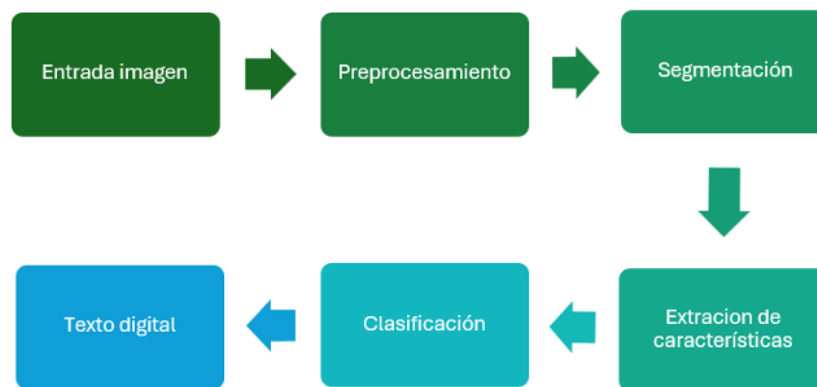


Figura 1: Flujo general del proceso OCR adaptado.

Adaptado de [?, ?]

Como se observa en la Figura ??, el proceso comienza con la entrada de una imagen, que puede provenir de una fotografía o de un lienzo digital. Posteriormente, en la etapa de preprocesamiento, se realizan ajustes como conversión a escala de grises, binarización y eliminación de ruido, con el fin de mejorar la calidad visual para su análisis [?]. Estas técnicas permiten reducir variaciones no deseadas en la imagen, facilitando el reconocimiento posterior de los caracteres.

En la fase de segmentación, el sistema identifica y separa los caracteres individuales o grupos de caracteres dentro de la imagen. Este proceso resulta crítico, ya que una segmentación incorrecta puede afectar directamente la precisión del reconocimiento. A continuación, durante la extracción de características, se analizan propiedades específicas como contornos, formas y patrones estructurales que permiten diferenciar un carácter de otro.

Finalmente, en la etapa de clasificación, el sistema compara las características extraídas con modelos previamente entrenados para determinar a qué carácter corresponde cada patrón identificado, generando como resultado el texto digital final [?]. Esta etapa constituye el núcleo del sistema OCR, donde se realiza la toma de decisiones basada en modelos de reconocimiento.

Históricamente, los motores OCR se basaban en la comparación de plantillas estáticas y en la extracción de características geométricas como contornos y proyecciones de píxeles. Sin embargo, los enfoques contemporáneos han migrado hacia arquitecturas de aprendizaje profundo que logran tasas de reconocimiento significativamente superiores. En particular, las redes neuronales convolucionales (Convolutional Neural Networks, CNN por sus siglas en inglés) han demostrado ser especialmente efectivas en la extracción automática de rasgos visuales de caracteres sin requerir ingeniería manual de características [?]. Una evolución relevante es la arquitectura CRNN (Convolutional Recurrent Neural Network), que combina capas convolucionales para la extracción de características visuales con capas recurrentes para modelar dependencias secuenciales entre caracteres, permitiendo el reconocimiento de texto como una secuencia continua en lugar de caracteres aislados [?].

La etapa de preprocesamiento resulta determinante para la precisión del sistema. La binarización mediante el método de Otsu, que selecciona automáticamente un umbral óptimo de separación entre píxeles de fondo y trazo a partir del histograma de niveles de gris de la imagen, es una técnica ampliamente adoptada por su robustez ante variaciones de iluminación [?]. Complementariamente, técnicas de normalización del tamaño y corrección de inclinación (deskewing) contribuyen a reducir la variabilidad que enfrenta el clasificador.

La escritura manuscrita infantil presenta desafíos adicionales en comparación con texto impreso o escritura adulta. La irregularidad en el tamaño de letras, la variabilidad en el espaciado, los trazos incompletos y la presión inconsistente sobre la superficie de escritura introducen ruido que degrada el desempeño de los modelos [?]. Por ello, en aplicaciones educativas orientadas a estudiantes de educación primaria resulta necesario considerar márgenes de error más amplios y mecanismos de validación complementarios, tal como se establece en los requerimientos no funcionales del presente sistema (RNF-03).

En el caso de la escritura manuscrita, el OCR enfrenta desafíos adicionales debido a la variabilidad en estilos de letra, inclinación, tamaño y calidad del trazo. Estas variaciones incrementan la complejidad del reconocimiento y pueden generar errores si no se cuenta con modelos suficientemente robustos. Por ello, su integración en entornos educativos requiere considerar posibles márgenes de error y mecanismos complementarios de validación que permitan mejorar la confiabilidad del sistema.

El desarrollo reciente de sistemas de reconocimiento de escritura manuscrita ha evolucionado hacia el uso de modelos basados en aprendizaje profundo, particularmente redes neuronales recurrentes (RNN) y arquitecturas tipo Transformer, las cuales permiten modelar secuencias completas de texto sin requerir segmentación explícita de caracteres individuales. Este enfoque resulta especialmente relevante en escenarios donde la escritura presenta variaciones significativas, como es el caso de la escritura infantil, en la cual existen inconsistencias en tamaño, inclinación y separación entre caracteres [?, ?].

Además, técnicas modernas como los mecanismos de atención han permitido mejorar la capacidad de los sistemas OCR para enfocarse en regiones relevantes de la imagen, incrementando la precisión en condiciones no ideales. Estas mejoras son particularmente útiles en aplicaciones educativas, donde las imágenes pueden presentar ruido, baja resolución o iluminación variable [?].

Por otra parte, la integración de modelos preentrenados ha permitido aprovechar grandes volúmenes de datos para mejorar el desempeño del reconocimiento, reduciendo la necesidad de entrenamiento desde cero y facilitando su adaptación a dominios específicos como la escritura manuscrita en educación básica [?, ?]. Este tipo de enfoques contribuye a mejorar la escalabilidad del sistema y su capacidad de adaptación a diferentes contextos.

No obstante, a pesar de estos avances, la precisión del OCR sigue dependiendo en gran medida de la calidad de la entrada, por lo que resulta fundamental garantizar condiciones adecuadas en la captura de la imagen y en el preprocesamiento, con el fin de maximizar el desempeño del sistema en aplicaciones reales.

2.4. NLP

El NLP es una rama de la inteligencia artificial que permite a los sistemas computacionales analizar, interpretar y generar lenguaje humano de manera automatizada [?]. Esta disciplina combina conocimientos de lingüística, informática y aprendizaje automático para procesar texto y extraer información significativa, permitiendo transformar datos textuales en información útil para distintos tipos de aplicaciones.

En el ámbito educativo, el NLP se ha utilizado en aplicaciones como análisis de redacción, detección de errores ortográficos, evaluación automática de textos y generación de retroalimentación personalizada [?]. Estas aplicaciones permiten apoyar el proceso de aprendizaje al proporcionar herramientas que facilitan la identificación de errores y la mejora continua del desempeño del estudiante.

A diferencia del OCR, que convierte imágenes en texto digital, el NLP trabaja directamente sobre el contenido textual, permitiendo su análisis lingüístico. Esta diferencia es fundamental dentro del sistema propuesto, ya que el OCR se encarga de la captura y digitalización del texto manuscrito, mientras que el NLP permite interpretar y analizar dicho contenido, estableciendo una relación directa entre ambas tecnologías. El proceso general de detección de errores ortográficos mediante NLP puede representarse de forma conceptual en la Figura ??.

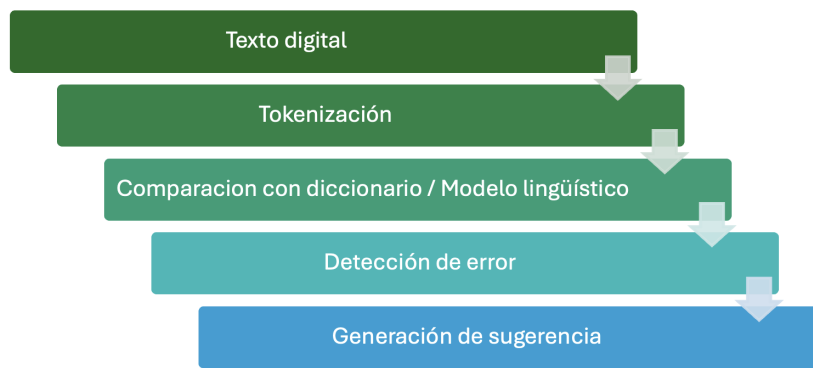


Figura 2: Flujo básico del proceso de corrección ortográfica mediante NLP.

Elaboración con base en [?]

Como se muestra en la Figura ??, el procedimiento inicia con el texto digital previamente obtenido. Posteriormente, en la etapa de tokenización, el sistema divide el texto en unidades mínimas, generalmente palabras o tokens individuales, lo cual facilita su análisis individual y contextual dentro de la oración.

En la siguiente fase, cada token es comparado con un diccionario léxico o modelo lingüístico entrenado. Si una palabra no coincide con las entradas válidas, el sistema la identifica como posible error ortográfico. Para generar una sugerencia, pueden emplearse algoritmos de similitud como la distancia de Levenshtein o modelos estadísticos que calculan la probabilidad de ocurrencia de una palabra dentro de un contexto determinado [?]. Este tipo de técnicas permite no solo detectar errores, sino también proponer alternativas adecuadas.

Este enfoque permite ofrecer retroalimentación automatizada en forma de sugerencias, lo cual resulta útil en entornos educativos cuando se busca apoyar el aprendizaje sin reemplazar la intervención del docente. No obstante, la precisión del sistema depende de la calidad del modelo lingüístico, del contexto del texto y de la claridad del contenido procesado.

La integración de NLP dentro del sistema propuesto permite complementar el análisis realizado por el OCR, cerrando el ciclo de procesamiento desde la captura del texto manuscrito hasta la generación de sugerencias ortográficas. Esta integración representa un enfoque más completo para el tratamiento de la información, combinando procesamiento visual y análisis lingüístico.

En los últimos años, el NLP ha experimentado avances significativos gracias al uso de modelos de representación distribuida del lenguaje, conocidos como embeddings, los cuales permiten capturar relaciones semánticas entre palabras en espacios vectoriales de alta dimensión. Estos modelos han demostrado ser efectivos para mejorar tareas de análisis

lingüístico, incluyendo la corrección ortográfica contextual y la detección de errores en textos escritos [?].

Asimismo, el uso de modelos más avanzados basados en arquitecturas modernas ha permitido mejorar la comprensión del contexto dentro de una oración, lo que resulta especialmente útil en aplicaciones educativas donde los errores pueden depender no solo de la forma de la palabra, sino de su uso dentro de una estructura gramatical. Esto permite generar sugerencias más precisas y adaptadas al contexto real del usuario.

2.5. Evaluación del trazo manuscrito

El trazo manuscrito constituye un elemento relevante dentro del proceso de adquisición de la escritura en educación primaria. La legibilidad de un texto no depende únicamente de la ortografía correcta, sino también de factores visuales y espaciales que permiten interpretar adecuadamente el contenido escrito [?]. Entre estos factores se encuentran la alineación sobre la línea base, la proporción del tamaño de las letras, el espaciado entre palabras y la uniformidad del trazo, los cuales influyen directamente en la claridad y comprensión del texto.

Diversos estudios sobre desarrollo de la escritura señalan que la evaluación de la letra manuscrita puede abordarse mediante parámetros observables que permitan identificar patrones de mejora o dificultad en los estudiantes [?]. En contextos educativos tradicionales, esta evaluación es realizada por el docente de forma visual y cualitativa; sin embargo, con el apoyo de herramientas digitales es posible establecer métricas básicas que complementen dicha valoración, aportando un enfoque más objetivo al análisis del desempeño.

Desde una perspectiva computacional, algunos de estos parámetros pueden aproximarse mediante técnicas de análisis de imagen, tales como la medición de altura promedio de caracteres, la variación de inclinación o la separación entre segmentos gráficos [?]. Estas técnicas permiten cuantificar características del trazo que, de otra forma, serían evaluadas únicamente de manera subjetiva. Si bien estos indicadores no sustituyen la evaluación pedagógica, pueden ofrecer información cuantitativa útil para el seguimiento del progreso del estudiante a lo largo del tiempo.

Los principales parámetros considerados para la evaluación básica del trazo en el presente proyecto se describen en la Tabla ??, donde se establecen tanto los criterios pedagógicos como sus posibles aproximaciones computacionales, permitiendo vincular la evaluación tradicional con el análisis automatizado.

Tabla 4: Parámetros de evaluación del trazo manuscrito.

Parámetro	Descripción	Posible aproximación computacional
Legibilidad	Claridad visual de los caracteres	Porcentaje de reconocimiento correcto por OCR
Alineación	Ubicación uniforme sobre línea base	Desviación vertical promedio
Tamaño	Proporción consistente de las letras	Altura media de caracteres
Espaciado	Separación adecuada entre letras	Distancia promedio entre segmentos
Uniformidad	Consistencia en forma y grosor del trazo	Variación en contornos detectados

Como se observa en la Tabla 4, los parámetros seleccionados combinan criterios pedagógicos tradicionales con posibles métricas de análisis digital. Esta integración permite establecer un modelo básico de evaluación que apoye el seguimiento del desempeño del estudiante sin reemplazar la interpretación docente, sino más bien complementándola.

La incorporación de estos indicadores dentro del sistema propuesto busca complementar el análisis ortográfico realizado mediante NLP, ofreciendo una visión más integral del proceso de escritura. De esta manera, el sistema no solo evalúa la corrección del texto, sino también la calidad del trazo manuscrito, integrando así componentes visuales y lingüísticos dentro de una misma solución tecnológica.

2.6. Arquitectura de la aplicación web

El desarrollo de aplicaciones web modernas suele estructurarse bajo patrones de diseño que permiten organizar de manera eficiente los componentes del sistema. Uno de los modelos más utilizados es la arquitectura Modelo Vista Controlador (Model-View-Controller, MVC por sus siglas en inglés), la cual divide la aplicación en tres capas principales: modelo, vista y controlador [?].

En la Figura ?? se presenta el esquema general de la arquitectura MVC aplicada al sistema propuesto.

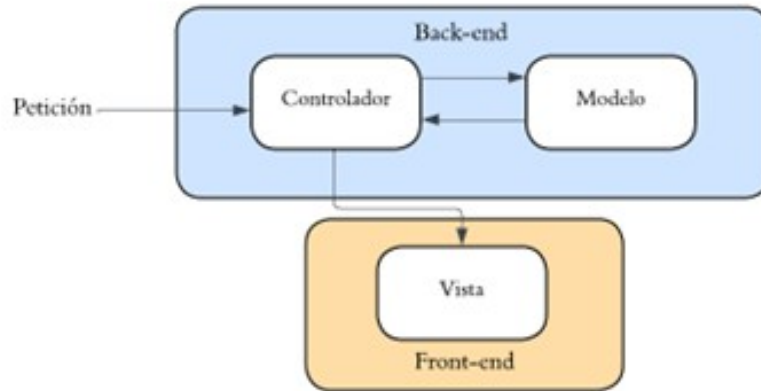


Figura 3: Arquitectura Modelo-Vista-Controlador aplicada al sistema [?].

Como se observa en la Figura ??, el flujo de operación inicia con una petición realizada desde el front-end. Esta solicitud es enviada al controlador, ubicado en el back-end, el cual actúa como intermediario entre la interfaz y la lógica del sistema. El controlador procesa la solicitud y se comunica con el modelo, donde se gestionan los datos y la lógica de negocio.

El modelo es responsable del almacenamiento de información, incluyendo usuarios, ejercicios realizados, resultados del OCR, análisis ortográfico mediante NLP y métricas asociadas a la calidad del trazo. Esta separación permite mantener organizada la lógica interna del sistema y facilita futuras modificaciones o ampliaciones.

Posteriormente, el controlador recibe la información procesada y genera una respuesta que es enviada a la vista, encargada de mostrar al usuario la retroalimentación correspondiente. Esta estructura modular favorece la mantenibilidad, escalabilidad y claridad del sistema [?].

Además, el despliegue del sistema en un servidor web funcional permite su acceso mediante navegador, garantizando un entorno controlado para la prueba piloto y la centralización de datos. La adopción del modelo MVC facilita la integración coherente de los módulos de OCR, NLP y evaluación del trazo dentro de una arquitectura estructurada.

2.7. Experiencia de Usuario y Diseño de Interfaces Educativas

El diseño de interfaces para aplicaciones educativas destinadas a niños de entre 8 y 10 años requiere un enfoque especializado que considere sus capacidades cognitivas, motoras y de alfabetización en desarrollo [?], [?]. La Experiencia de Usuario (UX) en este rango de edad no solo busca la eficiencia en la ejecución de tareas, sino también fomentar el compromiso y minimizar la frustración durante el proceso de aprendizaje [?]. Es vital entender que el diseño debe facilitar la transición entre lo digital y lo manual, preservando los beneficios

cognitivos que la neurociencia atribuye al trazo físico sobre el teclado [?], [?].

2.7.1. Consideraciones Cognitivas y Carga Visual

A diferencia de los usuarios adultos, los estudiantes de tercer y cuarto grado poseen una capacidad limitada para procesar múltiples estímulos simultáneos, por lo que la interfaz debe minimizar la carga cognitiva mediante un diseño limpio y el uso de metáforas visuales claras [?]. Esto implica la utilización de áreas de interacción amplias, botones de gran tamaño y tipografías legibles que cumplan con índices de lecturabilidad específicos para su nivel escolar, como el de Fernández-Huerta [?]. Un diseño visualmente ordenado permite que el alumno mantenga la atención en el objetivo principal: la práctica de la escritura manuscrita [?].

2.7.2. Arquitectura de Información y Navegación

La navegación en sistemas educativos infantiles debe ser sencilla y preferiblemente plana, ya que las estructuras jerárquicas profundas suelen confundir a los usuarios jóvenes [?]. Es recomendable que las funcionalidades principales sean accesibles con un mínimo de interacciones y que el uso de íconos representativos esté acompañado de etiquetas de texto sencillas que refuercen la autonomía del estudiante dentro de la plataforma [?]. Además, la interfaz de captura debe ser lo suficientemente robusta para generar imágenes de alta calidad que permitan un procesamiento eficiente por parte de arquitecturas avanzadas de reconocimiento de texto como TrOCR [?].

2.7.3. Retroalimentación y Sistemas de Respuesta

La retroalimentación inmediata es un componente crítico para el aprendizaje guiado, especialmente en la detección de errores ortográficos y la evaluación de la calidad del trazo [?]. El sistema debe proporcionar respuestas multisensoriales que informen al niño sobre sus aciertos y áreas de mejora de manera constructiva, reduciendo la ansiedad ante el error y promoviendo la persistencia en la actividad académica [?]. Esta respuesta constante ayuda a consolidar las conexiones neuronales necesarias para el dominio de la escritura [?].

2.7.4. Engagement y Gamificación

Para mantener la motivación en tareas que requieren repetición, resulta beneficioso integrar elementos de gamificación como barras de progreso, insignias o recompensas visuales [?]. El diseño debe equilibrar estos elementos lúdicos para que actúen como un refuerzo

pedagógico efectivo sin convertirse en una distracción del proceso de aprendizaje central [?]. El refuerzo positivo mediante herramientas digitales es una estrategia clave para evitar que el estudiante pierda el interés en el desarrollo de sus habilidades caligráficas tradicionales [?].

3 Capítulo 3: Análisis

En este capítulo se trabajará el análisis de los módulos de la aplicación para su correcto funcionamiento abarcando desde los límites y capacidades de la aplicación, así como los requerimientos, riesgos y mitigación de estos para tener una base sólida para el diseño y posterior desarrollo de nuestra solución.

3.1. Alcance y Límites de la aplicación

El propósito de este apartado es establecer de manera precisa el ámbito de desarrollo e implementación del proyecto. A continuación, se definen las capacidades operativas y el público objetivo que conforman el alcance del sistema, así como las restricciones técnicas, lingüísticas y de conectividad que delimitan su funcionamiento.

3.1.1. Alcance de la aplicación web

La aplicación web está dirigida exclusivamente a estudiantes de tercer y cuarto grado de educación primaria. La plataforma permitirá la recolección de textos manuscritos en letra de molde mediante la escritura directa en un lienzo en pantalla o a través de la carga de imágenes.

Una vez capturado, el texto será procesado mediante algoritmos de OCR para su digitalización, seguido por la intervención de un módulo de NLP encargado de analizar el contenido, detectar errores ortográficos y generar sugerencias automáticas de corrección.

Además, el sistema evaluará métricas cuantitativas del trazo manuscrito, considerando parámetros como la legibilidad, alineación, tamaño y espaciado.

3.1.2. Límites de la aplicación web

Las restricciones técnicas y operativas de la aplicación son en cuanto al estilo de escritura, el modelo de reconocimiento está estrictamente acotado al procesamiento de letra de molde (tipografía Century) [?], excluyendo la evaluación de letra cursiva u otras tipografías.

Respecto a la restricción lingüística, el análisis ortográfico y las sugerencias generadas se limitarán al nivel cognitivo y vocabulario correspondiente a los grados de tercero y cuarto de primaria.

En relación con el rol de la aplicación, el sistema funcionará exclusivamente como una herramienta de retroalimentación automatizada y complementaria, basada en un enfoque de aprendizaje ecléctico, sin sustituir la evaluación pedagógica integral del docente.

Finalmente, debido a su diseño basado en una arquitectura web cliente-servidor, existe una dependencia de conectividad, por lo que el funcionamiento de los módulos OCR y NLP requerirá una conexión activa a internet.

3.2. Factibilidad económica

Para fundamentar cuantitativamente la viabilidad económica del proyecto, se aplicó el modelo COCOMO I Básico (Constructive Cost Model), ampliamente utilizado en la estimación de proyectos de software a partir del tamaño del código fuente. Este modelo permite determinar el esfuerzo requerido, el tiempo de desarrollo y el costo comercial equivalente del sistema, proporcionando una base objetiva para evaluar si el proyecto es ejecutable con los recursos disponibles.

3.2.1. Clasificación del proyecto

De acuerdo con la taxonomía de COCOMO I, los proyectos se clasifican en tres modos según la complejidad del equipo y del entorno de desarrollo: orgánico, semiacoplado e integrado. El presente proyecto se clasifica como orgánico, dado que el equipo de desarrollo es pequeño (tres integrantes), existe familiaridad previa con las tecnologías utilizadas, y los requerimientos no presentan rigidez crítica. Este modo corresponde a sistemas de tamaño moderado desarrollados en entornos académicos o de investigación.

3.2.2. Estimación de líneas de código

Como entrada principal del modelo, se realizó una estimación de las Líneas de Código Fuente (LOC) por módulo del sistema. La estimación se elaboró con base en la arquitectura y los requerimientos funcionales definidos en las secciones 3.4, 3.5 y 3.7, considerando el stack tecnológico seleccionado (React, FastAPI, SQL Server). Los resultados se presentan en la Tabla ??.

Tabla 5: Estimación de líneas de código por módulo del sistema.

Módulo	Componentes principales	LOC estimadas
Front-End (React)	Autenticación, panel tutor, panel alumno, lienzo digital, visualización de resultados	3,500
Back-End (FastAPI)	Endpoints REST, lógica de negocio, integración OCR/NLP, seguridad JWT	3,000
Módulo OCR	Preprocesamiento de imagen, integración del modelo, extracción de métricas caligráficas	1,500
Módulo NLP	Tokenización, análisis ortográfico, generación de sugerencias	1,200
Base de datos (SQL Server)	DDL (tablas, relaciones, restricciones), procedimientos almacenados	800
Total		10,000

3.2.3. Aplicación del modelo COCOMO I Básico

A partir del total estimado de 10 KLOC, se aplicaron las fórmulas del modelo COCOMO I en modo orgánico para calcular el esfuerzo, el tiempo de desarrollo y el número de personas requeridas. Las fórmulas utilizadas son las siguientes:

Las fórmulas utilizadas son las siguientes:

a) Esfuerzo (E):

$$E = 2,4 \times (KLOC)^{1,05}$$

b) Tiempo de desarrollo (D):

$$D = 2,5 \times (E)^{0,38}$$

c) Personas requeridas (P):

$$P = \frac{E}{D}$$

Los resultados de la aplicación del modelo se presentan en la Tabla ??.

Tabla 6: Resultados del modelo COCOMO I Básico aplicado al sistema.

Parámetro	Fórmula aplicada	Resultado
Tamaño del proyecto	—	10 KLOC
Esfuerzo (E)	$2,4 \times (10)^{1,05}$	26.93 personas-mes
Tiempo de desarrollo (D)	$2,5 \times (26,93)^{0,38}$	8.74 meses
Personas requeridas (P)	$\frac{26,93}{8,74}$	$3,08 \approx 3$ personas

3.2.4. Estimación del costo comercial equivalente

Para determinar el costo que representaría el desarrollo del sistema en un entorno comercial, se utilizó como referencia el salario promedio mensual de un desarrollador de software de nivel junior a intermedio en México para el año 2025, estimado en \$20,000 MXN según datos del mercado laboral nacional [?].

El costo comercial equivalente se calcula de la siguiente manera:

$$Costo_{comercial} = E \times Salario_{mensual} = 26,93 \times \$20,000 = \$538,000 \text{ MXN}$$

Este valor representa el costo hipotético de contratar a un equipo de desarrolladores externos para construir un sistema equivalente, y sirve como referencia para evaluar el ahorro generado por el desarrollo académico.

En un escenario de despliegue productivo futuro, mediante contenedores Docker en plataformas como Google Cloud Platform o Amazon Web Services, el costo de infraestructura mensual estimado oscilaría entre 50y100 USD según el nivel de tráfico, lo que no compromete la factibilidad del presente proyecto dado que la prueba piloto se mantiene dentro del tier gratuito disponible. [?] [?]

3.3. Factibilidad Legal

La implementación del sistema requiere observar un conjunto de disposiciones jurídicas del ordenamiento mexicano que regulan directamente tres ámbitos: la protección de datos personales de menores de edad, los derechos de niñas, niños y adolescentes en entornos tecnológicos, y la propiedad intelectual del software académico. A continuación se analiza cada uno de estos marcos normativos y su aplicación concreta al sistema propuesto.

A. Protección de datos personales de menores de edad

El tratamiento de los datos generados por el sistema — incluyendo muestras de escritura manuscrita, credenciales de acceso, resultados de análisis y métricas de desempeño — constituye tratamiento de datos personales en los términos de la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP), publicada originalmente en el Diario Oficial de la Federación el 5 de julio de 2010 y actualizada mediante la versión publicada el 20 de marzo de 2025, vigente a partir del 21 de marzo de 2025 [?].

Dado que los usuarios finales son estudiantes de tercer y cuarto grado de educación primaria (entre 8 y 10 años de edad), se trata de datos personales de menores de edad, categoría que la ley somete a un régimen reforzado de protección. Los artículos específicos del ordenamiento que aplican al presente sistema son los siguientes:

Artículo 7 (Consentimiento para menores de edad).

La LFPDPPP establece que en el tratamiento de datos personales de menores de edad se deberá privilegiar el interés superior de la niña, el niño y el adolescente [?]. En concordancia con ello, la reforma de 2025 obliga expresamente a que el consentimiento para tratar datos de menores sea otorgado por padres, madres o tutores legales [?]. El sistema atiende este requerimiento mediante la figura del Tutor como actor principal del registro: únicamente el tutor autenticado puede dar de alta alumnos en la plataforma (RN-02), vinculando cada alumno a un responsable legal que actúa como representante en el tratamiento de sus datos (RN-03). Esto satisface el requisito de consentimiento informado, específico y libre exigido por la ley.

Artículo 19 y 20 (Medidas de seguridad técnica, administrativa y física).

La LFPDPPP obliga al responsable del tratamiento a implementar medidas de seguridad administrativas, técnicas y físicas que protejan los datos personales contra daño, pérdida, alteración, destrucción o acceso no autorizado [?]. El sistema da cumplimiento a este requerimiento mediante tres mecanismos: (1) autenticación basada en tokens JWT que impiden el acceso no autorizado a los datos de cada usuario según su rol; (2) almacenamiento de contraseñas mediante algoritmos de hash BCrypt (campo `contrasena_cifrada` de tipo VARCHAR(255) en la tabla Usuarios); y (3) una arquitectura de separación de roles que garantiza que un alumno solo puede visualizar sus propios resultados (RN-25) y que únicamente el tutor responsable puede consultar y modificar los resultados de sus alumnos (RN-24).

Derechos ARCO (Artículos 28–37).

La ley reconoce a los titulares los derechos de Acceso, Rectificación, Cancelación y Oposición (ARCO) respecto de sus datos personales [?]. Para el ejercicio de estos derechos por parte de menores de edad, la ley dispone que deberá realizarse a través de su representante legal [?]. En el sistema, el tutor puede modificar los resultados de ejercicios de

sus alumnos (RU-18) y el diseño contempla bajas lógicas en lugar de eliminación física de registros (RN-09), lo que permite la cancelación de datos sin comprometer la integridad referencial del sistema.

Aviso de privacidad.

El sistema deberá incorporar un aviso de privacidad accesible dentro de la interfaz, con lenguaje claro y adaptado al nivel educativo de los usuarios, informando sobre el responsable del tratamiento, las finalidades del uso de los datos, los mecanismos para el ejercicio de los derechos ARCO y el periodo de conservación de la información, en cumplimiento de los principios de licitud, información y proporcionalidad establecidos en la LFPDPPP [?].

B. Derechos de niñas, niños y adolescentes en entornos tecnológicos

La Ley General de los Derechos de Niñas, Niños y Adolescentes (LGDNNA), publicada en el Diario Oficial de la Federación el 4 de diciembre de 2014 y reformada por última vez el 27 de mayo de 2024, establece un catálogo de 20 derechos fundamentales aplicables a todas las personas menores de 18 años en territorio mexicano [?].

Dos artículos de esta ley resultan directamente relevantes para el presente proyecto:

Artículo 13, fracción XX (Derecho de acceso a las Tecnologías de la Información y Comunicación).

La LGDNNA reconoce expresamente el derecho de niñas, niños y adolescentes al acceso a las tecnologías de la información y comunicación, así como a los servicios de radiodifusión y telecomunicaciones, incluido el de banda ancha e internet, sin discriminación de ningún tipo o condición [?]. El presente sistema se alinea con este derecho al constituir una herramienta educativa digital orientada a fortalecer competencias de escritura, ampliando las posibilidades de acceso a recursos tecnológicos dentro del entorno escolar de educación básica.

Artículo 18 (Interés superior de la niñez como principio rector).

La ley establece que en todas las medidas concernientes a niñas, niños y adolescentes se tomará en cuenta como consideración primordial su interés superior [?]. Este principio se materializa en el diseño pedagógico del sistema: los ejercicios están basados en los programas de estudio de la SEP para tercer y cuarto grado, la retroalimentación automatizada es constructiva y no punitiva, y la interfaz está diseñada para minimizar la carga cognitiva y la frustración del estudiante.

C. Propiedad intelectual y licencias de software

El desarrollo del sistema se enmarca en el régimen de Trabajo Terminal de la Escuela Superior de Cómputo (ESCOM) del Instituto Politécnico Nacional, cuyas normativas institucionales establecen que los derechos patrimoniales de los productos académicos generados corresponden a la institución, mientras que los autores conservan sus derechos morales [?].

Respecto al uso de herramientas de terceros, todas las tecnologías seleccionadas para el desarrollo cuentan con licencias compatibles con fines académicos y no comerciales:

- **Tesseract OCR:** Licencia Apache 2.0, que permite uso, modificación y distribución sin restricciones para fines académicos.
- **FastAPI:** Licencia MIT, de uso libre sin condiciones.
- **React:** Licencia MIT.
- **Dataset Letras Alfabeto Español (Kaggle – Herrera, 2023):** Publicado bajo licencia Creative Commons, compatible con uso académico.
- **EMNIST:** Publicado por el NIST como dataset de acceso público para investigación.

El uso de estos componentes bajo sus respectivas licencias no genera obligaciones comerciales ni conflictos de propiedad intelectual con el marco institucional del proyecto.

D. Normatividad educativa

El contenido pedagógico de los ejercicios y el análisis ortográfico del sistema se fundamentan en los programas de estudio de la Secretaría de Educación Pública (SEP) para educación básica — específicamente en los libros *Nuestros Saberes* de tercer y cuarto grado (ciclo escolar 2025–2026) — lo que valida legalmente su pertinencia curricular dentro del sistema educativo nacional.

E. Síntesis de cumplimiento normativo

En la Tabla ?? se presenta un resumen del cumplimiento normativo del sistema respecto a las disposiciones legales identificadas como aplicables.

Tabla 7: Síntesis de cumplimiento normativo del sistema.

Marco normativo	Artículo(s) aplicable(s)	Mecanismo de cumplimiento en el sistema
LFPDPPP (2025)	Art. 7 — Consentimiento de menores	Registro de alumnos exclusivamente por el tutor responsable (RN-02, RN-03)
LFPDPPP (2025)	Arts. 19–20 — Seguridad técnica	JWT, BCrypt, separación de roles (RN-24, RN-25)
LFPDPPP (2025)	Arts. 28–37 — Derechos ARCO	Modificación de resultados por tutor (RU-18), baja lógica (RN-09)
LGDNNA (2024)	Art. 13 fracc. XX — Acceso a TIC	Herramienta digital educativa de acceso supervisado
LGDNNA (2024)	Art. 18 — Interés superior	Diseño pedagógico basado en programas SEP, retroalimentación constructiva
Reglamento ESCOM-IPN	Propiedad intelectual académica	Derechos patrimoniales al IPN, derechos morales a los autores
Licencias de software de terceros	Apache 2.0, MIT, CC	Uso académico sin restricciones en todos los componentes
Programas SEP 3.º y 4.º grado	Plan de Estudios 2017 / Nuestros Saberes 2024	Ejercicios diseñados con base en contenidos curriculares oficiales

En razón de lo anterior, el proyecto se considera legalmente factible. El diseño del sistema contempla de forma explícita los requerimientos normativos vigentes en materia de protección de datos de menores, derechos digitales de la infancia y propiedad intelectual académica, sin que ninguna disposición identificada represente un impedimento para su desarrollo o despliegue en un entorno educativo supervisado.

3.4. Requerimientos del usuario.

En esta sección se describen los requerimientos del usuario, los cuales representan las expectativas y servicios que esperan los usuarios finales de la aplicación web. Estos requerimientos han sido identificados con base al análisis del problema, con el objetivo de propiciar que la solución desarrollada sea útil.

Los requerimientos del usuario se definen, de manera general, qué debe hacer la aplicación web desde la perspectiva del usuario en un lenguaje natural, sin entregar detalles técnicos de implementación. Estos servirán como base para la definición de requerimientos funcionales y no funcionales.

Tabla 8: Requerimientos del usuario.

ID	Nombre	Descripción	Prioridad
RU-01	Registro de usuarios.	La aplicación web debe permitir el registro de usuarios. El tutor se registra de forma autónoma, mientras que los alumnos son registrados por el tutor, asociándolos a su grado y grupo.	Alta
RU-02	Autenticación de usuario.	El usuario debe poder autenticarse en la aplicación web para acceder a sus funcionalidades de acuerdo con su rol.	Alta
RU-03	Visualización de estado de alumnos.	El tutor debe poder visualizar el estado de desempeño de sus alumnos, clasificándolos como bajo desempeño, desempeño regular y desempeño de excelencia, con el fin de identificar su progreso.	Alta
RU-04	Clasificación de alumnos.	La aplicación web debe mostrar un gráfico con la cantidad de alumnos clasificados según su nivel de desempeño (bajo, regular y excelencia), actualizándose conforme a los resultados obtenidos en ortografía y legibilidad.	Media

ID	Nombre	Descripción	Prioridad
RU-05	Visualización de ejercicios.	La aplicación web debe permitir observar los ejercicios disponibles.	Alta
RU-06	Activación de ejercicios.	El tutor debe poder activar ejercicios a su elección, estableciendo una fecha límite de entrega cuando lo considere necesario.	Alta
RU-07	Visualización de ejercicios seleccionados.	La aplicación web debe mostrar los ejercicios seleccionados por el tutor y su fecha límite de entrega.	Media
RU-08	Visualización de ejercicios pendientes.	El alumno debe poder observar los ejercicios asignados y su fecha límite de entrega, para una correcta organización y cumplimiento de sus actividades en tiempo.	Alta
RU-09	Visualización de ejercicios vencidos por tutor.	La aplicación web debe permitir al tutor consultar los ejercicios cuya fecha límite ha expirado, para poder extender el plazo de entrega si lo considera necesario.	Media
RU-10	Visualización de ejercicios vencidos por alumno.	El alumno debe poder observar los ejercicios que no completó a tiempo, para conocer su estado y saber si el tutor ya habilitó su entrega.	Media
RU-11	Realizar intento de ejercicios pendientes.	El alumno debe poder realizar el ejercicio seleccionado por el tutor en una vista de captura de escritura.	Alta

ID	Nombre	Descripción	Prioridad
RU-12	Escritura digital en lienzo.	El alumno debe poder escribir directamente sobre un lienzo digital, contando con herramientas de goma y deshacer para corregir su trazo, con el fin de realizar el ejercicio de escritura manuscrita desde el dispositivo.	Alta
RU-13	Carga de fotografía como alternativa.	El alumno debe poder subir una fotografía de su escritura manuscrita en papel como alternativa al lienzo digital, para adaptarse a distintas condiciones de acceso.	Alta
RU-14	Envío del ejercicio para revisión.	El alumno debe poder enviar su ejercicio a la aplicación web para que sea analizado, una vez que considere que su escritura está lista.	Alta
RU-15	Visualización de ejercicios completados.	El alumno debe poder observar los ejercicios completados, así como sus resultados de escritura.	Media
RU-16	Visualización de ejercicios completados por alumno.	La aplicación web debe permitir al tutor observar las actividades realizadas por cada alumno, incluyendo sus resultados.	Media
RU-17	Modificación de resultados.	El tutor debe poder modificar los resultados de un ejercicio realizado por el alumno, en caso de no estar de acuerdo con el análisis ortográfico y caligráfico.	Baja
RU-18	Visualización de actividades sugeridas.	El alumno debe poder observar las actividades sugeridas por la aplicación web, correspondientes a sus análisis ortográficos y caligráficos.	Media

ID	Nombre	Descripción	Prioridad
RU-19	Realizar actividades sugeridas.	El alumno debe poder realizar las actividades sugeridas por la aplicación web, para trabajar en la mejora de los aspectos detectados en su escritura.	Media

3.5. Reglas de negocio

Tabla 9: Reglas de negocio.

ID	Nombre	Descripción
RN-01	Registro de tutores	Todo usuario que actúe como tutor debe estar registrado en el sistema con el rol de tutor.
RN-02	Registro de alumnos	Todo alumno debe estar registrado en el sistema para poder realizar intentos y consultar resultados.
RN-03	Tutor único por alumno	Cada alumno debe tener exactamente un tutor asignado.
RN-04	Tutor con múltiples alumnos	Un tutor puede estar asociado a uno o más alumnos.
RN-05	Correo obligatorio para tutores	Todo tutor debe proporcionar un correo electrónico válido.
RN-06	Integridad tutor-alumno	No se permite registrar un alumno sin asignarlo a un tutor existente.
RN-07	Persistencia de datos	La información de alumnos, tutores, intentos y resultados debe conservarse de manera persistente.
RN-08	Baja lógica de usuarios	La eliminación de usuarios debe realizarse de forma lógica (cambio de estado) y no física.
RN-09	Asociación de intento	Cada intento debe estar asociado a un único alumno y a un único ejercicio.
RN-10	Evidencia obligatoria del intento	Cada intento debe incluir una evidencia asociada.

ID	Nombre	Descripción
RN-11	Registro de fecha y hora	Cada intento debe almacenar la fecha y hora en que fue enviado.
RN-12	Análisis caligráfico único	Cada intento debe generar exactamente un análisis caligráfico.
RN-13	Análisis ortográfico único	Cada intento debe generar exactamente un análisis ortográfico.
RN-14	Generación automática de análisis	Los análisis del intento deben generarse automáticamente a partir del contenido del intento.
RN-15	Métricas caligráficas mínimas	El análisis caligráfico debe evaluar, como mínimo: alineación, tamaño, inclinación y espaciado.
RN-16	Rango de métricas caligráficas	Cada métrica caligráfica debe tener un valor dentro del rango de 0 a 10.
RN-17	Observaciones del análisis	El análisis caligráfico puede incluir observaciones adicionales generadas por el sistema.
RN-18	Detección de errores ortográficos	El análisis ortográfico debe detectar errores ortográficos en el texto del intento.
RN-19	Conteo de errores	El sistema debe almacenar el conteo total de errores ortográficos detectados por intento.
RN-20	Sugerencias de corrección	El sistema debe generar sugerencias de corrección ortográfica.
RN-21	Puntuación ortográfica	El análisis ortográfico debe asignar una puntuación ortográfica basada en los errores detectados.
RN-22	Historial y progreso del alumno	El historial y el progreso del alumno deben generarse a partir de los datos de análisis de sus intentos.
RN-23	Consulta por tutor	Un tutor puede consultar los resultados de los alumnos a su cargo.

ID	Nombre	Descripción
RN-24	Modificación por tutor	Un tutor puede modificar los resultados de los alumnos a su cargo únicamente si la política del sistema lo permite y queda registro del cambio.
RN-25	Consulta por alumno	Un alumno solo puede visualizar sus propios resultados.
RN-26	Condición para ejercicio correctivo	Un ejercicio correctivo automático solo puede generarse si el conteo de errores del intento es menor que 5.
RN-27	Disponibilidad de impresión	Todo ejercicio debe poder renderizarse para visualización e impresión/exportación.

3.6. Modelado de casos de uso

El modelado de casos de uso se utiliza ampliamente para apoyar la adquisición de requerimientos. Un caso de uso puede tomarse como un simple escenario que describa lo que espera el usuario de un sistema.

Cada caso de uso representa una tarea discreta que implica interacción externa con un sistema. En su forma más simple, un caso de uso se muestra como una elipse, con los actores que intervienen en el caso de uso representados como figuras humanas [?].

3.6.1. Diagrama de casos de uso

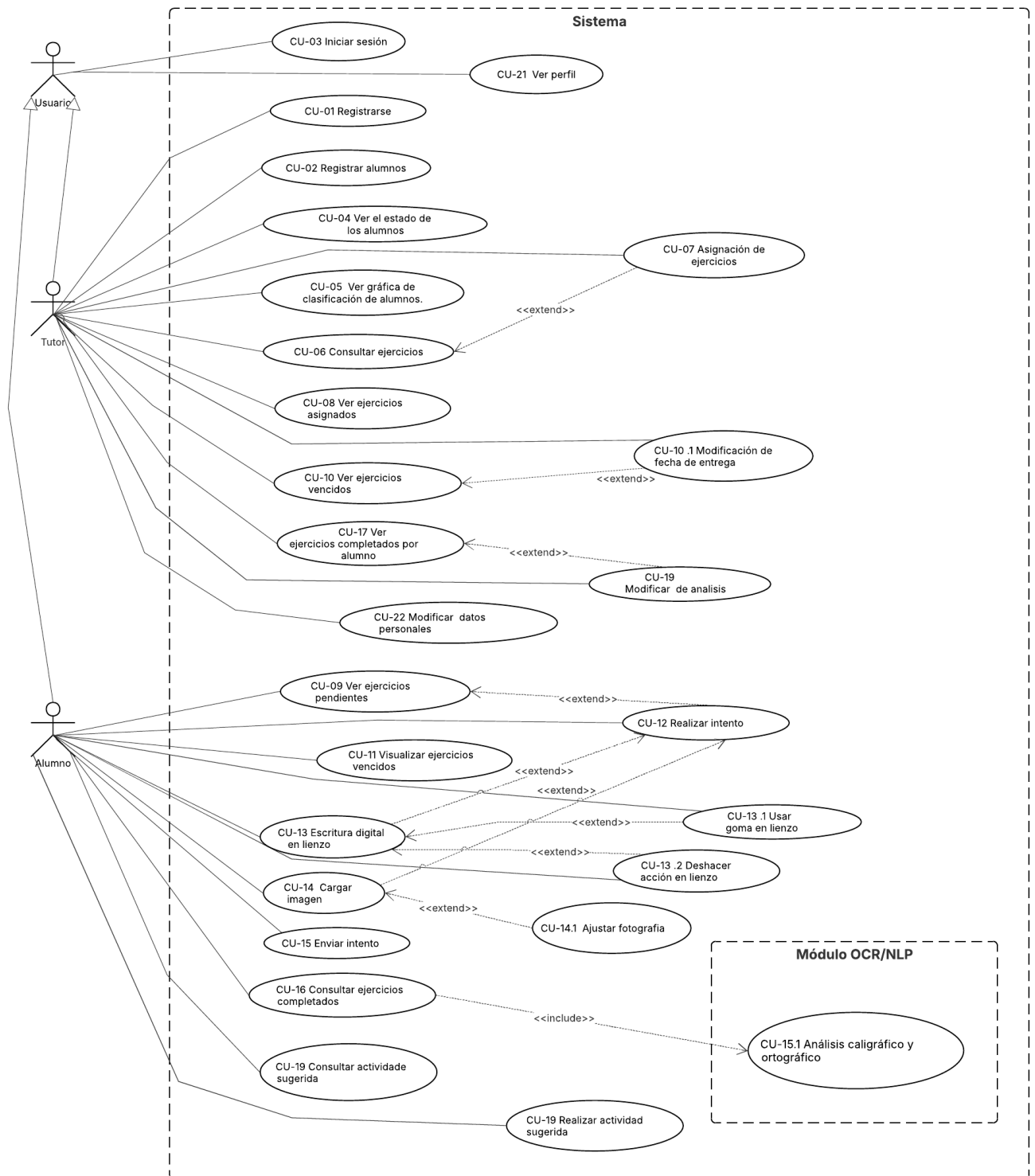


Figura 4: Diagrama de casos de uso

3.6.2. Especificación de los casos de uso

Tabla 10: Tabla de especificación CU-01

CU-01 Registro de tutor	
Actor	Tutor
Descripción	El tutor ingresa sus datos personales en el formulario de la aplicación web para crear su cuenta y acceder a sus funcionalidades.
Datos	Nombre, apellido paterno, nombre de usuario, correo electrónico, contraseña.
Estímulo	El tutor ingresa a la vista de registro, ingresa sus datos en el formulario y los envía.
Respuesta	La aplicación web valida los datos, crea la cuenta del tutor y le permite acceder a las funcionalidades del tutor.
Comentarios	El correo electrónico y nombre de usuario debe ser único en la aplicación web.

Tabla 11: Tabla de especificación CU-02

CU-02 Registrar alumno	
Actor	Tutor
Descripción	El tutor registra a un alumno en la aplicación web, ingresando sus datos e indicando el grado y grupo al que pertenece.
Datos	Nombre, apellido paterno, apellido materno, nombre de usuario, contraseña, grado y grupo.
Estímulo	El tutor, ya autenticado, ingresa a la vista “Alumnos”, captura los datos en el formulario “Alta de alumno” y los envía.
Respuesta	La aplicación web registra los datos del alumno y lo asocia con el tutor responsable.
Comentarios	El nombre de usuario del alumno debe ser único. Solo tutores registrados pueden dar de alta alumnos.

Tabla 12: Tabla de especificación CU-03

CU-03 Iniciar sesión	
Actor	Tutor, Alumno
Descripción	El usuario ingresa sus credenciales para autenticarse y acceder a las funcionalidades según su rol.
Datos	Nombre de usuario y contraseña.

CU-03 Iniciar sesión

Estímulo	El usuario accede a la aplicación e ingresa sus credenciales en el formulario de inicio de sesión.
Respuesta	La aplicación valida las credenciales, identifica el rol y redirige a la vista correspondiente.
Comentarios	Si las credenciales son incorrectas, se muestra un mensaje de error.

Tabla 13: Tabla de especificación CU-04

CU-04 Ver el estado de los alumnos

Actor	Tutor
Descripción	El tutor consulta el estado de desempeño de sus alumnos clasificados como bajo, regular y excelencia.
Datos	Lista de alumnos clasificados y métricas de progreso.
Estímulo	El tutor autenticado accede a su panel principal.
Respuesta	La aplicación muestra la lista de alumnos agrupados por categoría de desempeño.
Comentarios	Solo se muestran alumnos asociados al tutor autenticado.

Tabla 14: Tabla de especificación CU-05

CU-05 Ver gráfica de clasificación de alumnos

Actor	Tutor
Descripción	La aplicación presenta al tutor un gráfico con la distribución de alumnos según su nivel de desempeño.
Datos	Cantidad de alumnos en bajo, regular y excelencia.
Estímulo	El tutor accede al panel principal donde se muestra el gráfico.
Respuesta	La aplicación genera y actualiza dinámicamente el gráfico.
Comentarios	Puede mostrarse mediante gráfica de barras o anillo.

Tabla 15: Tabla de especificación CU-06

CU-06 Consultar ejercicios

Actor	Tutor
Descripción	La aplicación presenta al tutor el catálogo de ejercicios disponibles para asignar.
Datos	Nombre, descripción y contenido base del ejercicio.

CU-06 Consultar ejercicios	
Estímulo	El tutor accede a la sección de ejercicios.
Respuesta	La aplicación despliega el catálogo de ejercicios disponibles.
Comentarios	El tutor no crea ejercicios, solo los consulta y selecciona.

Tabla 16: Tabla de especificación CU-07

CU-07 Asignación de ejercicios	
Actor	Tutor
Descripción	El tutor selecciona uno o más ejercicios del catálogo y los activa para sus alumnos.
Datos	Ejercicios seleccionados, fecha límite opcional.
Estímulo	El tutor selecciona ejercicios y confirma la activación.
Respuesta	La aplicación registra los ejercicios activos y los publica a los alumnos.
Comentarios	La fecha límite es opcional.

Tabla 17: Tabla de especificación CU-08

CU-08 Ver ejercicios asignados	
Actor	Tutor
Descripción	El tutor consulta la lista de ejercicios activados y sus fechas límite.
Datos	Ejercicios activos y fecha de entrega.
Estímulo	El tutor accede al módulo de ejercicios asignados.
Respuesta	La aplicación muestra los ejercicios seleccionados por el tutor.
Comentarios	Debe distinguir ejercicios con y sin fecha límite.

Tabla 18: Tabla de especificación CU-09

CU-09 Ver ejercicios pendientes	
Actor	Alumno
Descripción	El alumno consulta los ejercicios pendientes asignados por el tutor.
Datos	Nombre del ejercicio, descripción y fecha límite.
Estímulo	El alumno accede a la sección de ejercicios pendientes.
Respuesta	La aplicación muestra la lista de ejercicios activos no completados.

	CU-09 Ver ejercicios pendientes
Comentarios	Solo se muestran ejercicios vigentes.

Tabla 19: Tabla de especificación CU-10

	CU-10 Visualización de ejercicios vencidos
Actor	Tutor
Descripción	El tutor consulta la lista de ejercicios cuya fecha límite de entrega ha expirado, con el fin de decidir si extiende el plazo para uno o más de ellos.
Datos	Ejercicios seleccionados vencidos, fecha de entrega.
Estímulo	El tutor accede a la sección de ejercicios desde su panel de gestión y filtra los ejercicios seleccionados.
Respuesta	La aplicación web muestra únicamente los ejercicios activados por ese tutor, indicando para cada uno su fecha límite de entrega si fue establecida.
Comentarios	Debe distinguirse visualmente si un ejercicio fue seleccionado y ya está vencido.

Tabla 20: Tabla de especificación CU-11

	CU-11 Visualización de ejercicios vencidos
Actor	Alumno
Descripción	El alumno consulta los ejercicios vencidos y no completados para conocer su estado y verificar si el tutor ha extendido el plazo de entrega.
Datos	Lista de ejercicios vencidos con nombre, fecha límite original y estado de habilitación por parte del tutor.
Estímulo	El alumno, autenticado en la aplicación web, accede a la sección de ejercicios vencidos desde su panel principal.
Respuesta	La aplicación web muestra los ejercicios no completados a tiempo, indicando si el tutor ha habilitado o no su entrega posterior.
Comentarios	Se debe distinguir entre ejercicios vencidos sin extensión y aquellos que el tutor habilitó nuevamente.

Tabla 21: Tabla de especificación CU-12

CU-12 Realizar intento de ejercicio pendiente	
Actor	Alumno
Descripción	El alumno selecciona un ejercicio pendiente asignado por el tutor y accede a la vista de captura de escritura para realizarlo.
Datos	Datos del ejercicio seleccionado.
Estímulo	El alumno elige un ejercicio de su lista de pendientes y confirma que desea realizarlo.
Respuesta	La aplicación web carga la vista de captura de escritura correspondiente al ejercicio seleccionado, habilitando las herramientas para que el alumno lo complete.
Comentarios	Solo se pueden realizar ejercicios de plazo vigente o extensión habilitada.

Tabla 22: Tabla de especificación CU-13

CU-13 Escritura digital en lienzo	
Actor	Alumno
Descripción	El alumno realiza el ejercicio de escritura manuscrita directamente sobre un lienzo digital cuadriculado, utilizando herramientas de la aplicación web; como goma y deshacer para corregir su trazo.
Datos	Trazos digitales, acciones de corrección, datos del ejercicio seleccionado.
Estímulo	El alumno accede a la vista de captura (desde CU-12) y comienza a escribir sobre el lienzo digital con su dispositivo.
Respuesta	La aplicación web registra los trazos del alumno sobre el lienzo digital, habilitando las herramientas de corrección para que pueda corregir su trazo antes de enviarlo.
Comentarios	El lienzo debe ser compatible con entrada táctil o con lápiz digital según el dispositivo. Esta opción es alternativa a CU-14 (carga de fotografía); el alumno usa una u otra.

Tabla 23: Tabla de especificación CU-13.1

CU-13.1 Uso de goma en lienzo	
Actor	Alumno

CU-13.1 Uso de goma en lienzo	
Descripción	El alumno utiliza la herramienta de goma para corregir errores en su escritura sobre el lienzo digital antes de enviar el ejercicio.
Datos	Área o trazo seleccionado para borrar.
Estímulo	El alumno, dentro de la vista del lienzo (CU-13), identifica un error en una parte específica de su escritura, selecciona la herramienta de goma y arrastra sobre el trazo que desea eliminar.
Respuesta	La aplicación web elimina únicamente el área o trazo que el alumno cubrió con la goma.
Comentarios	La goma permite una corrección selectiva y precisa.

Tabla 24: Tabla de especificación CU-13.2

CU-13.2 Deshacer acción en lienzo	
Actor	Alumno
Descripción	El alumno utiliza la acción de deshacer para revertir el último trazo completo registrado en el lienzo digital.
Datos	Historial de trazos registrados en el lienzo, último trazo a revertir, estado del lienzo tras la acción de deshacer.
Estímulo	El alumno, dentro de la vista del lienzo (CU-13), identifica que su última acción en el lienzo fue incorrecta y presiona el botón de “Deshacer”.
Respuesta	La aplicación web revierte la última acción realizada por el alumno.
Comentarios	Puede ejecutarse de forma consecutiva para revertir múltiples acciones.

Tabla 25: Tabla de especificación CU-14

CU-14 Cargar imagen como alternativa	
Actor	Alumno
Descripción	El alumno sube una imagen como alternativa al lienzo digital, ya sea tomando una foto con la cámara del dispositivo o subiendo un archivo de imagen guardado.

CU-14 Cargar imagen como alternativa	
Datos	Archivo de imagen (fotografía o imagen digital de la escritura manuscrita en formatos JPG, PNG o similar), ejercicio al que corresponde el intento.
Estímulo	El alumno accede a la vista de captura (desde CU-12), selecciona la opción de subir foto y elige “Tomar foto” o “Subir archivo”.
Respuesta	La aplicación web recibe la imagen cargada, muestra una vista previa para que el alumno la confirme y la asocia al intento del ejercicio, dejándola lista para su envío a revisión.
Comentarios	Es recomendable validar el tamaño y calidad mínima de la imagen. Esta opción es alternativa a CU-13; el alumno usa una u otra desde la misma vista de captura.

Tabla 26: Tabla de especificación CU-14.1

CU-14.1 Ajustar fotografía	
Actor	Alumno
Descripción	El alumno ajusta la fotografía antes de confirmarla.
Datos	Fotografía capturada en el momento, ajustes aplicados (recorte, rotación).
Estímulo	El alumno, tras tomar la foto con la cámara (CU-14), visualiza la vista previa y decide ajustarla antes de confirmarla.
Respuesta	La aplicación web aplica los ajustes y muestra la imagen corregida en la vista previa, lista para ser enviada a revisión.
Comentarios	Este caso solo aplica cuando el alumno eligió “Tomar foto”.

Tabla 27: Tabla de especificación CU-15

CU-15 Enviar intento	
Actor	Alumno
Descripción	El alumno envía su ejercicio completado a la aplicación web para que sea analizado, una vez que considera que su escritura está lista, usando el botón de envío disponible en la misma vista de captura.
Datos	Intento del ejercicio (imagen cargada), ejercicio al que corresponde, alumno autenticado, fecha y hora de envío.

CU-15 Enviar intento	
Estímulo	El alumno revisa su escritura en la vista de captura y presiona el botón “Enviar Revisión”.
Respuesta	La aplicación web registra el envío, marca el ejercicio como entregado, lo pone a disposición de los modelos de análisis y confirma al alumno que el envío fue exitoso.
Comentarios	Una vez enviado, el ejercicio no puede modificarse. Este es el último paso de CU-13 o CU-14.

Tabla 28: Tabla de especificación CU-15.1

CU-15.1 Análisis caligráfico y ortográfico	
Actor	Ninguno (proceso interno de la aplicación web)
Descripción	El sistema analiza automáticamente el intento de escritura enviado por el alumno, evaluando aspectos caligráficos y ortográficos mediante los módulos internos de OCR y NLP, con el fin de generar un resultado que refleje el desempeño del alumno.
Datos	Imagen del intento enviado (trazo digital o fotografía), identificador del ejercicio, identificador del alumno, resultado del análisis caligráfico y ortográfico generado por los módulos OCR y NLP.
Estímulo	El módulo de análisis se activa automáticamente una vez que el alumno confirma el envío del ejercicio en CU-15.
Respuesta	El módulo procesa la imagen mediante OCR para extraer el contenido escrito y aplica NLP para evaluar la ortografía y caligrafía. Los resultados del análisis se almacenan y quedan disponibles para el alumno en CU-16 y para el tutor en CU-17.
Comentarios	Este caso de uso es un <i>include</i> de CU-15, ya que el análisis siempre ocurre tras cada envío sin excepción. Al tratarse de un proceso interno, no requiere intervención del usuario.

Tabla 29: Tabla de especificación CU-16

CU-16 Ver ejercicios completados	
Actor	Alumno

CU-16 Ver ejercicios completados	
Descripción	El alumno consulta el historial de ejercicios que ha completado y enviado, junto con los resultados del análisis ortográfico y caligráfico obtenidos en cada uno.
Datos	Lista de ejercicios completados con nombre del ejercicio, fecha de entrega y resultados de escritura (análisis ortográfico y caligráfico).
Estímulo	El alumno autenticado, accede a la sección de ejercicios completados desde su panel principal.
Respuesta	La aplicación web muestra el historial de ejercicios completados por el alumno, mostrando para cada ejercicio el resultado de su análisis.
Comentarios	Solo se muestran ejercicios que el alumno haya enviado previamente (CU-15). Los resultados deben estar disponibles una vez que la aplicación web haya procesado el análisis del ejercicio enviado.

Tabla 30: Tabla de especificación CU-17

CU-17 Ver ejercicios completados por alumno	
Actor	Tutor
Descripción	El tutor consulta las actividades realizadas por cada alumno, incluyendo los resultados del análisis ortográfico y caligráfico obtenidos en los ejercicios completados.
Datos	Lista de alumnos con sus ejercicios completados, fecha de entrega y resultados de escritura correspondientes a cada intento.
Estímulo	El tutor accede a la vista de alumnos y selecciona al alumno que desea consultar.
Respuesta	La aplicación web muestra el historial de ejercicios completados por el alumno seleccionado, incluyendo los resultados del análisis de escritura para cada uno.
Comentarios	Solo se muestran alumnos vinculados al tutor autenticado. Este caso de uso es el punto de entrada a CU-18, donde el tutor puede modificar los resultados si no está de acuerdo con el análisis.

Tabla 31: Tabla de especificación CU-18

CU-18 Modificar análisis	
Actor	Tutor
Descripción	El tutor modifica los resultados del análisis ortográfico y caligráfico de un ejercicio realizado por un alumno, en caso que el análisis de la aplicación web no refleja correctamente su desempeño.
Datos	Ejercicio seleccionado, resultados originales del análisis automático, correcciones o ajustes realizados por el tutor, alumno al que corresponde el intento.
Estímulo	El tutor accede a la vista de alumnos, selecciona al alumno que desea consultar, el ejercicio completado y elige la opción de modificar análisis.
Respuesta	La aplicación guarda los cambios realizados por el tutor, actualiza los resultados del ejercicio y los refleja en la vista del alumno (CU-16).
Comentarios	Solo el tutor responsable del alumno puede modificar sus resultados.

Tabla 32: Tabla de especificación CU-19

CU-19 Consultar actividades sugeridas	
Actor	Alumno
Descripción	El alumno consulta las actividades de mejora sugeridas por la aplicación web, recomendadas a partir de los resultados de sus análisis ortográficos y caligráficos previos.
Datos	Lista de recomendaciones con actividades sugeridas (nombre de la actividad, descripción y área de mejora asociada ortografía o caligrafía), vinculados al intento realizado.
Estímulo	El alumno, accede a la sección de actividades sugeridas desde su panel principal.
Respuesta	La aplicación web muestra las actividades recomendadas personalizadas para el alumno, basadas en las áreas de oportunidad detectadas en sus análisis de escritura.
Comentarios	Si el alumno no tiene ejercicios completados aún, esta sección puede estar vacía o mostrar un mensaje orientador.

Tabla 33: Tabla de especificación CU-20

CU-20 Realizar actividades sugeridas	
Actor	Alumno
Descripción	El alumno realiza una actividad sugerida por la aplicación web con el fin de trabajar en la mejora de los aspectos ortográficos y caligráficos identificados en sus análisis previos.
Datos	Actividad sugerida seleccionada, área de mejora asociada (ortografía o caligrafía).
Estímulo	El alumno, desde la sección de actividades sugeridas (CU-19), selecciona una actividad y confirma que desea realizarla.
Respuesta	La aplicación web carga la actividad seleccionada y registra el progreso del alumno al completarla, actualizando su historial de mejora.
Comentarios	Las actividades sugeridas son distintas a los ejercicios asignados por el tutor.

Tabla 34: Tabla de especificación CU-21

CU-21 Ver perfil	
Actor	Tutor, Alumno
Descripción	El usuario podrá verificar sus datos personales en la vista de "Ver perfil".
Datos	Datos personales del usuario, registrados en la plataforma.
Estímulo	El usuario desde el panel principal accede a la vista de "Ver perfil".
Respuesta	La aplicación web carga la información del usuario.
Comentarios	Los alumnos no pueden modificar su información.

Tabla 35: Tabla de especificación CU-22

CU-22 Modificar datos personales	
Actor	Tutor
Descripción	El podrá modificar sus datos personales, mostrados en la vista "Ver perfil".
Datos	Datos modificados por el tutor.
Estímulo	El tutor desde la vista "Ver perfil" preciona el botón para editar el campo que decida.

CU-22 Modificar datos personales	
Respuesta	La aplicación web recibe el campo que se va a modificar y actualiza la pagina "Ver perfil"para visualizar sus datos modificados.
Comentarios	Solo el tutor puede modificar su información.

3.7. Requerimientos del sistema.

A continuación en esta sección se encuentran las Tablas ?? y ?? donde se detallan los requerimientos funcionales del sistema y los requerimientos no funcionales, así como sus tablas de especificación de requerimientos.

3.7.1. *Requerimientos funcionales*

Tabla 36: Tabla de requerimientos funcionales del sistema

ID	Nombre	Módulo
RF-01	Registro de tutor	Autenticación y Registro
RF-02	Validación de correo único	
RF-03	Registro de alumnos	
RF-04	Inicio de sesión	
RF-05	Control de acceso por rol	
RF-06	Notificación de credenciales incorrectas	
RF-07	Clasificación de desempeño de alumnos	Gestión de Alumnos
RF-08	Gráfico de distribución de desempeño	
RF-09	Historial de actividades por alumno	
RF-10	Catálogo de ejercicios	Gestión de Ejercicios
RF-11	Asignación de ejercicios	
RF-12	Ejercicios asignados por tutor	
RF-13	Ejercicios pendientes del alumno	
RF-14	Ejercicios vencidos	
RF-15	Extensión de fecha límite	
RF-16	Historial de ejercicios completados	
RF-17	Acceso a vista de captura	Captura de Escritura
RF-18	Captura Multimodal	
RF-19	Captura mediante cámara	
RF-20	Formatos soportados	
RF-21	Herramientas de pre-edición de imagen	

ID	Nombre	Módulo
RF-22	Gestión de estado de captura	
RF-23	Herramientas del lienzo digital	
RF-24	Previsualización y confirmación	
RF-25	Validación de envío vacío	
RF-26	Análisis OCR	Seguimiento y Retroalimentación
RF-27	Anaálisis NLP	
RF-28	Notificación de fallo en análisis	
RF-29	Modificación de resultados	
RF-30	Actividades sugeridas	
RF-31	Realización de actividades sugeridas	

3.7.2. Especificación de requerimientos funcionales

Tabla 37: Especificación de requerimiento RF-01

Requerimiento: La aplicación web permitirá el registro del tutor.	
Identificador: RF-01	Nombre: Registro de tutor.
Prioridad de desarrollo: Alta	
Entrada: El usuario llena el formulario de registro con Nombre, Apellido Paterno, correo, nombre de usuario y contraseña.	Salida: Confirmación de perfil registrado.
Descripción: Permite que los tutores puedan registrarse en la aplicación web. Precondición: El correo y nombre de usuario no deben estar previamente registrados en la aplicación web. Postcondición: Los datos quedan almacenados y disponibles.	

Tabla 38: Especificación de requerimiento RF-02

Requerimiento: La aplicación web notificará al usuario si el correo ya no esta disponible.	
Identificador: RF-02	Nombre: Validación de correo único.
Prioridad de desarrollo: Alta	
Entrada: La aplicación web recibe el correo del formulario de registro.	Salida: Notificación de que el correo ya existe.
Descripción: Notifica al usuario que el correo ya no esta disponible para el registro. Precondición: El usuario ha ingresado un correo electrónico en el formulario de registro o actualización de perfil. Postcondición: La aplicación confirma que el correo no existe en la base de datos y permite continuar con el registro/actualización ó rechaza el correo y muestra un mensaje como “El correo ya se encuentra registrado”.	

Tabla 39: Especificación de requerimiento RF-03

Requerimiento: La aplicación web permitirá al tutor el registro de los alumnos.	
Identificador: RF-03	Nombre: Registro de alumnos.
Prioridad de desarrollo: Alta	
Entrada: El tutor llena el formulario de "Alta de alumno" con los datos del alumno, nombre, apellido paterno, apellido materno (si lo requiere el tutor), contraseña temporal, grado y grupo.	Salida: Confirmación de alumno registrado.
Descripción: Permite que los tutores puedan registrar a sus alumnos en la aplicación web. Los alumnos deben tener únicamente un tutor. Precondición: El tutor ingreso los datos del alumno en el formulario "Alta de alumno", la aplicación web válida que el alumno no haya sido registrado anteriormente. Postcondición: La aplicación confirma el registro del alumno ó notificá que ya ha sido registrado.	

Tabla 40: Especificación de requerimiento RF-04

Requerimiento: La aplicación web permitirá a tutores y alumnos autenticarse mediante nombre de usuario y contraseña para acceder a sus funcionalidades.	
Identificador: RF-04	Nombre: Inicio de sesión.
Prioridad de desarrollo: Alta	
Entrada: El usuario ingresa su correo electrónico y contraseña en el formulario de inicio de sesión.	Salida: Redirección a la vista correspondiente según el rol del usuario autenticado.
Descripción: Permite que tutores y alumnos inicien sesión en la aplicación web ingresando sus credenciales registradas, identificando su rol y redirigiendo a la vista adecuada para cada uno. Precondición: El usuario debe tener una cuenta registrada en la aplicación web con correo electrónico y contraseña válidos. Postcondición: El usuario queda autenticado y accede a las funcionalidades correspondientes a su rol dentro de la aplicación.	

Tabla 41: Especificación de requerimiento RF-05

Requerimiento: La aplicación web restringirá el acceso a las funcionalidades según el rol del usuario autenticado, diferenciando las vistas y acciones disponibles para tutor y alumno.	
Identificador: RF-05	Nombre: Control de acceso por rol.
Prioridad de desarrollo: Alta	
Entrada: Rol del usuario identificado durante el inicio de sesión.	Salida: Acceso habilitado únicamente a las vistas y acciones correspondientes al rol del usuario autenticado.
Descripción: La aplicación web diferencia las funcionalidades disponibles para cada tipo de usuario. El tutor tiene acceso a la gestión de alumnos, ejercicios y resultados, mientras que el alumno accede únicamente a sus ejercicios, capturas y retroalimentación. Precondición: El usuario debe estar autenticado con un rol asignado (tutor o alumno). Postcondición: El usuario accede exclusivamente a las funcionalidades habilitadas para su rol, sin posibilidad de acceder a las del otro.	

Tabla 42: Especificación de requerimiento RF-06

Requerimiento: La aplicación web notificará al usuario cuando las credenciales ingresadas durante el inicio de sesión sean incorrectas.	
Identificador: RF-06	Nombre: Notificación de credenciales incorrectas.
Prioridad de desarrollo: Media	
Entrada: Nombre de usuario y/o contraseña incorrectos ingresados por el usuario en el formulario de inicio de sesión.	Salida: Mensaje de error indicando que las credenciales son incorrectas.
Descripción: Cuando un usuario ingresa credenciales que no coinciden con ninguna cuenta registrada, la aplicación web muestra un mensaje de error. Precondición: El usuario debe haber ingresado credenciales inválidas. Postcondición: El acceso es denegado.	

Tabla 43: Especificación de requerimiento RF-07

Requerimiento: La aplicación web permitirá al tutor visualizar el estado de desempeño de sus alumnos, clasificándolos en tres categorías: bajo desempeño, desempeño regular y desempeño de excelencia.	
Identificador: RF-07	Nombre: Clasificación de desempeño de alumnos.
Prioridad de desarrollo: Alta	
Entrada: Solicitud del tutor para consultar el estado de desempeño de sus alumnos.	Salida: Lista de alumnos agrupados por categoría de desempeño: bajo desempeño, desempeño regular y desempeño de excelencia.
Descripción: Permite al tutor identificar el nivel de progreso de cada alumno mediante una clasificación basada en los resultados de los análisis ortográficos y caligráficos obtenidos en los ejercicios completados. Precondición: El tutor debe estar autenticado y tener alumnos registrados con al menos un ejercicio analizado. Postcondición: El tutor visualiza a sus alumnos agrupados por categoría de desempeño, con la información actualizada conforme se registren nuevas evaluaciones.	

Tabla 44: Especificación de requerimiento RF-08

Requerimiento: La aplicación web presentará al tutor un gráfico que muestre la distribución de alumnos según su nivel de desempeño, actualizándose automáticamente al registrar nuevas evaluaciones.	
Identificador: RF-08	Nombre: Gráfico de distribución de desempeño.
Prioridad de desarrollo: Media	
Entrada: Datos de clasificación de desempeño de los alumnos del tutor autenticado.	Salida: Gráfico con la cantidad de alumnos en cada categoría de desempeño (bajo, regular y excelencia).
Descripción: La aplicación web genera y muestra al tutor un gráfico con la distribución de sus alumnos según su nivel de desempeño, permitiéndole tener una visión general del grupo de forma visual e intuitiva. Precondición: El tutor debe estar autenticado y tener alumnos con resultados de análisis disponibles. Postcondición: El gráfico se muestra actualizado con la información más reciente y se actualiza automáticamente al registrar nuevas evaluaciones.	

Tabla 45: Especificación de requerimiento RF-09

Requerimiento: La aplicación web permitirá al tutor visualizar las actividades realizadas por cada alumno, incluyendo los resultados del análisis ortográfico y caligráfico obtenidos.	
Identificador: RF-09	Nombre: Historial de actividades por alumno.
Prioridad de desarrollo: Alta	
Entrada: Selección del alumno a consultar por parte del tutor autenticado.	Salida: Historial de ejercicios completados por el alumno seleccionado, con sus resultados de análisis ortográfico y caligráfico.
Descripción: Permite al tutor consultar el historial de actividades de cada uno de sus alumnos, visualizando los ejercicios que han completado junto con los resultados del análisis de escritura correspondientes a cada intento. Precondición: El tutor debe estar autenticado y seleccionar un alumno vinculado a su cuenta que tenga al menos un ejercicio completado. Postcondición: El tutor visualiza el historial completo de actividades del alumno seleccionado con sus resultados de análisis disponibles.	

Tabla 46: Especificación de requerimiento RF-10

Requerimiento: La aplicación web mostrará al tutor el catálogo completo de ejercicios disponibles proporcionados por la plataforma.	
Identificador: RF-10	Nombre: Catálogo de ejercicios.
Prioridad de desarrollo: Alta	
Entrada: Solicitud del tutor para acceder al catálogo de ejercicios desde su panel de gestión.	Salida: Lista de ejercicios disponibles con nombre, descripción y nivel de dificultad.
Descripción: Permite al tutor explorar todos los ejercicios disponibles en la plataforma para conocer las actividades que puede asignar a sus alumnos. El tutor no crea ejercicios, únicamente los consulta y selecciona para asignarlos. Precondición: El tutor debe estar autenticado. Postcondición: El tutor visualiza el catálogo completo de ejercicios y puede proceder a asignar los que considere pertinentes para sus alumnos.	

Tabla 47: Especificación de requerimiento RF-11

Requerimiento: La aplicación web permitirá al tutor asignar ejercicios a sus alumnos, con la opción de establecer una fecha límite de entrega.	
Identificador: RF-11	Nombre: Asignación de ejercicios.
Prioridad de desarrollo: Alta	
Entrada: Ejercicio seleccionado del catálogo y fecha límite de entrega (opcional) ingresada por el tutor.	Salida: Confirmación de asignación y disponibilidad del ejercicio en la lista de pendientes del alumno.
Descripción: Permite al tutor seleccionar uno o más ejercicios del catálogo y asignarlos a sus alumnos. El tutor puede opcionalmente establecer una fecha límite de entrega; si no lo hace, el ejercicio permanece activo indefinidamente. Precondición: El tutor debe estar autenticado y haber consultado el catálogo de ejercicios (RF-10). Postcondición: El ejercicio queda registrado como asignado y aparece en la lista de pendientes de los alumnos correspondientes al tutor.	

Tabla 48: Especificación de requerimiento RF-12

Requerimiento: La aplicación web mostrará al tutor la lista de ejercicios que ha asignado previamente, indicando para cada uno su fecha límite de entrega si fue establecida.	
Identificador: RF-12	Nombre: Ejercicios asignados por tutor.
Prioridad de desarrollo: Alta	
Entrada: Solicitud del tutor para consultar los ejercicios que ha asignado a sus alumnos.	Salida: Lista de ejercicios asignados con nombre del ejercicio y fecha límite de entrega correspondiente.
Descripción: Permite al tutor consultar únicamente los ejercicios que ha asignado a sus alumnos, con la información de la fecha límite de entrega de cada uno. Debe distinguirse visualmente si un ejercicio tiene o no fecha límite asignada. Precondición: El tutor debe estar autenticado y haber asignado al menos un ejercicio previamente (RF-11). Postcondición: El tutor visualiza la lista de ejercicios asignados con su estado y fecha límite correspondiente.	

Tabla 49: Especificación de requerimiento RF-13

Requerimiento: La aplicación web mostrará al alumno los ejercicios pendientes asignados por su tutor, ordenados por fecha límite de entrega y con indicador visual de proximidad de vencimiento.	
Identificador: RF-13	Nombre: Ejercicios pendientes del alumno.
Prioridad de desarrollo: Alta	
Entrada: Solicitud del alumno para consultar sus ejercicios pendientes desde su panel principal.	Salida: Lista de ejercicios pendientes asignados por el tutor, ordenados por fecha límite, con indicador visual de proximidad de vencimiento o vencidos.

Descripción: Permite al alumno visualizar los ejercicios que su tutor ha asignado y que aún no ha completado, junto con su fecha límite de entrega, para organizar sus actividades a tiempo. Solo se muestran ejercicios activos asignados al alumno autenticado.

Precondición: El alumno debe estar autenticado y tener ejercicios asignados por su tutor que no hayan sido completados.

Postcondición: El alumno visualiza sus ejercicios pendientes ordenados por fecha límite, con indicadores visuales que le permiten identificar los próximos a vencer o ya vencidos.

Tabla 50: Especificación de requerimiento RF-14

Requerimiento: La aplicación web permitirá a tutores y alumnos consultar los ejercicios cuya fecha límite de entrega ha expirado.	
Identificador: RF-14	Nombre: Ejercicios vencidos.
Prioridad de desarrollo: Media	
Entrada: Solicitud del usuario (tutor o alumno) para consultar los ejercicios con fecha límite expirada.	Salida: Lista de ejercicios vencidos con nombre, fecha límite original y estado de entrega.
Descripción: Permite a tutores y alumnos visualizar los ejercicios cuya fecha límite ha expirado. El tutor los consulta para decidir si extiende el plazo de entrega, mientras que el alumno los consulta para conocer su estado y verificar si el tutor ha habilitado una extensión. Debe distinguirse visualmente si un ejercicio vencido fue entregado o no. Precondición: El usuario debe estar autenticado y existir ejercicios con fecha límite expirada asociados a su cuenta. Postcondición: El usuario visualiza la lista de ejercicios vencidos con su estado correspondiente. Solo el tutor puede proceder a extender el plazo de entrega (RF-15).	

Tabla 51: Especificación de requerimiento RF-15

Requerimiento: La aplicación web permitirá al tutor extender la fecha límite de entrega de un ejercicio vencido, reactivándolo en la lista de pendientes del alumno.	
Identificador: RF-15	Nombre: Extensión de fecha límite.
Prioridad de desarrollo: Media	
Entrada: Ejercicio vencido seleccionado y nueva fecha límite de entrega ingresada por el tutor.	Salida: Confirmación de actualización de fecha límite y reactivación del ejercicio en la lista de pendientes del alumno.
Descripción: Cuando un ejercicio ha superado su fecha límite, el tutor puede extender el plazo estableciendo una nueva fecha, lo que reactiva el ejercicio y lo vuelve a mostrar como pendiente para los alumnos que no lo han completado. Precondición: El tutor debe estar autenticado, haber consultado los ejercicios vencidos (RF-14) y la nueva fecha debe ser posterior a la fecha actual. Postcondición: El ejercicio queda actualizado con la nueva fecha límite y reaparece en la lista de ejercicios pendientes del alumno.	

Tabla 52: Especificación de requerimiento RF-16

Requerimiento: La aplicación web permitirá al alumno consultar el historial de ejercicios que ha completado y enviado, junto con los resultados del análisis de escritura obtenidos en cada uno.	
Identificador: RF-16	Nombre: Historial de ejercicios completados.
Prioridad de desarrollo: Alta	
Entrada: Solicitud del alumno para consultar su historial de ejercicios completados desde su panel principal.	Salida: Lista de ejercicios completados con nombre, fecha de entrega y resultados del análisis ortográfico y caligráfico.
Descripción: Permite al alumno revisar los ejercicios que ha enviado previamente, visualizando los resultados del análisis de escritura generados para cada uno. Los resultados estarán disponibles una vez que se haya procesado el análisis correspondiente. Precondición: El alumno debe estar autenticado y haber enviado al menos un ejercicio con resultados de análisis disponibles. Postcondición: El alumno visualiza su historial de ejercicios completados con los resultados de análisis ortográfico y caligráfico correspondientes a cada intento.	

Tabla 53: Especificación de requerimiento RF-17

Requerimiento: La aplicación web permitirá al alumno acceder a la vista de captura de escritura al seleccionar un ejercicio pendiente para realizarlo.	
Identificador: RF-17	Nombre: Acceso a vista de captura.
Prioridad de desarrollo: Alta	
Entrada: Selección de un ejercicio pendiente por parte del alumno desde su lista de pendientes.	Salida: Carga de la vista de captura de escritura con las herramientas habilitadas para realizar el ejercicio.
Descripción: Cuando el alumno selecciona un ejercicio pendiente, la aplicación web carga la vista de captura de escritura, habilitando las opciones disponibles para completar el ejercicio: escritura en lienzo digital o carga de imagen. Precondición: El alumno debe estar autenticado y seleccionar un ejercicio dentro del plazo vigente o con extensión habilitada por el tutor. Postcondición: La vista de captura queda cargada y el alumno puede proceder a realizar el ejercicio mediante las opciones de captura disponibles (RF-18).	

Tabla 54: Especificación de requerimiento RF-18

Requerimiento: La aplicación web permitirá la captura de texto manuscrito mediante dos modalidades: carga de archivos de imagen y un lienzo digital integrado para escritura.	
Identificador: RF-18	Nombre: Captura Multimodal.
Prioridad de desarrollo: Alta	
Entrada: Selección de la modalidad de captura por parte del alumno: carga de archivo de imagen o escritura en lienzo digital.	Salida: Habilitación de la modalidad seleccionada dentro de la vista de captura para que el alumno complete el ejercicio.

Descripción: La aplicación web ofrece al alumno dos modalidades de captura de escritura manuscrita dentro de la misma vista, accesibles mediante pestañas:

a) carga de un archivo de imagen con la escritura en papel.

b) escritura directa sobre un lienzo digital integrado.

El alumno utiliza únicamente una de las dos modalidades por intento.

Precondición: El alumno debe haber accedido a la vista de captura de escritura (RF-17).

Postcondición: La modalidad seleccionada queda habilitada y el alumno puede proceder a capturar su escritura manuscrita para enviarla a revisión.

Tabla 55: Especificación de requerimiento RF-19

Requerimiento: La aplicación web permitirá al alumno tomar una fotografía directamente con la cámara de su dispositivo como método alternativo de captura de escritura.	
Identificador: RF-19	Nombre: Captura mediante cámara.
Prioridad de desarrollo: Media	
Entrada: Acción del alumno de activar la cámara del dispositivo desde la modalidad de carga de imagen.	Salida: Fotografía capturada con vista previa disponible para confirmar o repetir la toma antes de adjuntarla al ejercicio.
Descripción: Dentro de la modalidad de carga de imagen (RF-18), el alumno puede optar por tomar una fotografía directamente con la cámara de su dispositivo en lugar de cargar un archivo existente. La aplicación muestra una vista previa de la fotografía capturada para que el alumno la confirme o repita la toma antes de proceder. Precondición: El alumno debe estar en la modalidad de carga de imagen dentro de la vista de captura y el dispositivo debe contar con cámara disponible. Postcondición: La fotografía capturada queda disponible como imagen del intento, lista para ser ajustada o enviada a revisión.	

Tabla 56: Especificación de requerimiento RF-20

Requerimiento: La aplicación web aceptará únicamente imágenes en formato JPEG (.jpeg, .jpg) y PNG (.png) para la carga de escritura manuscrita.	
Identificador: RF-20	Nombre: Formatos soportados.
Prioridad de desarrollo: Alta	
Entrada: Archivo de imagen seleccionado por el alumno para cargar como intento de ejercicio.	Salida: Aceptación del archivo si cumple con los formatos soportados, o mensaje de error indicando el formato no permitido.
Descripción: La aplicación web valida el formato del archivo de imagen cargado por el alumno, aceptando únicamente archivos con extensión .jpeg, .jpg y .png. Si el archivo no cumple con los formatos soportados, la aplicación web notifica al alumno con un mensaje de error y le solicita cargar un archivo válido. Precondición: El alumno debe estar en la modalidad de carga de imagen dentro de la vista de captura (RF-18) e intentar cargar un archivo de imagen. Postcondición: Si el formato es válido, la imagen queda disponible como intento del ejercicio. Si no lo es, la aplicación web muestra un mensaje de error y el alumno debe seleccionar un archivo con formato soportado.	

Tabla 57: Especificación de requerimiento RF-21

Requerimiento: La aplicación web contará con herramientas para rotar imágenes en incrementos de 90° y recortar mediante selección rectangular antes de su envío.	
Identificador: RF-21	Nombre: Herramientas de pre-edición de imagen.
Prioridad de desarrollo: Media	
Entrada: Imagen cargada o capturada por el alumno y acciones de rotación o recorte aplicadas sobre ella.	Salida: Imagen ajustada con los cambios aplicados, disponible en la vista previa para su confirmación y envío.

Descripción: Una vez que el alumno carga o captura una imagen de su escritura manuscrita, la aplicación web le proporciona herramientas de pre-edición que le permiten rotar la imagen en incrementos de 90° y recortarla mediante selección rectangular, con el fin de asegurar que la escritura sea legible antes de enviarla al análisis.

Precondición: El alumno debe haber cargado un archivo de imagen (RF-18) o capturado una fotografía con la cámara (RF-19) dentro de la vista de captura.

Postcondición: La imagen editada queda disponible en la vista previa con los ajustes aplicados, lista para ser confirmada y enviada a revisión.

Tabla 58: Especificación de requerimiento RF-22

Requerimiento: La aplicación web permitirá al alumno descartar la imagen actual cargada mediante una función de limpieza que restablezca el estado de la vista previa y elimine la referencia del archivo.	
Identificador: RF-22	Nombre: Reinicio de captura.
Prioridad de desarrollo: Media	
Entrada: Acción del alumno de descartar la imagen actualmente cargada en la vista de captura.	Salida: Vista previa restablecida a su estado inicial y referencia del archivo eliminada, habilitando una nueva carga de imagen.
Descripción: Permite al alumno descartar la imagen cargada o capturada si decide reemplazarla por otra. La función de limpieza restablece completamente el estado de la vista previa y elimina la referencia al archivo anterior, dejando la vista lista para una nueva captura o carga de imagen. Precondición: El alumno debe tener una imagen cargada o capturada en la vista de captura que desee descartar. Postcondición: La vista de captura regresa a su estado inicial sin ninguna imagen referenciada, permitiendo al alumno cargar o capturar una nueva imagen.	

Tabla 59: Especificación de requerimiento RF-23

Requerimiento: La aplicación web proporcionará al alumno un conjunto de herramientas dentro del lienzo digital para facilitar la escritura manuscrita y su corrección.	
Identificador: RF-23	Nombre: Herramientas del lienzo digital.
Prioridad de desarrollo: Alta	
Entrada: Interacción del alumno con las herramientas disponibles en el lienzo digital durante la realización del ejercicio.	Salida: Trazos y correcciones reflejados en tiempo real sobre el lienzo digital.
Descripción: El lienzo digital integrado en la vista de captura proporciona al alumno las siguientes herramientas para la escritura y corrección de trazos: ajuste de grosor de trazo (RF-23.1), herramienta de borrador (RF-23.2), deshacer acciones (RF-23.3), rehacer acciones (RF-23.4) y limpieza completa del lienzo (RF-23.5). Cada herramienta se detalla en su especificación correspondiente. Precondición: El alumno debe haber accedido a la modalidad de lienzo digital dentro de la vista de captura (RF-18). Postcondición: El alumno puede escribir y corregir su escritura sobre el lienzo digital utilizando las herramientas disponibles antes de proceder a la previsualización y envío del ejercicio.	

Tabla 60: Especificación de requerimiento RF-23.1

Requerimiento: La aplicación web permitirá ajustar el grosor del trazo en el lienzo digital.	
Identificador: RF-23.1	Nombre: Ajuste de grosor de trazo.
Prioridad de desarrollo: Media	
Entrada: Selección del grosor de trazo deseado por el alumno dentro del lienzo digital.	Salida: Trazos posteriores reflejados con el grosor seleccionado sobre el lienzo digital.

Descripción: Permite al alumno ajustar el grosor del trazo antes o durante la escritura en el lienzo digital, adaptándolo a sus necesidades de escritura manuscrita. Precondición: El alumno debe estar utilizando la modalidad de lienzo digital dentro de la vista de captura (RF-18). Postcondición: Los trazos realizados después del ajuste se muestran con el grosor seleccionado en el lienzo digital.

Tabla 61: Especificación de requerimiento RF-23.2

Requerimiento: La aplicación web proporcionará una herramienta de borrador para eliminar trazos de forma selectiva sobre el lienzo digital.	
Identificador: RF-23.2	Nombre: Herramienta de borrador.
Prioridad de desarrollo: Alta	
Entrada: Acción del alumno de activar el borrador y arrastrarlo sobre los trazos que desea eliminar.	Salida: Trazos seleccionados eliminados del lienzo digital de forma inmediata.
Descripción: Permite al alumno corregir errores en su escritura eliminando trazos de manera selectiva mediante el borrador, sin afectar el resto del contenido del lienzo digital. Precondición: El alumno debe estar utilizando la modalidad de lienzo digital y tener trazos registrados sobre el lienzo (RF-23). Postcondición: Los trazos sobre los que se aplicó el borrador quedan eliminados del lienzo, permitiendo al alumno continuar escribiendo en el área corregida.	

Tabla 62: Especificación de requerimiento RF-23.3

Requerimiento: La aplicación web permitirá al alumno deshacer la última acción realizada sobre el lienzo digital.	
Identificador: RF-23.3	Nombre: Deshacer acciones.
Prioridad de desarrollo: Alta	
Entrada: Acción del alumno de activar la función de deshacer dentro del lienzo digital.	Salida: Reversión de la última acción registrada sobre el lienzo, restaurando el estado anterior.

Descripción: Permite al alumno revertir la última acción realizada sobre el lienzo digital, ya sea un trazo o una corrección con el borrador, facilitando la corrección de errores sin necesidad de limpiar el lienzo completo. Precondición: El alumno debe estar utilizando la modalidad de lienzo digital y haber realizado al menos una acción sobre el lienzo (RF-23). Postcondición: La última acción registrada queda revertida y el lienzo refleja el estado anterior a dicha acción.

Tabla 63: Especificación de requerimiento RF-23.4

Requerimiento: La aplicación web permitirá al alumno rehacer la última acción deshecha sobre el lienzo digital.	
Identificador: RF-23.4	Nombre: Rehacer acciones.
Prioridad de desarrollo: Media	
Entrada: Acción del alumno de activar la función de rehacer dentro del lienzo digital.	Salida: Restauración de la última acción revertida sobre el lienzo digital.
Descripción: Permite al alumno restaurar una acción previamente deshecha sobre el lienzo digital, en caso de que decida mantenerla después de haberla revertido. Precondición: El alumno debe haber utilizado la función de deshacer (RF-23.3) al menos una vez sobre el lienzo digital. Postcondición: La acción revertida queda restaurada y el lienzo refleja el estado posterior a dicha acción.	

Tabla 64: Especificación de requerimiento RF-23.5

Requerimiento: La aplicación web permitirá al alumno limpiar completamente el lienzo digital, eliminando todos los trazos registrados.	
Identificador: RF-23.5	Nombre: Limpieza completa del lienzo.
Prioridad de desarrollo: Media	
Entrada: Acción del alumno de activar la función de limpieza completa dentro del lienzo digital.	Salida: Lienzo digital completamente vacío, listo para iniciar una nueva escritura.

Descripción: Permite al alumno eliminar todos los trazos registrados en el lienzo digital de forma inmediata, restableciendo el lienzo a su estado inicial para comenzar la escritura desde cero.

Precondición: El alumno debe estar utilizando la modalidad de lienzo digital y tener al menos un trazo registrado sobre el lienzo (RF-23).

Postcondición: El lienzo queda completamente vacío y el alumno puede iniciar una nueva escritura desde cero.

Tabla 65: Especificación de requerimiento RF-24

Requerimiento: La aplicación web mostrará al alumno una vista previa de su captura de escritura antes de confirmar el envío, aplicable tanto para imagen cargada como para el lienzo digital.	
Identificador: RF-24	Nombre: Previsualización y confirmación.
Prioridad de desarrollo: Alta	
Entrada: Captura de escritura generada por el alumno, ya sea mediante imagen cargada o trazos en el lienzo digital convertidos a imagen.	Salida: Vista previa de la captura de escritura disponible para que el alumno la revise y confirme antes de proceder al envío.
Descripción: Antes de enviar el ejercicio, la aplicación web muestra al alumno una vista previa de su captura de escritura. En el caso del lienzo digital, los trazos son convertidos a imagen. Precondición: El alumno debe haber generado contenido en la vista de captura, ya sea mediante imagen cargada (RF-18) o trazos en el lienzo digital (RF-23). Postcondición: Si el alumno confirma, el ejercicio procede al envío. Si decide regresar, la vista de captura se mantiene con el contenido anterior para continuar editando.	

Tabla 66: Especificación de requerimiento RF-25

Requerimiento: La aplicación web deshabilitará el botón de envío cuando el alumno no haya registrado trazos en el lienzo digital ni cargado una imagen, evitando envíos vacíos.	
Identificador: RF-25	Nombre: Validación de envío vacío.
Prioridad de desarrollo: Alta	

Entrada: Estado de la vista de captura sin trazos registrados en el lienzo ni imagen cargada.	Salida: Botón de envío deshabilitado visualmente.
Descripción: La aplicación web verifica si el alumno ha generado contenido en la vista de captura, ya sea trazos en el lienzo digital o una imagen cargada. Si no existe contenido válido, el botón de envío permanece deshabilitado. Precondición: El alumno debe encontrarse en la vista de captura de escritura sin haber registrado trazos ni cargado una imagen. Postcondición: El botón de envío se habilita únicamente cuando existe contenido en la vista de captura, permitiendo al alumno proceder con el envío del ejercicio.	

Tabla 67: Especificación de requerimiento RF-26

Requerimiento: La aplicación web analizará automáticamente la imagen del intento de escritura enviado mediante el módulo de reconocimiento óptico de caracteres (OCR), evaluando los aspectos caligráficos del trazo.	
Identificador: RF-26	Nombre: Análisis OCR.
Prioridad de desarrollo: Alta	
Entrada: Imagen del intento de escritura enviado por el alumno (trazo digital convertido a imagen o fotografía cargada).	Salida: Resultado del análisis caligráfico generado por el módulo OCR, incluyendo el texto reconocido y las observaciones sobre la calidad del trazo.
Descripción: Una vez que el alumno envía su intento de escritura, el módulo OCR integrado en la aplicación web procesa la imagen para reconocer los caracteres escritos y evaluar los aspectos caligráficos del trazo, como la forma, legibilidad y trazado de las letras. Precondición: El alumno debe haber enviado un intento de ejercicio con una imagen válida generada desde el lienzo digital o cargada desde su dispositivo. Postcondición: El resultado del análisis caligráfico queda almacenado y disponible para ser procesado por el módulo NLP (RF-27), visualizado por el alumno (RF-16) y consultado por el tutor (RF-09).	

Tabla 68: Especificación de requerimiento RF-27

Requerimiento: La aplicación web analizará el texto reconocido por el módulo OCR mediante procesamiento de lenguaje natural (NLP), evaluando los aspectos ortográficos de la escritura del alumno.	
Identificador: RF-27	Nombre: Análisis NLP.
Prioridad de desarrollo: Alta	
Entrada: Texto reconocido por el módulo OCR (RF-26) a partir de la imagen del intento de escritura del alumno.	Salida: Resultado del análisis ortográfico con identificación de errores, palabras incorrectas.
Descripción: A partir del texto extraído por el módulo OCR, el módulo NLP evalúa la ortografía de la escritura del alumno, identificando errores como palabras mal escritas, acentuación incorrecta y uso inadecuado de caracteres. Los resultados del análisis ortográfico se combinan con los del análisis caligráfico (RF-26) para generar un resultado integral del intento. Precondición: El módulo OCR debe haber procesado exitosamente la imagen del intento y generado el texto reconocido (RF-26). Postcondición: El resultado del análisis ortográfico queda almacenado y disponible para su visualización por el alumno (RF-16), consulta por el tutor (RF-09) y generación de recomendaciones por intento.	

Tabla 69: Especificación de requerimiento RF-28

Requerimiento: La aplicación web notificará al alumno cuando el análisis automático de escritura falle, permitiéndole realizar un nuevo intento de envío.	
Identificador: RF-28	Nombre: Notificación de fallo en análisis.
Prioridad de desarrollo: Media	
Entrada: Error generado por el módulo OCR (RF-26) o NLP (RF-27) al procesar el intento de escritura enviado por el alumno.	Salida: Mensaje de notificación al alumno indicando el fallo en el análisis.

Descripción: Si el módulo OCR o NLP falla al procesar el intento de escritura enviado, la aplicación web notifica al alumno del error de forma clara.

Precondición: El alumno debe haber enviado un intento de ejercicio que el módulo de análisis no pudo procesar correctamente.

Postcondición: El alumno es notificado del fallo y puede realizar un nuevo intento de envío con la misma captura de escritura.

Tabla 70: Especificación de requerimiento RF-29

Requerimiento: La aplicación web permitirá al tutor modificar los resultados del análisis ortográfico y caligráfico de un ejercicio realizado por un alumno, cuando no esté de acuerdo con el análisis automático.	
Identificador: RF-29	Nombre: Modificación de resultados.
Prioridad de desarrollo: Media	
Entrada: Ejercicio completado seleccionado y correcciones ingresadas por el tutor sobre los resultados del análisis automático.	Salida: Resultados actualizados reflejados en el historial del alumno.
Descripción: Cuando el tutor considera que el análisis automático generado por los módulos OCR y NLP no refleja correctamente el desempeño del alumno, puede modificar los resultados del intento. Precondición: El tutor debe estar autenticado, haber consultado el historial de actividades del alumno (RF-09) y el intento debe tener resultados de análisis disponibles. Postcondición: Los resultados modificados quedan almacenados y se reflejan en el historial del alumno (RF-16).	

Tabla 71: Especificación de requerimiento RF-30

Requerimiento: La aplicación web presentará al alumno actividades de mejora sugeridas, ordenadas por prioridad según las debilidades ortográficas y caligráficas.	
Identificador: RF-30	Nombre: Actividades sugeridas.
Prioridad de desarrollo: Alta	
Entrada: Resultados del análisis ortográfico (RF-27) y caligráfico (RF-26) generados a partir del intento de escritura del alumno.	Salida: Lista de actividades de mejora sugeridas ordenadas por prioridad y enfocadas en las áreas de oportunidad detectadas.
Descripción: La aplicación web genera recomendaciones de actividades de mejora personalizadas para el alumno. Cada recomendación cuenta con un atributo de prioridad que permite al alumno identificar sus debilidades ortografía y caligrafía. Precondición: El alumno debe haber completado y enviado al menos un intento con resultados de análisis OCR y NLP disponibles (RF-26, RF-27). Postcondición: Las actividades sugeridas quedan registradas y disponibles para que el alumno las consulte por orden de prioridad.	

Tabla 72: Especificación de requerimiento RF-31

Requerimiento: La aplicación web permitirá al alumno realizar las actividades sugeridas, generando un nuevo intento evaluado por los módulos OCR y NLP para medir el progreso en las áreas de ortografía y caligrafía identificadas.	
Identificador: RF-31	Nombre: Realización de actividades sugeridas.
Prioridad de desarrollo: Alta	
Entrada: Actividad sugerida seleccionada por el alumno desde su lista de recomendaciones ordenadas por prioridad (RF-30).	Salida: Nuevo intento evaluado con resultados de análisis ortográfico o caligráfico, reflejando el progreso del alumno respecto al intento que originó la recomendación.

Descripción: El alumno selecciona una actividad sugerida y la realiza mediante el módulo de captura de escritura (RF-17 al RF-25), generando un nuevo intento que es evaluado por los módulos OCR (RF-26) y NLP (RF-27) según el tipo de la actividad.

Precondición: El alumno debe estar autenticado y tener actividades sugeridas disponibles con prioridad asignada (RF-30).

Postcondición: El nuevo intento queda registrado y evaluado, el progreso del alumno queda actualizado y se generan nuevas recomendaciones si el análisis identifica áreas de oportunidad adicionales. La actividad realizada se marca como completada en el historial del alumno.

3.7.3. Requerimientos no funcionales

Tabla 73: Requerimientos no funcionales del sistema

Requerimiento	Nombre	Descripción
RNF-01	Usabilidad	La aplicación web debe garantizar la accesibilidad, comprensión y correcta interacción del usuario en múltiples dispositivos. Especificación: Los textos deben mantener un índice de legibilidad Fernández-Huerta mayor o igual a 80. [38] La interfaz debe ser responsiva y se deben bloquear los gestos de desplazamiento del navegador.
RNF-02	Eficiencia	La aplicación web debe responder a las acciones del usuario en tiempos que no interrumpen el flujo de trabajo. Especificación: La interfaz debe proporcionar retroalimentación visual al cambiar de herramienta en un tiempo menor o igual a 100 ms. [39] Los procesos de procesamiento y exportación del dibujo deben ejecutarse de manera asíncrona, sin generar bloqueos perceptibles.

Requerimiento	Nombre	Descripción
RNF-03	Confiabilidad	La aplicación web debe prevenir fallos derivados de entradas incorrectas por parte del usuario y garantizar condiciones operativas mínimas. Especificación: Las imágenes procesadas deben cumplir con una resolución mínima de 1024 x 768 píxeles. Si la validación falla, la aplicación web debe interrumpir el flujo de envío y emitir un mensaje de alerta solicitando una captura válida.
RNF-04	Interoperabilidad	La aplicación web debe generar salidas de datos en formatos estrictos para garantizar la compatibilidad y precisión de sistemas externos dependientes (módulo OCR). Especificación: La exportación de imágenes desde el lienzo digital debe realizarse obligatoriamente en formato PNG.

3.8. Módulo de Ejercicios y Retroalimentación

El módulo de ejercicios y retroalimentación es el componente pedagógico central del sistema, encargado de transformar los resultados de los análisis ortográfico y caligráfico en acciones correctivas concretas. Su propósito es proporcionar al alumno material de práctica focalizado en sus áreas de oportunidad específicas y al tutor las herramientas para supervisar y complementar dicho proceso. A diferencia de los módulos de captura y análisis, cuya naturaleza es predominantemente técnica, este módulo establece el puente entre el procesamiento computacional y el contexto pedagógico en el que opera la aplicación.

3.8.1. Justificación pedagógica de los ejercicios

La selección y diseño de los ejercicios dentro de la plataforma se fundamentan en los requerimientos del plan de estudios de la Secretaría de Educación Pública (SEP) para tercer y cuarto grado de primaria [?, ?, ?], etapa en la cual el alumno transita de la decodificación básica hacia la consolidación de la ortografía convencional y la fluidez del trazo manuscrito. En este periodo, los errores no corregidos tienden a fijarse como hábitos, lo que justifica la intervención temprana y personalizada que el sistema propone.

Los ejercicios se categorizan en dos grandes rubros, en correspondencia directa con las métricas evaluadas por el sistema:

Ejercicios de refuerzo caligráfico Orientados a mejorar las métricas de trazo extraídas por el módulo OCR: alineación, tamaño de letra, espaciado e inclinación. Se contemplan dos tipos:

1. **Copia de modelos (pangramas y frases cortas):** Se utilizan frases que contienen todas las letras del alfabeto —pangramas— y oraciones adaptadas al vocabulario infantil. La práctica de este tipo de textos obliga al alumno a trabajar la proporción de tamaño y el espaciado correcto entre caracteres de distinta frecuencia de uso, dado que los pangramas incluyen letras poco comunes que requieren mayor atención en el trazo.
2. **Trazos de direccionalidad:** Ejercicios enfocados en grupos de letras que comparten patrones motores similares, como *a, c, d, g, q* o *b, p, d, q*. Su objetivo es corregir la inclinación del trazo y reducir la confusión espacial derivada de la similitud morfológica entre estos caracteres, un fenómeno frecuente en alumnos de este nivel educativo.

Ejercicios de corrección ortográfica Generados dinámicamente a partir de los errores detectados por el módulo NLP y asignados según el tipo de falla identificada. Se contemplan tres modalidades:

1. **Uso de grafías conflictivas:** Ejercicios de completar palabras enfocados en las reglas ortográficas de mayor dificultad en este nivel educativo: uso de *b/v, c/s/z, g/j* y el uso de la *h*. Cada ejercicio presenta al alumno el contexto de la palabra con la grafía omitida, de modo que la decisión ortográfica requiera razonamiento y no memorización mecánica.
2. **Acentuación:** Ejercicios de identificación y colocación de tildes, con énfasis en la clasificación de palabras en agudas, graves y esdrújulas. El sistema selecciona palabras del mismo campo semántico que el texto original del alumno para mantener la coherencia con el ejercicio que motivó el análisis.
3. **Reescritura focalizada:** Cuando el módulo NLP detecta una palabra mal escrita en el texto libre del alumno, el sistema genera automáticamente un ejercicio de repetición espaciada en el que el alumno debe reescribir correctamente esa palabra específica dentro de un nuevo contexto oracional. Esta modalidad es la de mayor personalización, pues el contenido del ejercicio se construye directamente a partir del error individual del alumno.

3.8.2. Estructura del catálogo de ejercicios

El catálogo de ejercicios disponibles en la plataforma se almacena en la tabla **Ejercicios** de la base de datos. Cada registro del catálogo contiene los atributos necesarios para que el sistema pueda tanto presentarlo al alumno como procesarlo automáticamente para la generación de variantes. La Tabla ?? describe los atributos del catálogo y su propósito dentro del módulo.

Tabla 74: Atributos del catálogo de ejercicios del sistema

Atributo	Tipo	Descripción
id_ejercicio	Entero	Identificador único del ejercicio en el catálogo.
titulo	VARCHAR(150)	Nombre descriptivo del ejercicio visible para el tutor.
descripcion	VARCHAR(MAX)	Instrucciones del ejercicio para el alumno.
tipo	VARCHAR(30)	Categoría del ejercicio: caligrafico u ortografico .
contenido_base	VARCHAR(MAX)	Texto base, pangrama, frase o patrón de letra sobre el que se construye el ejercicio.
id_estatus	Entero	Estado del ejercicio en el catálogo (activo/inactivo).

El campo **tipo** actúa como discriminador para que el sistema pueda filtrar y asignar ejercicios de forma coherente con el tipo de error detectado: errores caligráficos activan ejercicios de tipo **caligrafico**, mientras que errores ortográficos activan ejercicios de tipo **ortografico**. Esta separación garantiza que la retroalimentación entregada al alumno sea siempre pertinente al área de oportunidad identificada en su intento.

La relación entre el catálogo y el tutor se gestiona a través de la tabla **Ejercicios_Tutor**, que representa la activación de un ejercicio del catálogo por parte de un tutor específico y contiene su propio ciclo de vida independiente, conforme al diseño descrito en la Sección 4.5.

3.9. Criterios de selección de dataset

En este apartado se presenta el análisis de los datasets necesarios para el entrenamiento y validación del módulo de reconocimiento óptico de caracteres (OCR). Dado que el sistema está orientado a la escritura manuscrita en letra de molde de estudiantes de tercer y cuarto

grado de educación primaria, la selección de datos de entrenamiento representa un factor determinante en la precisión final del modelo. Por ello, se establecen criterios de selección, se evalúan los conjuntos de datos públicamente disponibles y se identifican las brechas que deberán atenderse en la fase de diseño.

3.9.1. Determinación de criterios de selección del dataset

Para garantizar que los datos de entrenamiento sean adecuados al contexto del sistema, se establecieron criterios de selección a partir de los requerimientos funcionales y no funcionales definidos en la sección 3.7. Estos criterios permiten evaluar de manera objetiva la pertinencia de cada dataset disponible y fundamentar las decisiones de selección.

En la Tabla ?? se presentan los criterios definidos para la evaluación de datasets, junto con su justificación y el nivel de prioridad asignado.

Tabla 75: Criterios de selección del dataset.

ID	Criterio	Descripción	Prioridad
CR-01	Licencia de uso académico	El dataset debe contar con una licencia que permita su uso sin restricciones para investigación y desarrollo académico no comercial.	Alta
CR-02	Soporte de caracteres en español	El dataset debe incluir los 27 caracteres del alfabeto español, incluyendo la letra ñ y las cinco vocales con tilde (á, é, í, ó, u).	Alta
CR-03	Tipo de escritura	Las muestras deben corresponder a letra de molde (imprenta), ya que el sistema excluye explícitamente la escritura cursiva (véase sección 3.1.2).	Alta
CR-04	Volumen de muestras	El dataset debe aportar un número suficiente de muestras por clase para garantizar variabilidad estadística. Se establece un mínimo de 1,000 imágenes por carácter.	Alta

ID	Criterio	Descripción	Prioridad
CR-05	Formato de imagen compatible	Las imágenes deben estar disponibles en formato PNG o JPEG, compatibles con el módulo de preprocesamiento descrito en los requerimientos no funcionales (RNF-04).	Media
CR-06	Disponibilidad de etiquetas	El dataset debe incluir anotaciones que permitan la correspondencia entre imagen y carácter o palabra reconocida, para facilitar el entrenamiento supervisado.	Alta
CR-07	Escritura infantil	Se prioriza que las muestras provengan de escritura infantil o, en su defecto, que incluyan irregularidades propias de este tipo de escritura (trazos variables, tamaños inconsistentes).	Media
CR-08	Accesibilidad y descarga pública	El dataset debe estar disponible para su descarga directa, sin requerir acuerdos institucionales formales que puedan retrasar el desarrollo del proyecto.	Media

3.9.2. Análisis de datasets disponibles

A partir de los criterios definidos, se realizó una revisión de los principales conjuntos de datos disponibles públicamente que han sido utilizados en tareas de reconocimiento de escritura manuscrita. En la Tabla ?? se comparan estos datasets en función de los criterios establecidos, indicando mediante una escala de cumplimiento si cada criterio es satisfecho de forma completa ($\checkmark$), parcial (Δ) o no satisfecho ($\times$).

Tabla 76: Evaluación de datasets según los criterios de selección.

Dataset	CR-01 Licencia	CR-02 Español	CR-03 Molde	CR-04 Volumen	CR-05 Formato	CR-06 Etiquetas	CR-07 Infantil	CR-08 Acceso
Letras Alfabeto Español (Kaggle, Herrera, 2023) [?]	✓	✓	✓	✓	✓	✓	×	✓
EMNIST Balanced (Cohen et al., 2017) [?]	✓	×	✓	✓	✓	✓	×	✓
IAM Handwriting Database (Marti & Bunke, 2002) [?]	Δ	×	✓	✓	✓	✓	×	Δ
Tesseract spa.traineddata (Google, Tesseract OCR) [?]	✓	✓	✓	—	—	—	×	✓
Nota: El dataset Tesseract spa.traineddata es un modelo preentrenado y no un conjunto de imágenes. Se incluye en la comparativa dado que representa una alternativa de punto de partida para el reconocimiento de texto en español, aplicable mediante técnicas de fine-tuning. El volumen y formato no aplican en este caso, ya que el acceso es a pesos del modelo y no a imágenes crudas.								

A continuación, se describen los hallazgos más relevantes de cada dataset evaluado:

1. Letras Alfabeto Español (Kaggle – Herrera, 2023). Este dataset contiene imágenes de los 27 caracteres del alfabeto español en forma aislada, lo que lo convierte en el único conjunto identificado con soporte nativo de la letra ñ y las vocales acentuadas, atendiendo directamente el criterio CR-02. Cada subcarpeta contiene entre 972 y 1,221 imágenes de 350×350 píxeles en letra de molde manuscrita, satisfaciendo los criterios de volumen (CR-04), formato (CR-05) y tipo de escritura (CR-03). El límite inferior de 972 imágenes en algunas clases se considera estadísticamente aceptable dado que la mayoría supera el umbral de 1,000 establecido, y puede complementarse mediante técnicas de aumentación de datos durante la fase de diseño. Dado que las muestras corresponden a escritura adulta, el criterio CR-07 no es satisfecho.
2. EMNIST Balanced. Este dataset, derivado del NIST Special Database 19, contiene

131,600 imágenes de caracteres manuscritos distribuidos en 47 clases balanceadas, con una escala suficiente para garantizar variabilidad estadística (CR-04). Su limitación principal para el presente proyecto radica en que el conjunto de caracteres corresponde al alfabeto latino básico (A–Z, a–z, 0–9) sin incluir caracteres específicos del español (CR-02), lo que genera una brecha directa con los requerimientos del sistema definidos en OE-01.

3. IAM Handwriting Database. Contiene formularios completos de texto manuscrito en inglés, etiquetados a nivel de línea y palabra. Si bien su volumen y calidad de etiquetado son considerables, su licencia requiere registro institucional (CR-01 parcial) y su contenido está en inglés (CR-02 no satisfecho), lo que limita su aplicabilidad directa en el presente proyecto.
4. Tesseract spa.traineddata. El modelo preentrenado de Tesseract para español, disponible de forma abierta bajo licencia Apache 2.0, ofrece una capacidad base de reconocimiento de texto en español. Su fortaleza principal radica en el soporte completo de caracteres en español (CR-02) y su accesibilidad directa (CR-08). No obstante, este modelo está optimizado para texto impreso o tipografiado, mostrando una precisión reducida ante la variabilidad propia de la escritura manuscrita infantil (CR-07).

3.9.3. Identificación de brechas y justificación de selección

El análisis comparativo revela que ningún dataset disponible cumple la totalidad de los criterios establecidos de forma individual. En particular, se identifican tres brechas críticas que deben ser atendidas en el diseño del sistema:

1. Ausencia de caracteres españoles en datasets de alto volumen. El dataset con mayor volumen y variabilidad (EMNIST) no incluye los caracteres propios del español. Esta brecha implica que dicho dataset solo puede utilizarse como complemento para los caracteres del alfabeto latino compartido (A–Z, a–z), y no puede resolver por sí solo el objetivo OE-01.
2. Cobertura parcial del alfabeto español en datasets accesibles. El único dataset identificado con soporte de caracteres específicos del español es el de Kaggle (Herrera, 2023), cuya cobertura de muestras por carácter es limitada. Esto implica la necesidad de complementar este dataset con técnicas de aumentación de datos para garantizar la variabilidad necesaria para el entrenamiento y validación del modelo OCR.
3. Inexistencia de datasets públicos de escritura infantil en español en letra de molde. Ninguno de los datasets identificados proviene de escritura infantil, lo que representa

la brecha más significativa respecto al contexto de uso del sistema. Esta condición es inherente al dominio del proyecto, dado que la escritura infantil presenta irregularidades específicas (variabilidad en tamaño, inclinación y espaciado) que los modelos entrenados exclusivamente con escritura adulta pueden no capturar con suficiente precisión, tal como se señaló en la sección 1.3 del marco teórico.

Debido a lo anterior, la estrategia de dataset adoptada para el proyecto se basa en la combinación de los siguientes componentes:

- Dataset primario: Letras Alfabeto Español (Kaggle – Herrera, 2023), por ser el único conjunto con soporte nativo de los caracteres específicos del español requeridos por el sistema.
- Dataset complementario: EMNIST Balanced, como fuente de variabilidad adicional para los caracteres del alfabeto latino compartido con el español (A–Z, a–z, 0–9).
- Punto de partida del modelo: Tesseract `spa.traineddata`, como modelo base sobre el cual aplicar técnicas de ajuste fino (fine-tuning) con los datos recopilados.

La estrategia detallada de combinación, preprocesamiento y aumentación de datos será descrita en el Capítulo 4, en el contexto del diseño del módulo OCR. Asimismo, las muestras recopiladas durante la prueba piloto con 25 alumnos (descrita en el Capítulo 1) constituirán un conjunto de validación con escritura infantil real, permitiendo evaluar el desempeño del modelo en condiciones representativas del entorno de uso previsto.

4 Capítulo 4: Diseño

En este capítulo se presenta la fase de diseño técnico y arquitectónico de la plataforma web, tomando como base los requerimientos funcionales, no funcionales y las reglas de negocio definidos en el capítulo anterior. A lo largo de esta sección se detallará la estructuración general del sistema, el diseño del flujo de interacción de los usuarios, la especificación de las interfaces de programación (API REST) y el modelado de la base de datos. El propósito principal de esta fase es establecer un modelo sólido que garantice la integración eficiente, la escalabilidad y el correcto funcionamiento de los módulos de captura, OCR y NLP.

4.1. Arquitectura del Módulo de Autenticación

El módulo de autenticación es el punto de entrada transversal de la plataforma: cualquier operación protegida —registro de intentos, consulta de alumnos, asignación de ejercicios— depende de que el usuario haya obtenido previamente un token válido. El diseño sigue el patrón arquitectónico MVC establecido para el sistema y se apoya en el estándar JSON Web Token (JWT) para la transmisión segura de identidad y rol entre el Front-End y el Back-End.

La arquitectura del módulo se divide en tres componentes principales:

1. Capa de Presentación (Front-End): Compuesta por los componentes `Login.jsx` y `Registro.jsx`. Esta capa es responsable de la validación de formato en el cliente (longitud de campos, caracteres permitidos), el envío de las credenciales al Back-End mediante peticiones HTTP y el almacenamiento del token recibido en `localStorage` para su uso en peticiones subsecuentes.
2. Capa de Lógica de Negocio (Back-End): Implementada en el controlador `auth.py` mediante FastAPI. Esta capa gestiona la verificación de credenciales contra la base de datos, el cifrado y la validación de contraseñas con `bcrypt`, la emisión de tokens JWT firmados y la verificación del estado activo del usuario. Adicionalmente, el módulo `dependencies.py` expone las dependencias reutilizables `require_tutor` y `require_alumno`, que actúan como guardas de acceso en todos los demás controladores del sistema.
3. Capa de Persistencia (Base de Datos): El módulo opera exclusivamente sobre la tabla `Usuario`, consultando el campo `contrasena_cifrada` (almacenado como hash `bcrypt` en `VARCHAR(255)`) y el campo `id_estatus` para verificar que la cuenta se encuentre activa antes de emitir el token.

4.1.1. Diseño del Token JWT

El token emitido por el sistema sigue el estándar JWT (RFC 7519) firmado con el algoritmo simétrico HMAC-SHA256 (HS256). El payload del token contiene exclusivamente los campos necesarios para que cualquier controlador pueda tomar decisiones de autorización sin consultar la base de datos en cada petición:

Código 4.1: Estructura del payload JWT emitido por el Back-End

```
1 {  
2   "id_usuario": <integer>,  
3   "nombre":      <string>,  
4   "tipo_usuario": "tutor" | "alumno",  
5   "exp":         <unix timestamp>  
6 }
```

El campo `tipo_usuario` es el discriminador de rol que consumen las dependencias `require_tutor` y `require_alumno`. El token tiene una vigencia de 24 horas (`ACCESS_TOKEN_EXPIRE_MINUTES = 1440`), tras la cual el usuario debe autenticarse nuevamente. La clave de firma (`SECRET_KEY`) se gestiona mediante variables de entorno y no se incluye en el repositorio.

4.1.2. Diseño del Flujo de Interacción

El módulo contempla dos flujos principales: inicio de sesión y registro de tutor.

Flujo de Inicio de Sesión:

1. El usuario introduce su `username` y contraseña en el componente `Login.jsx` y envía la petición.
2. El Front-End realiza una petición `POST` al endpoint `/api/auth/login` con el payload en formato JSON.
3. El Back-End consulta la tabla `Usuario` filtrando por `username`. Si el registro no existe, retorna un error `401 Unauthorized` con un mensaje genérico para evitar la enumeración de usuarios.
4. Se verifica que `id_estatus = 1` (cuenta activa). Si el valor es distinto, se retorna `403 Forbidden`.
5. Se verifica la contraseña ingresada contra el hash almacenado utilizando `bcrypt`. Un fallo retorna `401 Unauthorized` con el mismo mensaje genérico del paso 3.
6. Si todas las validaciones son exitosas, se genera el token JWT con el payload descrito en la sección anterior y se retorna al Front-End junto con un mensaje de

confirmación.

7. El Front-End almacena el token en `localStorage` y redirige al usuario a su panel correspondiente según el campo `tipo_usuario` incluido en el payload decodificado.

Flujo de Registro de Tutor:

1. El tutor introduce sus datos en el componente `Registro.jsx`. El Front-End aplica validaciones de formato antes del envío.
2. El Front-End realiza una petición `POST` al endpoint `/api/auth/register` con el payload en formato JSON.
3. El Back-End verifica que el correo electrónico y el `username` no existan previamente en la tabla `Usuario`. En caso de duplicado, retorna `400 Bad Request` con el campo en conflicto especificado.
4. La contraseña es validada contra las reglas de complejidad definidas en el modelo `Pydantic` (longitud mínima de 8 caracteres, al menos una mayúscula, una minúscula, un dígito y un carácter especial). Una contraseña inválida retorna `422 Unprocessable Entity`.
5. La contraseña válida se cifra con `bcrypt` y se inserta un nuevo registro en la tabla `Usuario` con `tipo_usuario = 'tutor'` e `id_estatus = 1` (activo por defecto).
6. El Back-End retorna `200 OK` con un mensaje de confirmación. El tutor debe iniciar sesión de forma explícita para obtener su token.

Nota de diseño: el registro está restringido a cuentas de tipo tutor. Los alumnos son dados de alta únicamente por su tutor asignado a través del módulo de gestión de alumnos, lo que garantiza que ningún alumno pueda auto-registrarse en la plataforma.

4.1.3. Diseño de APIs

Para soportar los flujos descritos, el diseño de la API contempla los siguientes endpoints en el controlador `auth.py`:

Endpoint 1: Inicio de Sesión

El endpoint `/api/auth/login` opera mediante el método `POST` y no requiere autenticación previa. Recibe un payload JSON con `username` y `password` y, ante credenciales válidas, retorna el token JWT.

Código 4.2: Payload de solicitud y respuesta del endpoint de login

```
1 /* Solicitud */
2 {
```

```

3   "username": "string",
4   "password": "string"
5 }
6
7 /* Respuesta 200 OK */
8 {
9   "message": "Inicio de sesion exitoso",
10  "token":   "eyJhbGciOiI...
11 }

```

Endpoint 2: Registro de Tutor

El endpoint `/api/auth/register` opera mediante el método `POST` y no requiere autenticación previa. Recibe los datos del nuevo tutor y aplica las validaciones de unicidad y complejidad de contraseña descritas en el flujo anterior.

Código 4.3: Payload de solicitud del endpoint de registro

```

1 /* Solicitud */
2 {
3   "nombre":      "string",
4   "username":    "string (min 6 chars, [a-zA-Z0-9_])",
5   "email":       "string",
6   "password":    "string (min 8 chars, mayus + minus + num + especial)
7   "
8 }
9
10 /* Respuesta 200 OK */
11 {
12   "message": "Usuario registrado exitosamente"
13 }

```

4.1.4. Diseño del Control de Acceso Basado en Roles

El sistema implementa un control de acceso basado en roles (RBAC) a través de dos dependencias reutilizables definidas en `dependencies.py`: `require_tutor` y `require_alumno`. Ambas se inyectan como parámetros en los endpoints que requieren autenticación, siguiendo el mecanismo de inyección de dependencias de FastAPI.

El flujo de validación es el siguiente:

1. El cliente incluye el token en el encabezado HTTP de cada petición protegida:
`Authorization: Bearer <token>`.

2. La dependencia `get_current_user` extrae y decodifica el token usando la misma `SECRET_KEY` y algoritmo HS256. Si el token está vencido o es inválido, retorna 401 `Unauthorized`.
3. Las dependencias `require_tutor` y `require_alumno` verifican el campo `tipo_usuario` del payload decodificado. Si el rol no corresponde al requerido por el endpoint, retornan 403 `Forbidden`.
4. Si la validación es exitosa, el objeto `current_user` (con `id_usuario`, `tipo_usuario` y `nombre`) queda disponible para la lógica del controlador sin necesidad de realizar consultas adicionales a la base de datos.

La Tabla ?? resume los endpoints del sistema clasificados por el tipo de autenticación que requieren.

Tabla 77: Clasificación de endpoints por nivel de acceso

Endpoint	Método	Acceso requerido
/api/auth/login	POST	Público
/api/auth/register	POST	Público
/api/intentos/registrar	POST	Alumno (JWT)
/api/intentos/tutor/{id}	GET	Tutor (JWT)
/api/intentos/calificar/{id}	PUT	Tutor (JWT)
/api/alumnos/...	*	Tutor (JWT)
/api/ejercicios/...	*	Tutor / Alumno (JWT)
/api/perfil/...	*	Tutor / Alumno (JWT)

4.2. Diseño Preliminar del Módulo de Análisis OCR y NLP

El presente apartado constituye un diseño preliminar enmarcado en la metodología de desarrollo en espiral adoptada por el proyecto. Dado que la implementación completa de este módulo corresponde a la siguiente iteración (TT2), las decisiones que se presentan a continuación tienen carácter arquitectónico y conceptual: establecen las interfaces, los contratos de datos y la posición del módulo dentro del sistema, pero dejan abiertas las decisiones de modelo, hiperparámetros y bibliotecas específicas, las cuales serán evaluadas y refinadas con base en las pruebas de la iteración actual. Esto es consistente con el enfoque en espiral, cuya característica distintiva es precisamente diferir las decisiones de alto riesgo técnico hasta que se cuente con evidencia empírica suficiente para tomarlas.

El módulo de análisis OCR y NLP constituye el núcleo de procesamiento inteligente de la plataforma. Su función es transformar la imagen de escritura manuscrita capturada por el alumno en dos conjuntos de resultados cuantitativos y cualitativos: las métricas

caligráficas (alineación, tamaño, espaciado e inclinación) y el análisis ortográfico del texto detectado. Los fundamentos teóricos de ambas tecnologías han sido descritos en las Secciones 2.3 y 2.4 del presente documento; esta sección se limita a su diseño de integración dentro de la arquitectura del sistema.

La arquitectura de este módulo sigue el patrón MVC establecido para el sistema y se divide en tres componentes principales:

1. **Capa de Procesamiento (Back-End):** Un controlador dedicado `analisis.py` recibirá como entrada la imagen en formato Base64 almacenada en la tabla `Intentos` y orquestará la ejecución secuencial del sub-módulo OCR y del sub-módulo NLP. Este controlador será invocado de forma **diferida** respecto al registro del intento, es decir, el alumno recibe confirmación inmediata de envío y el procesamiento ocurre de manera asíncrona en segundo plano, evitando tiempos de espera perceptibles en la interfaz.
2. **Sub-módulo OCR:** Responsable de preprocesar la imagen y extraer el texto manuscrito en formato digital, así como de calcular las métricas caligráficas del trazo.
3. **Sub-módulo NLP:** Responsable de analizar el texto producido por el OCR, detectar errores ortográficos y generar sugerencias de corrección contextualizadas al nivel léxico de tercer y cuarto grado de primaria.

4.2.1. *Diseño del Flujo de Procesamiento*

El flujo general del módulo, desde la captura del intento hasta el almacenamiento de resultados, se describe a continuación:

1. El alumno envía su intento mediante el endpoint `/api/intentos/registrar`. El Back-End almacena la imagen en Base64 en la tabla `Intentos` con los campos `texto_detectado_ocr` y los registros de análisis en estado nulo, confirmando el envío al cliente de forma inmediata.
2. Un proceso en segundo plano —cuyo mecanismo de programación será definido en TT2, ya sea mediante un scheduler como el ya utilizado en `cerrar_ejercicio.py`, una cola de tareas o una llamada asíncrona— detecta el nuevo intento pendiente de análisis.
3. El **sub-módulo OCR** recibe la imagen en Base64, la decodifica y ejecuta la siguiente secuencia de procesamiento:
 - a) **Preprocesamiento:** conversión a escala de grises, binarización (método de Otsu [?]), eliminación de ruido y corrección de inclinación (*deskewing*), conforme a las técnicas descritas en la Sección 2.3.

- b) **Extracción de métricas caligráficas:** cálculo de los cuatro parámetros definidos en el sistema: `alineacion_score`, `tamano_letra_score`, `espaciado_score` e `inclinacion_score`. La metodología exacta de cálculo será determinada en TT2.
 - c) **Reconocimiento de texto:** aplicación del modelo de reconocimiento para obtener el texto digital. La elección del modelo (TrOCR, Tesseract u otro) queda abierta a la iteración siguiente, sujeta a las pruebas de precisión sobre escritura infantil descritas en el objetivo específico OE-03.
- 4. El texto reconocido se escribe en el campo `texto_detectado_ocr` de la tabla `Intentos` y los scores caligráficos se persisten en la tabla `Analisis_Caligrafico`.
- 5. El **sub-módulo NLP** recibe el texto digitalizado y ejecuta:
 - a) **Tokenización:** división del texto en unidades léxicas individuales.
 - b) **Detección de errores ortográficos:** comparación de cada token contra el modelo lingüístico, considerando el nivel cognitivo y vocabulario de tercer y cuarto grado de primaria.
 - c) **Generación de sugerencias:** producción de correcciones contextualizadas mediante algoritmos de similitud o modelos estadísticos, conforme a lo descrito en la Sección 2.4.
- 6. Los resultados del análisis ortográfico se persisten en la tabla `Analisis_Ortografico`: el `ortografia_score`, la `cantidad_errores` y el campo `sugerencias_json` con la estructura definida en la Sección 4.X.2.
- 7. Con ambos análisis completados, el proceso actualiza el registro de `Progreso_Alumno` mediante el trigger de base de datos descrito en la Sección 4.X+1, reflejando el nuevo promedio acumulado del alumno.

4.2.2. Contratos de Datos entre Módulos

Dado que el diseño de integración se establece en TT1 para orientar el desarrollo de TT2, se definen a continuación los contratos de datos que las interfaces entre módulos deben respetar. Estos contratos son fijos y no están sujetos a cambio en la iteración siguiente, pues representan el acuerdo entre la base de datos ya implementada y el módulo por desarrollar.

Entrada del sub-módulo OCR El sub-módulo OCR recibirá la imagen codificada en Base64 directamente del campo `imagen_codificada` de la tabla `Intentos`, identificada por su `id_intento`. No se requiere ningún endpoint adicional, ya que el proceso opera

sobre datos ya persistidos.

Salida del sub-módulo OCR hacia la base de datos

Código 4.4: Estructura de resultados del sub-módulo OCR hacia la BD

```
1  /* Tabla Intentos      campo a actualizar */
2  {
3    "texto_detectado_ocr": "string (texto reconocido)"
4  }
5
6  /* Tabla Analisis_Caligrafico      registro a insertar */
7  {
8    "id_intento":          <integer>,
9    "alineacion_score":    <decimal(5,2)>,
10   "tamano_letra_score":   <decimal(5,2)>,
11   "espaciado_score":      <decimal(5,2)>,
12   "inclinacion_score":    <decimal(5,2)>,
13   "observaciones":        "string | null"
14 }
```

Salida del sub-módulo NLP hacia la base de datos

Código 4.5: Estructura de resultados del sub-módulo NLP hacia la BD

```
1  /* Tabla Analisis_Ortografico      registro a insertar */
2  {
3    "id_intento":          <integer>,
4    "cantidad_errores":    <integer>,
5    "ortografia_score":    <decimal(5,2)>,
6    "errores_detectados":  "string | null",
7    "sugerencias_json": [
8      {
9        "palabra_incorrecta": "string",
10       "posicion":            <integer>,
11       "sugerencia":          "string",
12       "contexto":            "string"
13     }
14   ]
15 }
```

El campo `sugerencias_json` es validado al momento de la inserción mediante la restricción `CK_sugerencias_json` definida en el esquema de base de datos, la cual verifica que el contenido sea JSON válido mediante la función `ISJSON()` de SQL Server.

4.2.3. Diseño del Endpoint de Consulta de Resultados

Para que el Front-End pueda presentar los resultados del análisis al alumno y al tutor, se contempla el siguiente endpoint en el controlador `analisis.py`:

Endpoint: Consulta de Análisis por Intento El endpoint `/api/analisis/{id_intento}` operará mediante el método `GET` y requerirá autenticación JWT con rol de tutor o alumno. Su propósito es retornar de forma consolidada los resultados del análisis OCR y NLP asociados a un intento específico.

Código 4.6: Estructura de respuesta del endpoint de análisis

```
1  /* Respuesta 200 OK */
2  {
3      "id_intento":          <integer>,
4      "texto_detectado_ocr": "string | null",
5      "caligrafia": {
6          "alineacion_score": <decimal>,
7          "tamano_letra_score": <decimal>,
8          "espaciado_score": <decimal>,
9          "inclinacion_score": <decimal>,
10         "observaciones":    "string | null"
11     },
12     "ortografia": {
13         "ortografia_score": <decimal>,
14         "cantidad_errores": <integer>,
15         "sugerencias": [
16             {
17                 "palabra_incorrecta": "string",
18                 "posicion":           <integer>,
19                 "sugerencia":         "string",
20                 "contexto":           "string"
21             }
22         ]
23     }
24 }
```

Si el análisis aún no ha sido procesado (campos nulos en la BD), el endpoint retornará los campos de `caligrafia` y `ortografia` como `null`, indicando al cliente que el procesamiento se encuentra pendiente.

4.2.4. Decisiones Abiertas para TT2

En concordancia con el desarrollo en espiral, las siguientes decisiones técnicas de alto riesgo se posponen deliberadamente a la siguiente iteración, donde serán evaluadas mediante pruebas controladas:

- **Modelo OCR:** la elección entre TrOCR, Tesseract [?] u otro motor queda condicionada a las pruebas de precisión sobre muestras de escritura infantil en letra de molde (tipografía Century), conforme al objetivo específico OE-03 que establece una precisión mínima del 85 %.
- **Metodología de métricas caligráficas:** la definición de los algoritmos de cálculo de `alineacion_score`, `tamano_letra_score`, `espaciado_score` e `inclinacion_score` depende del análisis de las características del trazo reconocido por el motor OCR elegido.
- **Modelo NLP:** la selección entre un corrector basado en diccionario, distancia de Levenshtein o un modelo estadístico más avanzado [?] queda sujeta a la evaluación de su desempeño con el vocabulario del nivel educativo objetivo.
- **Mecanismo de ejecución diferida:** la decisión de implementar el procesamiento asíncrono mediante un scheduler, una cola de tareas (como Celery) o corutinas de FastAPI queda abierta a la iteración siguiente, donde se evaluará el impacto en el rendimiento del servidor.

Lo que sí queda definido y es inmutable para TT2 son los contratos de datos descritos en la Sección 4.X.2 y el esquema de base de datos ya implementado, que garantizan la integración sin fricciones del módulo una vez desarrollado.

4.3. Arquitectura del Módulo de Captura y Registro de Intentos

El diseño del módulo de captura sigue el patrón arquitectónico MVC establecido para el sistema. Este módulo gestiona el flujo completo desde que el alumno genera una muestra de escritura (ya sea mediante fotografía o lienzo digital) hasta que dicha muestra es almacenada de forma segura y puesta a disposición del tutor a través del panel de seguimiento. La arquitectura de este proceso se divide en tres componentes principales:

1. Capa de Presentación (Front-End): Compuesta por los componentes React (**CapturaEscritura** y **LienzoDigital**). Esta capa es responsable de la validación inicial de los archivos (tipo MIME y tamaño), la renderización del canvas nativo de HTML5 y la conversión de la imagen final a una cadena codificada en formato Base64.
2. Capa de Lógica de Negocio (Back-End): Implementada mediante FastAPI. Recibe las peticiones HTTP con la carga útil (payload) que contiene la imagen codificada.

Se encarga de validar la autenticación del usuario mediante tokens JWT y gestionar las transacciones con la base de datos.

3. Capa de Persistencia (Base de Datos): Basada en SQL Server, utiliza la tabla Intentos para almacenar la información de la actividad. El diseño de la base de datos contempla el almacenamiento de la imagen directamente como una cadena Base64 en el campo imagen_codificada (tipo VARCHAR(MAX)), asegurando la trazabilidad entre el alumno, el ejercicio asignado por el tutor y los resultados posteriores de los algoritmos OCR y NLP.

4.3.1. Diseño de Flujo de Interacción

El registro y consulta de intentos se rige por el siguiente flujo secuencial:

Flujo de Registro de Intento (Alumno):

1. El alumno accede al ejercicio activo y genera su respuesta mediante carga de imagen o trazado en el lienzo digital.
2. El cliente Front-End codifica la muestra en formato Base64.
3. Se envía una petición POST al endpoint protegido del Back-End con los datos: id_alumno, id_ejercicio_tutor e imagen_codificada.
4. El Back-End verifica el token JWT del alumno (dependencia require_alumno).
5. Se inserta un nuevo registro en la tabla Intentos. En este punto, los campos texto_detectado_ocr y métricas asociadas quedan con valores nulos, pendientes del procesamiento posterior diferido de las IA.
6. El sistema retorna una confirmación de éxito al Front-End.

Flujo de Consulta de Intentos (Tutor):

1. El tutor inicia sesión y accede al panel de seguimiento de sus alumnos.
2. El cliente Front-End envía una petición GET al endpoint del Back-End solicitando los intentos vinculados a su identificador.
3. El Back-End verifica el token JWT del tutor (dependencia require_tutor).
4. Mediante una consulta SQL que relaciona las tablas Intentos, Usuario y Ejercicios_Tutor, se extrae el historial de entregas.
5. Los datos (incluyendo la imagen en Base64) son retornados al cliente y renderizados en la vista del profesor para su evaluación manual o revisión de resultados automáticos.

4.3.2. Diseño APIs

Para soportar las operaciones descritas, el diseño de la API contempla la implementación de los siguientes endpoints en el controlador `intentos.py`:

Endpoint 1: Registro de Intento

El endpoint `/api/intentos/registrar` opera mediante el método POST y requiere autenticación obligatoria con rol de alumno a través de un Bearer Token JWT. Su función principal es recibir un payload en formato JSON, compuesto por un `id_ejercicio_tutor` (entero) y una `imagen_codificada` (cadena en Base64), para registrar la información del intento en la base de datos.

Código 4.7: Estructura del payload JSON esperado por el Back-end

```
1 {  
2   "id_ejercicio_tutor": "integer",  
3   "imagen_codificada": "string (Base64)"  
4 }
```

Como parte de su lógica interna de seguridad, el sistema extrae el `id_alumno` directamente del token decodificado para prevenir cualquier suplantación de identidad, y finaliza el proceso ejecutando la sentencia `INSERT INTO Intentos (id_alumno, id_ejercicio_tutor, imagen_codificada) VALUES (...)`.

Endpoint 2: Consulta de Intentos por Tutor

El endpoint `/api/intentos/tutor/id_tutor` utiliza el método GET y exige autenticación mediante un Bearer Token JWT con rol de tutor. Su propósito es recuperar una lista de todos los intentos realizados por los alumnos asignados a un tutor específico. Dentro de su lógica principal, el sistema valida que el `id_tutor` proporcionado en la ruta coincida con el identificador extraído del token; en caso de discrepancia, la petición es rechazada retornando un error 403 Forbidden. Una vez superada esta validación de seguridad, la operación se concreta ejecutando una consulta `SELECT` que relaciona las tablas `Intentos`, `Ejercicios_Tutor` y `Usuario` mediante cláusulas `INNER JOIN`, aplicando el filtro correspondiente al `id_tutor` solicitado para extraer y devolver la información.

4.4. Diseño de la base de datos

4.4.1. Diagrama entidad-relación

El diagrama entidad-relación constituye un elemento fundamental dentro del diseño de la aplicación web, ya que permite estructurar de manera formal la organización, almacenamiento y relación de los datos involucrados en el proceso de captura, procesamiento y

evaluación de la escritura manuscrita.

La aplicación web propuesta gestiona el proceso de aprendizaje y evaluación de la escritura en alumnos de educación primaria. En este contexto, un tutor es responsable de supervisar a múltiples alumnos, asignándoles ejercicios orientados al desarrollo de habilidades de escritura.

El núcleo del sistema se encuentra en la entidad Intento, la cual representa cada ejecución de un ejercicio por parte del alumno. A partir de este intento, se procesa una imagen mediante OCR para obtener el texto digital, el cual es posteriormente analizado mediante NLP para evaluar la ortografía. De forma complementaria, se realiza un análisis caligráfico basado en características del trazo manuscrito.

Los resultados de estos análisis permiten generar recomendaciones personalizadas y actualizar el progreso del alumno, facilitando el seguimiento continuo de su desempeño.

En la Figura ?? se presenta el diagrama entidad-relación, el cual describe la estructura de datos y las relaciones entre las entidades involucradas en el proceso de evaluación de la escritura.

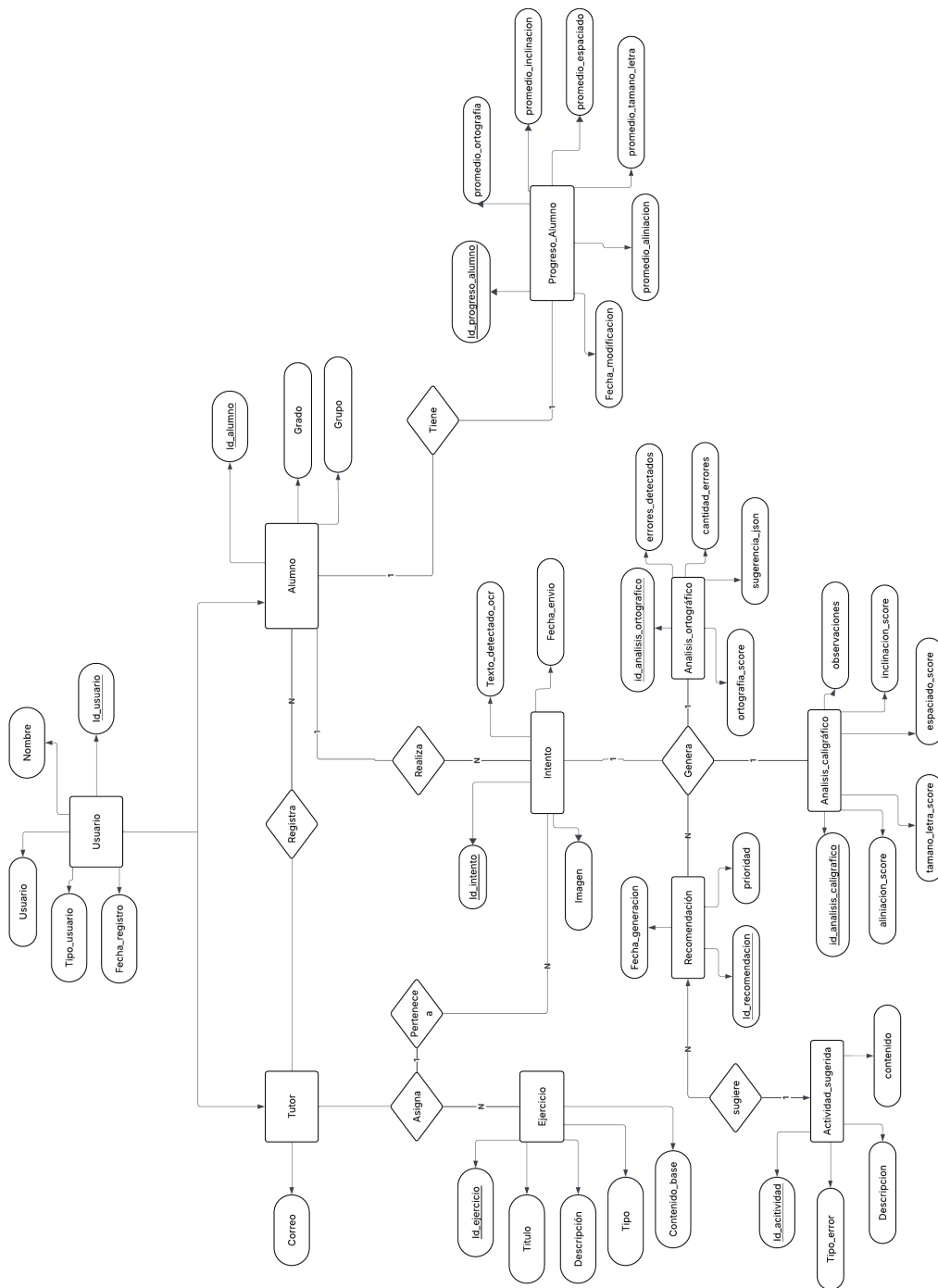


Figura 5: Diagrama entidad-relación propuesto

4.4.2. Diccionario de entidades

A. Gestión de usuarios

En la Tabla ?? se describen las entidades correspondientes a la gestión de usuarios.

Tabla 78: Entidades de gestión de usuarios.

Entidad	Descripción	Atributos clave
Estatus	Entidad que permite indicar el estado de un registro, facilitando su control sin eliminación física.	id_estatus, descripcion
Usuario	Entidad base para el acceso a la aplicación web.	id_usuario, nombre, tipo_usuario, fecha_registro, FK id_estatus
Tutor	Usuario con permisos para registrar alumnos y asignar ejercicios.	correo
Alumno	Usuario final que realiza los ejercicios de escritura.	id_alumno, grado, grupo

Las entidades presentadas permiten gestionar el acceso y la organización de los usuarios dentro del sistema.

B. Proceso de evaluación

En la Tabla ?? se presentan las entidades relacionadas con el proceso de evaluación de la escritura.

Tabla 79: Entidades del proceso de evaluación del sistema

Entidad	Descripción	Atributos clave
Ejercicio	Catalogo de ejercicios.	id_ejercicio, titulo, contenido_base, tipo
Ejercicio_Tutor	Ejercicios asignados por el tutor.	id_ejercicio_tutor, FK id_tutor, FK id_ejercicio, fecha_inicio y fecha_fin, FK id_estatus
Intento	Ejecución de un ejercicio por parte del alumno.	id_intento, FK id_alumno, FK id_ejercicio_tutor, imagen, texto_detectado_ocr, fecha_envio
Analisis_ortografico	Resultado del procesamiento NLP.	id_analisis_ortografico, cantidad_errores, ortografia_score

Entidad	Descripción	Atributos clave
sugerencia_ortografica	Errores detectados del procesamiento NLP, en conjunto con su sugerencia.	id_sugerencia_ortografica, palabra_incorrecta, posicion, sugerencia, contexto
Analisis_caligrafico	Resultado del análisis del trazo manuscrito.	id_analisis_caligrafico, inclinacion_score, espaciado_score, tamaño_letra_score

Estas entidades representan el núcleo del procesamiento de la información dentro de la aplicación web.

C. Seguimiento y retroalimentación

En la Tabla ?? se describen las entidades asociadas al seguimiento y retroalimentación del alumno.

Tabla 80: Entidades del proceso de evaluación del sistema

Entidad	Descripción	Atributos clave
Recomendación	Sugerencia generada automáticamente.	id_recomendación, FK Id_intento, FK Actividad_sugerida, fecha_generación, estado, prioridad.
Actividad_sugerida	Catalogo de ejercicios propuestos para corregir errores.	id_actividad, descripcion, tipo_error, tipo_ejercicio, contenido, FK id_estatus
Progreso_alumno	Indicadores acumulados del desempeño.	id_progreso_alumno, FK id_alumno, promedio_ortografia, promedio_caligrafia

Estas entidades permiten el seguimiento del desempeño y la generación de retroalimentación personalizada.

4.4.3. Análisis de relaciones

El modelo entidad-relación define las interacciones entre las entidades del sistema mediante relaciones con cardinalidades específicas, las cuales permiten representar la forma en que los datos se asocian dentro del proceso de evaluación de la escritura.

A continuación, se describen las principales relaciones identificadas en el modelo:

- **Herencia (Usuario → Tutor / Alumno)**

Se establece una relación de especialización en la cual la entidad Usuario actúa como entidad base, de la que derivan los roles de Tutor y Alumno. Esta estructura permite reutilizar atributos comunes, como nombre y fecha de registro, y asignar funcionalidades específicas según el tipo de usuario.

- **Tutor – Alumno (1:N)**

Un tutor puede supervisar a muchos alumnos, mientras que cada alumno está asociado a un único tutor responsable.

- **Tutor – Ejercicio (M:N)**

Un tutor puede asignar múltiples ejercicios del catálogo, y un mismo ejercicio del catálogo puede ser asignado por múltiples tutores. Esta relación se en la entidad de asociación *Ejercicio_Tutor*.

- **Alumno – Intento (1:N)**

Un alumno puede realizar múltiples intentos a lo largo del tiempo, ya que cada intento representa una ejecución individual de un ejercicio. Esta relación permite almacenar el historial de participación del alumno.

- **Intento – Análisis ortográfico (1:1)**

Cada intento genera exactamente un análisis ortográfico, el cual contiene la evaluación del texto detectado mediante NLP, incluyendo errores y sugerencias.

- **Intento – Análisis caligráfico (1:1)**

Cada intento genera un único análisis caligráfico, el cual evalúa características del trazo manuscrito como alineación, tamaño y espaciado.

- **Intento – Recomendación (1:N)**

Un intento puede generar múltiples recomendaciones, dependiendo de la cantidad y tipo de errores detectados en los análisis realizados.

- **Alumno – Progreso_alumno (1:1)**

Cada alumno cuenta con un único registro en la entidad Progreso_alumno, el cual almacena indicadores acumulados de su desempeño, como promedios de ortografía y caligrafía. Esta relación nos permite obtener un promedio global para el análisis ortográfico y caligráfico, facilitando la calificación de los alumnos.

4.4.4. Diagrama relacional

En la Figura ?? se presenta el diagrama relacional de la aplicación web, donde se definen las tablas, sus atributos, así como las relaciones mediante llaves primarias y foráneas.

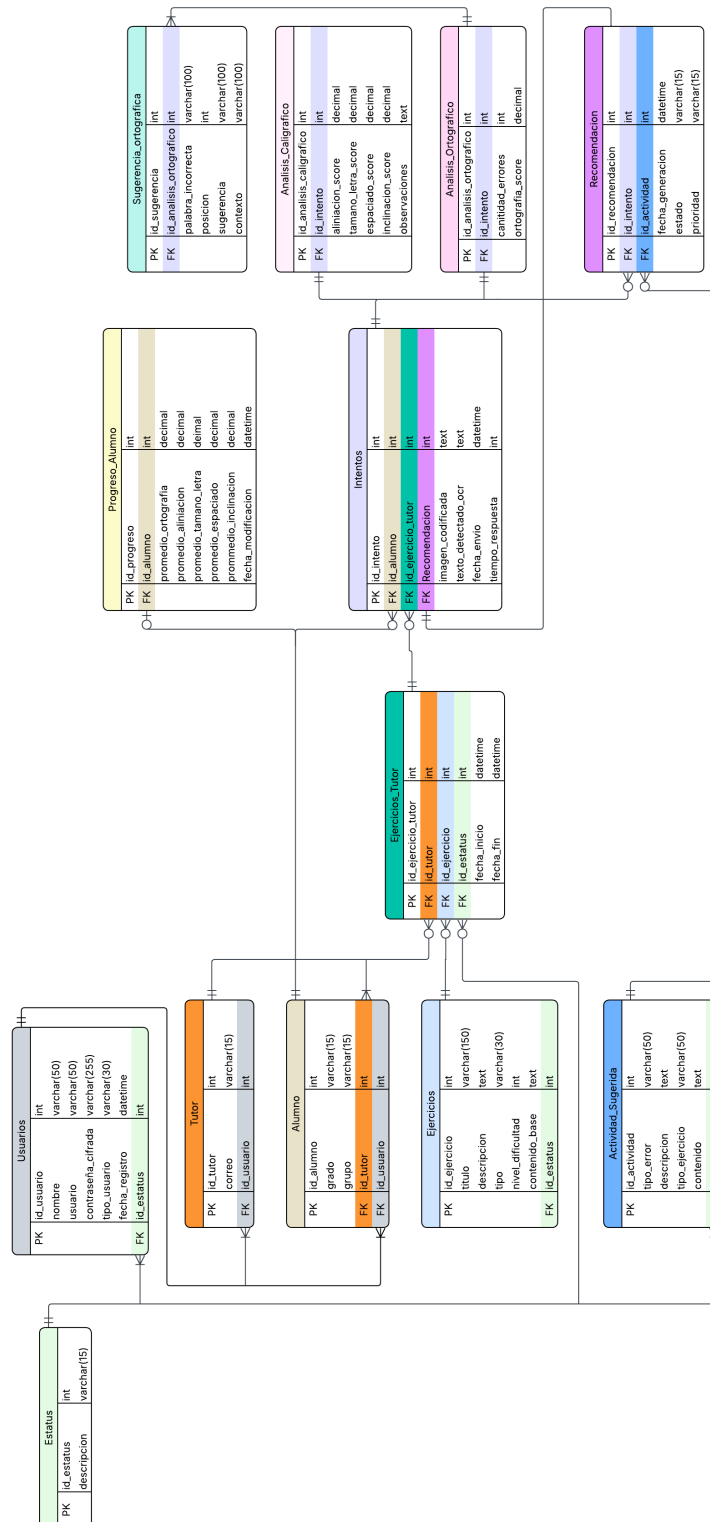


Figura 6: Diagrama relacional propuesto

4.4.5. Decisiones de diseño de base de datos

Gestión del texto detectado (OCR) El atributo `texto_detectado_ocr` en la entidad `Intento` representa un elemento crítico dentro de la aplicación web, ya que corresponde a la salida generada por el módulo de reconocimiento óptico de caracteres (OCR). Este dato constituye la entrada principal para el procesamiento mediante técnicas de procesamiento de lenguaje natural (NLP), por lo que su correcta generación y almacenamiento es fundamental para garantizar la calidad del análisis ortográfico.

El uso de este formato proporciona flexibilidad y evita la rigidez de un esquema relacional estrictamente normalizado, facilitando futuras ampliaciones del sistema.

Cálculo del progreso del alumno La entidad `Progreso_alumno` funciona como una tabla de agregación que almacena indicadores globales del desempeño del estudiante. Su actualización se realiza mediante triggers a nivel de base de datos, los cuales recalculan automáticamente los promedios de ortografía y caligrafía al registrar un nuevo intento.

Este enfoque se justifica por las siguientes razones:

- **Atomicidad:** el trigger se ejecuta dentro de la misma transacción que el `INSERT` del intento, garantizando que `Progreso_alumno` siempre refleje el estado actualizado sin depender de una llamada adicional desde el controlador.
- **Consistencia:** se elimina el riesgo de inconsistencia de datos ante un fallo del backend posterior a la inserción del intento pero previo a la actualización del progreso.
- **Separación de responsabilidades:** el controlador no requiere conocer la lógica de cálculo de promedios, delegando esa responsabilidad a la base de datos, lo cual es coherente con el patrón de diseño MVC adoptado en el sistema.

Gestión del historial y trazabilidad La vista `vw_historial_alumno` permite mantener un registro cronológico de los resultados obtenidos por el alumno en cada intento, sin comprometer la normalización del modelo de datos. A diferencia de una tabla adicional, la vista consolida la información existente en las entidades `Intento`, `Analisis_ortografico` y `Analisis_caligrafico` mediante una consulta predefinida, evitando la redundancia de datos.

4.5. Arquitectura del Módulo de Gestión de Ejercicios

El módulo de gestión de ejercicios administra el ciclo de vida completo de las actividades de escritura dentro de la plataforma. Su diseño contempla dos actores con responsabilidades distintas: el tutor, que selecciona ejercicios del catálogo y los asigna a sus alumnos con una vigencia determinada; y el alumno, que consulta los ejercicios disponibles clasificados por

su estado. El módulo también incorpora un proceso automatizado que cierra los ejercicios cuya fecha de vencimiento ha sido alcanzada, garantizando la coherencia del estado del sistema sin intervención manual.

La arquitectura del módulo se divide en tres componentes principales:

1. **Capa de Presentación (Front-End):** Compuesta por los componentes `TabEjercicios.jsx` (vista del tutor para gestionar asignaciones), `TabProximos.jsx`, `TabCompletados.jsx` y `TabVencidos.jsx` (vistas del alumno que clasifican los ejercicios según su estado). Cada componente consulta su endpoint correspondiente y renderiza la información sin lógica de negocio propia.
2. **Capa de Lógica de Negocio (Back-End):** Implementada en el controlador `ejercicios.py` mediante FastAPI. Gestiona la consulta del catálogo, la activación y desactivación de ejercicios por parte del tutor, y la consulta de ejercicios en sus tres estados posibles desde la perspectiva del alumno. Adicionalmente, el proceso `cerrar_ejercicio.py` implementa un scheduler que verifica y cierra automáticamente los ejercicios vencidos cada hora.
3. **Capa de Persistencia (Base de Datos):** El módulo opera sobre dos tablas principales: `Ejercicios`, que actúa como catálogo inmutable de actividades disponibles en la plataforma, y `Ejercicios_Tutor`, que representa la asignación de un ejercicio del catálogo a un tutor específico, con su propio ciclo de vida controlado por el campo `id_estatus` y la fecha `fecha_desactivacion`.

4.5.1. *Diseño del Ciclo de Vida de un Ejercicio*

Un ejercicio atraviesa los siguientes estados a lo largo de su ciclo de vida dentro del sistema:

1. **Disponible en catálogo:** el ejercicio existe en la tabla `Ejercicios` con `id_estatus = 1` y puede ser seleccionado por cualquier tutor para su asignación.
2. **Activo:** el tutor ha creado un registro en `Ejercicios_Tutor` con `id_estatus = 1`. A partir de este momento, el ejercicio es visible para los alumnos del tutor en su panel de ejercicios próximos.
3. **Vencido:** el ejercicio transiciona a `id_estatus = 2` por uno de dos mecanismos: desactivación manual por parte del tutor mediante el endpoint de eliminación, o cierre automático por el scheduler cuando la fecha `fecha_desactivacion` es anterior a la fecha y hora actuales del servidor. Un ejercicio vencido deja de aparecer en la lista de próximos del alumno y pasa a la lista de vencidos si no fue entregado, o permanece en completados si el alumno ya realizó un intento.

Nota de diseño: el sistema no elimina físicamente los registros de `Ejercicios_Tutor`. La transición entre estados se gestiona exclusivamente mediante el campo `id_estatus`, lo que preserva la trazabilidad histórica de las asignaciones y permite al tutor consultar ejercicios previamente activos sin pérdida de información.

4.5.2. *Diseño del Flujo de Interacción*

El módulo contempla tres flujos principales de interacción.

Flujo de Asignación de Ejercicio (Tutor):

1. El tutor accede a su panel y solicita el catálogo de ejercicios disponibles mediante una petición GET al endpoint público `/api/ejercicios/`.
2. El tutor selecciona un ejercicio y establece opcionalmente una fecha de vencimiento. El Front-End envía una petición POST al endpoint `/api/ejercicios/tutor` con el identificador del ejercicio y la fecha de cierre.
3. El Back-End verifica mediante JWT que el `id_usuario` del token coincide con el `id_usuario` del payload, evitando que un tutor active ejercicios en nombre de otro.
4. Se verifica que el tutor no tenga ya activo el mismo ejercicio. En caso de duplicado, se retorna 400 Bad Request.
5. Se inserta un nuevo registro en `Ejercicios_Tutor` con `id_estatus = 1`. El ejercicio queda inmediatamente disponible para los alumnos del tutor.

Flujo de Consulta de Ejercicios (Alumno):

Desde la perspectiva del alumno, los ejercicios se presentan clasificados en tres pestañas. La lógica de clasificación reside íntegramente en el Back-End mediante tres consultas SQL independientes:

- **Próximos:** ejercicios activos (`id_estatus = 1`) asignados por el tutor del alumno que aún no tienen un intento registrado por ese alumno. Se excluyen mediante una subconsulta `NOT IN` sobre la tabla `Intentos`.
- **Completados:** ejercicios que tienen al menos un intento registrado por el alumno. Se retorna únicamente el intento más reciente por ejercicio, determinado mediante la función de ventana `ROW_NUMBER() OVER (PARTITION BY id_ejercicio_tutor ORDER BY fecha_envio DESC)`, incluyendo la calificación y retroalimentación del tutor si ya fueron registradas.
- **Vencidos:** ejercicios con `id_estatus = 2` que no tienen ningún intento registrado por el alumno, es decir, los que el alumno no alcanzó a entregar antes del cierre.

Flujo de Cierre Automático (Scheduler):

1. Al iniciar el servidor, el evento `startup` de FastAPI invoca `iniciar_scheduler()`, que registra una tarea periódica con una frecuencia de ejecución de una hora mediante la biblioteca APScheduler.
2. Cada hora, la tarea `cerrar_ejercicios_vencidos()` ejecuta un `UPDATE` sobre la tabla `Ejercicios_Tutor`, transitando a `id_estatus = 2` todos los registros cuya `fecha_desactivacion` sea distinta de nulo y anterior a la fecha y hora actuales del servidor (`GETDATE()`).
3. La operación es idempotente: ejecutarla múltiples veces sobre los mismos datos no produce efectos secundarios, ya que la condición `id_estatus = 1` del filtro impide modificar registros ya cerrados.

4.5.3. Diseño de APIs

Para soportar los flujos descritos, el diseño de la API contempla los siguientes endpoints en el controlador `ejercicios.py`:

Endpoint 1: Consulta del Catálogo

El endpoint `/api/ejercicios/` opera mediante el método `GET` y no requiere autenticación. Retorna todos los ejercicios con `id_estatus = 1` disponibles en la tabla `Ejercicios`.

Endpoint 2: Consulta de Ejercicios Activos del Tutor

El endpoint `/api/ejercicios/tutor/{id_usuario}` opera mediante el método `GET` y requiere autenticación JWT con rol de tutor. Retorna los ejercicios que el tutor ha activado y que se encuentran actualmente con `id_estatus = 1`.

Endpoint 3: Activación de Ejercicio

El endpoint `/api/ejercicios/tutor` opera mediante el método `POST` y requiere autenticación JWT con rol de tutor. Recibe el identificador del ejercicio del catálogo y una fecha de vencimiento opcional.

Código 4.8: Payload de solicitud del endpoint de activación de ejercicio

```
1 /* Solicitud */
2 {
3   "id_ejercicio": <integer>,
4   "id_usuario":   <integer>,
5   "fecha_fin":    "datetime | null"
6 }
7
8 /* Respuesta 201 Created */
```

```

9 {
10   "message": "Ejercicio activado correctamente"
11 }

```

Endpoint 4: Desactivación Manual de Ejercicio

El endpoint `/api/ejercicios/tutor/{id_usuario}/{id_ejercicio}` opera mediante el método `DELETE` y requiere autenticación JWT con rol de tutor. Actualiza el campo `id_estatus` a 2 en el registro correspondiente de `Ejercicios_Tutor`, sin eliminar el registro físicamente.

Endpoints 5, 6 y 7: Consulta de Ejercicios del Alumno

Los endpoints `/api/ejercicios/alumno/{id_alumno}/proximos`, `/api/ejercicios/alumno/{id_alumno}/completados` y `/api/ejercicios/alumno/{id_alumno}/vencidos` operan mediante el método `GET` y requieren autenticación JWT con rol de alumno. En los tres casos, el Back-End verifica que el `id_alumno` de la ruta coincida con el `id_usuario` del token, retornando `403 Forbidden` en caso de discrepancia.

La Tabla ?? resume los endpoints del módulo con su método y nivel de acceso correspondiente.

Tabla 81: Endpoints del módulo de gestión de ejercicios

Endpoint	Método	Acceso requerido
<code>/api/ejercicios/</code>	GET	Público
<code>/api/ejercicios/tutor/{id_usuario}</code>	GET	Tutor (JWT)
<code>/api/ejercicios/tutor</code>	POST	Tutor (JWT)
<code>/api/ejercicios/tutor/{id_usuario}/{id_ejercicio}</code>	DELETE	Tutor (JWT)
<code>/api/ejercicios/alumno/{id_alumno}/proximos</code>	GET	Alumno (JWT)
<code>/api/ejercicios/alumno/{id_alumno}/completados</code>	GET	Alumno (JWT)
<code>/api/ejercicios/alumno/{id_alumno}/vencidos</code>	GET	Alumno (JWT)

5 Capítulo 5: Desarrollo

En este capítulo se presenta la fase de desarrollo del sistema, en la cual se lleva a cabo la implementación de la arquitectura y el diseño previamente definidos en la etapa de análisis. A partir de los requerimientos funcionales, no funcionales y las reglas de negocio establecidas en el capítulo anterior, se detalla la construcción de los distintos módulos que conforman la plataforma.

5.1. Desarrollo de la base de datos

El desarrollo de la base de datos presentado en esta sección se basa en el diagrama relacional y decisiones de diseño de la base de datos. Se llevó a cabo la implementación en el sistema gestor de bases de datos SQL Server, traduciendo cada entidad, atributo y relación en estructuras físicas mediante sentencias T-SQL.

5.1.1. Creación del esquema

Durante esta etapa, se implementaron restricciones de integridad y definición de claves primarias y foráneas, con el objetivo de garantizar la consistencia, eficiencia y correcto funcionamiento de la aplicación web.

A. Gestión de usuarios

Este módulo está compuesto por las tablas **Usuarios**, **Tutor** y **Alumno**, las cuales representan la estructura principal de acceso y perfiles del sistema. La tabla **Estatus** actúa como catálogo compartido y debe crearse previamente, ya que es referenciada por **Usuarios** mediante una clave foránea.

Código 5.9: Implementacion de entidades de usuario en la base de datos

```
-- Estatus (catalogo compartido, debe crearse primero)
IF OBJECT_ID('Estatus', 'U') IS NULL
CREATE TABLE Estatus (
    id_estatus      INT          IDENTITY(1,1) NOT NULL,
    descripcion     VARCHAR(30)  NOT NULL,
    CONSTRAINT PK_Estatus PRIMARY KEY (id_estatus),
    CONSTRAINT UQ_Estatus_descripcion UNIQUE (descripcion)
);
GO

-- Usuarios
IF OBJECT_ID('Usuarios', 'U') IS NULL
```

```

CREATE TABLE Usuarios (
    id_usuario          INT          IDENTITY(1,1) NOT NULL,
    nombre              VARCHAR(50)  NOT NULL,
    usuario             VARCHAR(50)  NOT NULL,
    contrasena_cifrada  VARCHAR(255) NOT NULL,
    tipo_usuario        VARCHAR(30)  NOT NULL,
    fecha_registro      DATETIME     NOT NULL DEFAULT GETDATE(),
    id_estatus          INT          NOT NULL,
    CONSTRAINT PK_Usuarios PRIMARY KEY (id_usuario),
    CONSTRAINT UQ_Usuarios_usuario UNIQUE (usuario),
    CONSTRAINT FK_Usuarios_Estatus
        FOREIGN KEY (id_estatus)
        REFERENCES Estatus(id_estatus)
);
GO

-- Tutor
IF OBJECT_ID('Tutor', 'U') IS NULL
CREATE TABLE Tutor (
    id_tutor           INT          NOT NULL,
    correo             VARCHAR(100) NOT NULL,
    id_usuario         INT          NOT NULL,
    CONSTRAINT PK_Tutor PRIMARY KEY (id_tutor),
    CONSTRAINT UQ_Tutor_correo UNIQUE (correo),
    -- id_tutor hereda el id_usuario de la tabla Usuarios
    CONSTRAINT FK_Tutor_Usuario
        FOREIGN KEY (id_usuario)
        REFERENCES Usuarios(id_usuario)
);
GO

-- Alumno
IF OBJECT_ID('Alumno', 'U') IS NULL
CREATE TABLE Alumno (
    id_alumno          INT          IDENTITY(1,1) NOT NULL,
    grado              VARCHAR(15)  NOT NULL,
    grupo              VARCHAR(15)  NOT NULL,
    id_tutor           INT          NOT NULL,
    id_usuario         INT          NOT NULL,
    CONSTRAINT PK_Alumno PRIMARY KEY (id_alumno),
    CONSTRAINT FK_Alumno_Tutor

```

```

        FOREIGN KEY (id_tutor)
        REFERENCES Tutor(id_tutor),
    CONSTRAINT FK_Alumno_Usuario
        FOREIGN KEY (id_usuario)
        REFERENCES Usuarios(id_usuario)
);
GO

```

B. Proceso de evaluación

Este módulo agrupa las tablas centrales del flujo de evaluación: **Ejercicio_Tutor**, **Intento** y **Analisis_ortografico**. La tabla **Ejercicio_Tutor** materializa la relación M:N entre tutores y ejercicios del catálogo, incorporando atributos propios de la asignación. La tabla **Intento** registra cada ejecución de un ejercicio por parte del alumno, almacenando la imagen capturada y el texto reconocido por el módulo OCR. La tabla **Analisis_ortografico** almacena los resultados del procesamiento NLP, utilizando un campo JSON para representar las sugerencias de corrección de forma flexible.

Código 5.10: Implementación de entidades del proceso de evaluación

```

-- Ejercicio_Tutor (tabla de asociaci n M:N con atributos propios)
IF OBJECT_ID('Ejercicio_Tutor', 'U') IS NULL
CREATE TABLE Ejercicio_Tutor (
    id_ejercicio_tutor    INT            IDENTITY(1,1) NOT NULL,
    id_tutor              INT            NOT NULL,
    id_ejercicio          INT            NOT NULL,
    fecha_asignacion      DATETIME       NOT NULL DEFAULT GETDATE(),
    fecha_limite          DATETIME       NULL,
    id_estatus            INT            NOT NULL,
    CONSTRAINT PK_Ejercicio_Tutor PRIMARY KEY (id_ejercicio_tutor),
    CONSTRAINT FK_EjercicioTutor_Tutor
        FOREIGN KEY (id_tutor)
        REFERENCES Tutor(id_tutor),
    CONSTRAINT FK_EjercicioTutor_Ejercicio
        FOREIGN KEY (id_ejercicio)
        REFERENCES Ejercicio(id_ejercicio),
    CONSTRAINT FK_EjercicioTutor_Estatus
        FOREIGN KEY (id_estatus)
        REFERENCES Estatus(id_estatus)
);
GO

-- Intentos

```

```

IF OBJECT_ID('Intentos', 'U') IS NULL
CREATE TABLE Intentos (
    id_intento          INT          IDENTITY(1,1) NOT NULL,
    id_alumno           INT          NOT NULL,
    id_ejercicio_tutor  INT          NOT NULL,
    imagen_codificada   VARCHAR(MAX) NOT NULL,
    texto_detectado_ocr VARCHAR(MAX)  NULL,
    fecha_envio         DATETIME2    NOT NULL DEFAULT GETDATE(),
    retroalimentacion   VARCHAR(MAX)  NULL,
    CONSTRAINT PK_Intentos          PRIMARY KEY (id_intento),
    CONSTRAINT FK_Intentos_alumno   FOREIGN KEY (id_alumno)
        REFERENCES Alumno(id_alumno),
    CONSTRAINT FK_Intentos_ET       FOREIGN KEY (id_ejercicio_tutor
        ) REFERENCES Ejercicios_Tutor(id_ejercicio_tutor)
);
GO

-- An lisis ortogr fico
IF OBJECT_ID('Analisis_Ortografico', 'U') IS NULL
CREATE TABLE Analisis_Ortografico (
    id_analisis_ortografico INT          NOT NULL IDENTITY(1,1),
    id_intento              INT          NOT NULL,
    cantidad_errores        INT          NOT NULL DEFAULT 0,
    sugerencias_json         NVARCHAR(MAX)  NULL,
    ortografia_score         DECIMAL(5,2)  NULL,
    CONSTRAINT PK_Analisis_Ortografico PRIMARY KEY (
        id_analisis_ortografico),
    CONSTRAINT UQ_Analisis_Ort_intento UNIQUE      (id_intento),
    CONSTRAINT FK_Analisis_Ort_intento FOREIGN KEY (id_intento)
        REFERENCES Intentos(id_intento),
    CONSTRAINT CK_sugerencias_json CHECK (sugerencias_json IS
        NULL OR ISJSON(sugerencias_json) = 1)
);
GO

```

5.1.2. Implementación de restricciones e integridad referencial

Con el objetivo de garantizar la consistencia y calidad de los datos almacenados en la aplicación web, se definieron restricciones de integridad a nivel de base de datos. Estas restricciones operan de forma independiente a la lógica del backend, asegurando que

ninguna operación pueda comprometer la coherencia del modelo de datos.

A continuación se describen las restricciones implementadas agrupadas por categoría.

Unicidad de campos críticos Se establecieron restricciones **UNIQUE** en los campos que deben identificar de forma exclusiva a cada registro dentro del sistema, evitando duplicados que comprometan la integridad de la información.

Código 5.11: Restricciones de unicidad en campos críticos

```
-- Unicidad de nombre de usuario en Usuarios
CONSTRAINT UQ_Usuarios_usuario
    UNIQUE (usuario),

-- Unicidad de correo electrónico en Tutor
CONSTRAINT UQ_Tutor_correo
    UNIQUE (correo),

-- Un alumno solo puede tener un intento por ejercicio asignado
CREATE UNIQUE INDEX UQ_Intentos_original
ON Intentos (id_alumno, id_ejercicio_tutor)
WHERE id_recomendacion IS NULL;

-- Un tutor no puede asignar el mismo ejercicio si ya tiene una
  asignación activa.
CREATE UNIQUE INDEX UQ_ET_activo
    ON Ejercicio_Tutor (id_tutor, id_ejercicio)
    WHERE id_estatus = 1;

-- Relación 1:1 entre Intento y Análisis ortográfico
CONSTRAINT UQ_Analisis_Ort_intento
    UNIQUE (id_intento),

-- Relación 1:1 entre Intento y Análisis caligráfico
CONSTRAINT UQ_Analisis_Cal_intento
    UNIQUE (id_intento)
```

Validación de rangos en scores Los atributos de puntuación en las tablas **Analisis_Ortografico** y **Analisis_Caligrafico** deben estar acotados en un rango de 0 a 10, representando la escala de evaluación del sistema. Se implementaron restricciones **CHECK** para garantizar que ningún valor fuera de este rango pueda ser insertado o actualizado.

Código 5.12: Restricciones de rango en atributos de puntuación

```
-- Scores en Analisis_Ortografico
CONSTRAINT CK_ortografia_score
    CHECK (ortografia_score IS NULL
        OR (ortografia_score >= 0 AND ortografia_score <= 10)),

-- Scores en Analisis_Caligrafico
CONSTRAINT CK_alineacion_score
    CHECK (alineacion_score IS NULL
        OR (alineacion_score >= 0 AND alineacion_score <= 10)),

CONSTRAINT CK_tamano_letra_score
    CHECK (tamano_letra_score IS NULL
        OR (tamano_letra_score >= 0 AND tamano_letra_score <= 10)),

CONSTRAINT CK_espaciado_score
    CHECK (espaciado_score IS NULL
        OR (espaciado_score >= 0 AND espaciado_score <= 10)),

CONSTRAINT CK_inclinacion_score
    CHECK (inclinacion_score IS NULL
        OR (inclinacion_score >= 0 AND inclinacion_score <= 10))
```

Integridad referencial Se definieron claves foráneas en todas las entidades que mantienen relaciones entre sí, garantizando que no puedan existir registros huérfanos en el sistema. La siguiente tabla resume las relaciones de integridad referencial implementadas.

Tabla 82: Relaciones de integridad referencial del sistema

Tabla origen	Tabla destino	Cardinalidad
Usuarios	Estatus	N:1
Tutor	Usuarios	1:1
Alumno	Usuarios	1:1
Alumno	Tutor	N:1
Ejercicio_Tutor	Tutor	N:1
Ejercicio_Tutor	Ejercicio	N:1
Ejercicio_Tutor	Estatus	N:1
Intentos	Alumno	N:1
Intentos	Ejercicio_Tutor	N:1
Analisis_Ortografico	Intentos	1:1

Tabla origen	Tabla destino	Cardinalidad
Sugerencia_ortografica	Analisis_Ortografico	N:1
Analisis_Caligrafico	Intentos	1:1

Obligatoriedad de campos Se establecieron restricciones NOT NULL en todos los atributos que son indispensables para el funcionamiento del sistema, garantizando que ningún registro pueda ser insertado sin la información mínima requerida. Los campos marcados como NULL corresponden a información opcional o generada de forma asíncrona, como el texto detectado por OCR o los scores de análisis, los cuales se completan una vez procesado el intento.

5.1.3. Implementación de la vista vw_historial_alumno

La vista vw_historial_alumno se creó con el objetivo de consolidar en una única consulta predefinida la información distribuida entre las tablas *Intentos*, *Analisis_Ortografico*, *Analisis_Caligrafico*, *Alumno* y *Ejercicio*. Su implementación responde a tres necesidades concretas del sistema:

- **Rendimiento:** evita la ejecución repetitiva de consultas con múltiples JOIN cada vez que el tutor consulta el historial de un alumno (RF-09) o el alumno revisa sus ejercicios completados (RF-16).
- **Separación de responsabilidades:** el controlador accede al historial mediante una consulta simple sobre la vista, sin necesidad de conocer la estructura interna de las tablas involucradas, lo cual es coherente con el patrón MVC adoptado.
- **Normalización:** permite mantener el modelo de datos normalizado sin sacrificar la facilidad de consulta, ya que la vista actúa como capa de abstracción sobre las relaciones existentes sin almacenar datos redundantes.

Se utilizaron LEFT JOIN en las tablas de análisis para contemplar el caso en que un intento recién enviado aún no haya sido procesado por los módulos OCR y NLP, mostrando NULL en los scores mientras el análisis está pendiente.

Código 5.13: Implementación de la vista vw_historial_alumno

```
CREATE VIEW vw_historial_alumno AS
SELECT
    -- Alumno
    a.id_alumno,
    u.nombre                                AS nombre_alumno,
    -- Ejercicio
    e.id_ejercicio,
```

```

e.titulo                AS titulo_ejercicio,
e.tipo                  AS tipo_ejercicio,
-- Intento
i.id_intento,
i.fecha_envio,
i.retroalimentacion,
CASE
    WHEN i.id_recomendacion IS NULL
    THEN 'Original'
    ELSE 'Reintento'
END                      AS tipo_intento,
-- Análisis ortográfico
ao.ortografia_score,
ao.cantidad_errores,
-- Análisis caligráfico
ac.alineacion_score,
ac.tamano_letra_score,
ac.espaciado_score,
ac.inclinacion_score
FROM Intentos i
INNER JOIN Alumno a
    ON a.id_alumno = i.id_alumno
INNER JOIN Usuarios u
    ON u.id_usuario = a.id_usuario
INNER JOIN Ejercicios_Tutor et
    ON et.id_ejercicio_tutor = i.id_ejercicio_tutor
INNER JOIN Ejercicio e
    ON e.id_ejercicio = et.id_ejercicio
LEFT JOIN Analisis_Ortografico ao
    ON ao.id_intento = i.id_intento
LEFT JOIN Analisis_Caligrafico ac
    ON ac.id_intento = i.id_intento;
GO

```

Una vez creada la vista, el historial completo de un alumno se obtiene desde el controlador mediante una consulta directa, filtrando por `id_alumno` y ordenando por fecha de envío descendente, sin necesidad de replicar la lógica de JOIN en cada petición.

Código 5.14: Consulta del historial de un alumno mediante la vista

```

SELECT *
FROM vw_historial_alumno
WHERE id_alumno = @id_alumno

```

```
ORDER BY fecha_envio DESC;
```

5.1.4. Implementación de triggers

Los triggers implementados tienen como objetivo mantener actualizada la tabla **Progreso_Alumno** de forma automática cada vez que se registra un nuevo análisis, sin requerir intervención del controlador. Se definieron dos triggers independientes: uno sobre **Analisis_Ortografico** y otro sobre **Analisis_Caligrafico**, cada uno responsable de actualizar los promedios correspondientes a su dimensión de evaluación.

Ambos triggers manejan dos escenarios: la creación del primer registro de progreso del alumno y la actualización de un registro existente, garantizando la consistencia de los datos en cualquier situación.

Trigger sobre Analisis_Ortografico

Código 5.15: Trigger de actualización del promedio ortográfico

```
CREATE TRIGGER trg_actualizar_progreso_ortografia
ON Analisis_Ortografico
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @id_alumno INT;
    DECLARE @nuevo_promedio DECIMAL(5,2);

    -- Obtener el alumno del intento recién analizado
    SELECT @id_alumno = i.id_alumno
    FROM inserted ins
    INNER JOIN Intentos i ON i.id_intento = ins.id_intento;

    -- Calcular el nuevo promedio ortográfico del alumno
    SELECT @nuevo_promedio = AVG(ao.ortografia_score)
    FROM Analisis_Ortografico ao
    INNER JOIN Intentos i ON i.id_intento = ao.id_intento
    WHERE i.id_alumno = @id_alumno
    AND ao.ortografia_score IS NOT NULL;

    -- Insertar o actualizar según si ya existe el registro
    IF EXISTS (
```

```

        SELECT 1 FROM Progreso_Alumno
        WHERE id_alumno = @id_alumno
    )
BEGIN
    UPDATE Progreso_Alumno
    SET promedio_ortografia = @nuevo_promedio,
        fecha_modificacion = GETDATE()
    WHERE id_alumno = @id_alumno;
END
ELSE
BEGIN
    INSERT INTO Progreso_Alumno (
        id_alumno,
        promedio_ortografia,
        promedio_aliniacion,
        promedio_tamano_letra,
        promedio_espaciado,
        promedio_inclinacion,
        fecha_modificacion
    )
    VALUES (
        @id_alumno,
        @nuevo_promedio,
        NULL, NULL, NULL, NULL,
        GETDATE()
    );
END
END;
GO

```

Trigger sobre Analisis_Caligrafico

Código 5.16: Trigger de actualización de promedios caligráficos

```

CREATE TRIGGER trg_actualizar_progreso_caligrafico
ON Analisis_Caligrafico
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

```

```

DECLARE @id_alumno          INT;
DECLARE @prom_alineacion    DECIMAL(5,2);
DECLARE @prom_tamano_letra  DECIMAL(5,2);
DECLARE @prom_espaciado     DECIMAL(5,2);
DECLARE @prom_inclinacion   DECIMAL(5,2);

-- Obtener el alumno del intento recién analizado
SELECT @id_alumno = i.id_alumno
FROM inserted ins
INNER JOIN Intentos i ON i.id_intento = ins.id_intento;

-- Calcular los nuevos promedios caligráficos del alumno
SELECT
    @prom_alineacion      = AVG(ac.alineacion_score),
    @prom_tamano_letra    = AVG(ac.tamano_letra_score),
    @prom_espaciado       = AVG(ac.espaciado_score),
    @prom_inclinacion     = AVG(ac.inclinacion_score)
FROM Analisis_Caligrafico ac
INNER JOIN Intentos i ON i.id_intento = ac.id_intento
WHERE i.id_alumno = @id_alumno
AND ac.alineacion_score IS NOT NULL
AND ac.tamano_letra_score IS NOT NULL
AND ac.espaciado_score IS NOT NULL
AND ac.inclinacion_score IS NOT NULL;

-- Insertar o actualizar según si ya existe el registro
IF EXISTS (
    SELECT 1 FROM Progreso_Alumno
    WHERE id_alumno = @id_alumno
)
BEGIN
    UPDATE Progreso_Alumno
    SET promedio_aliniacion      = @prom_alineacion,
        promedio_tamano_letra    = @prom_tamano_letra,
        promedio_espaciado       = @prom_espaciado,
        promedio_inclinacion     = @prom_inclinacion,
        fecha_modificacion       = GETDATE()
    WHERE id_alumno = @id_alumno;
END
ELSE
BEGIN

```

```

INSERT INTO Progreso_Alumno (
    id_alumno,
    promedio_ortografia,
    promedio_aliniacion,
    promedio_tamano_letra,
    promedio_espaciado,
    promedio_inclinacion,
    fecha_modificacion
)
VALUES (
    @id_alumno,
    NULL,
    @prom_alineacion,
    @prom_tamano_letra,
    @prom_espaciado,
    @prom_inclinacion,
    GETDATE()
);

END
END;
GO

```